
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 21:26:51
Файлов исходного кода: 111
Суммарный объём кода: 1.98 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planar-coloring]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends
          # На Ubuntu политика ImageMagick иногда запрещает
          # разрешаем то, что нужно пайплайну (text -> pdf)
          for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```



```
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не выполнила");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin]);
```

```

        <span className="text-[11px] text-slate
    </div>
    <div className="flex items-center gap-2">
        <button
            type="button"
            onClick={reset}
            className="inline-flex items-center gap-2 text-slate-800 transition-colors"
        >
            <RotateCcw className="w-3.5 h-3.5" />
        </button>
        <button
            type="button"
            onClick={() => void runCode()}
            disabled={running}
            className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
        >
            {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
        </button>
    </div>
</div>

<textarea
    value={code}
    onChange={(event) => setCode(event.target.value)}
    onKeyDown={(event) => {
        if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
            event.preventDefault();
            void runCode();
        }
    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
    placeholder="Напишите Python-код..."

```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```
-----
+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...
```

`vite.config.ts`

Полное содержимое после изменений

```
````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},
```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, strictly required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, описанных ниже.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-теги.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сухари»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни интерактива
8. Графы. Компоненты связности	`graph-components`	Нет визуализации
24. Классы сложности, сведение задач	`complexity-classes`	Нет визуализации

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.in
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 {/* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}` }
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 {/* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className=
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl fon
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
))
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок покрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, покрывающий
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; vertical-align: middle;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; vertical-align: middle;">ум на его части не вычислишь. Приходится хранить матрицу.  
<span class="font-mono text-emerald-300">log N × log M</span>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: сложение матриц – перекрывающиеся блоки посчитают несколько раз.  
</b> – у минимума нет обратной операции. Стрелка

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">

<h3 class="text-xl font-bold text-amber-400">Проблема разреженных матриц

<div class="bg-rose-950/30 p-5 rounded-lg border-rose-500/50">

<ul class="list-disc pl-5 space-y-2 text-slate-300">

<li><b>Сумма через разреженную таблицу – O(N × M).</b> Префиксные (билет 5).</li>

<li><b>Память.</b> N = M = 1000 даёт 10<sup>6</sup> элементов. Считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> В разреженной матрице.  
<span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i][j-1]</span></li>

<li><b>Выход за границу в запросе.</b> В разреженной матрице.  
<span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i][j-1]</span> и <span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i-1][j]</span>

<li><b>Нужны обновления.</b> Разреженная матрица (билет 1), а не она.</li>

</ul>

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

Вторая половина этого сравнения:

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — левый
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)</pre>
 <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
 if key < root.key: root.left = erase(root.left, key)
 elif key > root.key: root.right = erase(root.right, key)
 else: root = merge(root.left, root.right)
 return root</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 }

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
x, и под p.</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \ x выше / \ \
 x y -----> α p
 / \ \ p выше / \ \
 α β <----- β γ</pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + s
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Тонкая: одна матрица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение.
 прямоугольника от угла. Всё остальное получается так же.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычислить.
 </div>
 <p class="text-slate-300 text-sm mb-2">
 максимума или gcd вычитания не существует:
 е запросы – это
 платит она за это памятью
 </div>
 </div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
 - часть демонстрации: лента подсвечивает, из
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф планарный, то он можно вложить в плоскость так,
 чтобы никакие две его вершины не оказались бы в одной точке.
 2. Если граф планарный, то он можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке, поэтому
 можно раскрасить карты, чтобы соседние страны
 не сливались цветами, Всегда можно всего 4-х цветов.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">В strongly connected component (SCC) графа — это подмножество вершин, в котором каждая вершина достижима из каждой другой вершины в том же подмножестве. Другими словами, в SCC каждая вершина является частью цикла.
 <p class="text-xl font-mono text-white">Граф можно разбить на компоненты сильной связности, инвертированные в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где все улицы имеют одностороннее движение. Если вы можете добраться из точки А в точку Б по односторонним улицам, и наоборот, то А и Б находятся в одной "компоненте сильной связности".
 <p class="text-slate-300 text-sm mb-4">ВЫХ направлениях по односторонним улицам. Как это выглядит на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: класс P (простые задачи) vs класс NP (нестрогие задачи).</p>
 <p class="text-base sm:text-lg font-monospace">
 P: класс decision-задач, решаемых за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 <p>).

 Сведение (Reduction) A ≤p B: задача A сводится к задаче B.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP – нестрогий, для него нет быстрой (полиномиальной)
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
 <div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-400">Мосты

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400"
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
 `

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем
 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад
 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);
 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-l
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
 <thead>
```

```
<tr class="text-slate-500 border-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-500
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной комнате
 абелем (мостом). Сами свитчи — это точки со-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400 border-
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

 # PEP 709: с 3.12 компрекеншны встроены во фрейм
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ:

---

```
import type { DemoKind } from "../components/hints/MiniHint";
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину"],
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всё остальное"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок вершин произвольный.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Решение — сжатие координат с помощью сортировки и бинарного поиска.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова дерево"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева). Ожидаемая высота $O(\log n)$.",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id })))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 520 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Базовая база
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база — это структура данных,
 дерево как массив, куча, очередь и стек, бинарное дерево.
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <!-- Таблица мнемоники графовых алгоритмов -->
 <div class="my-8 overflow-x-auto">
```

```

 <td class="p-3 font-mono text-emera
</tr>
<tr class="hover:bg-slate-800/50 bg-sla
 <td class="p-3 font-bold text-white
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 text-slate-500 itali
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 font-mono text-emera
</tr>
<tr class="hover:bg-slate-800/50 bg-sla
 <td class="p-3 font-bold text-white
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 font-mono text-emera
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate
 <td class="p-3 text-slate-500 itali
</tr>
<tr class="hover:bg-slate-800/50 bg-sla
 <td class="p-3 font-bold text-white
 <td class="p-3 border-r border-slate
 <td class="p-3 border-r border-slate

```

```

 чейках n /
 n /
 па.</p>
 <pre class="bg-slate-950 border border-slate-700
tree = [None, 40, 20, 60, 10, 30, 50, 70] # tree[0] н
корень n=1
дети 40: 2*1=2 → 20 и 2*1+1=3 → 60
дети 20: 2*2=4 → 10 и 2*2+1=5 → 30
родитель 30: 5 // 2 = 2 → 20</pre>
 <p class="text-slate-300 text-sm mb-4">Цена: пу
 > деревья – кучу (ниже) и дерево отрезков из
 emerald-300">2n+1 и <span class="font-m
 ">(n-1)//2.</p>
 <details class="bg-slate-900 border border-slate
 <summary class="cursor-pointer px-4 py-2 text
 <div class="px-4 pb-4 text-slate-300 text-sm
 На уровне <span class="font-mono text-emera
 /span> вершин, а перед ним суммарно <span c
 class="font-mono text-emerald-300">k
 lass="font-mono text-emerald-300">n
 0">2n и <span class="font-mono text-
 де, включая heapq.
 </div>
 </details>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-emerald-400 m
 <p class="text-slate-300 text-sm mb-4">Куча
 ель ≤ своих детей. Отсюда магия: минимум всег
 минимума стоят <span class="font-mono text-e
 елей/детей. Полностью сортировать кучу не нуж
 <pre class="bg-slate-950 border border-slate-700
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heapp
h[0] # 1 – минимум виден без всякого об
heapq.heappop(h) # 1, куча починилась за O(log n)</pre>

```

```
sorted([3, 1, 2]) # [1, 2, 3] – новый список
sorted([(1, "6"), (1, "a")], key=lambda t: t[0]) # по первому элементу
<p class="text-slate-300 text-sm">Что лежит внутри?
 отвечают: слияние двух рангов длины k стоит O(k log k)
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l border-r border-slate-700">
 <h3 class="text-xl font-bold text-purple-400 mb-4">AVL-деревья
 <p class="text-slate-300 text-sm mb-4">AVL-деревья
 >каждой вершины выполнено условие баланса
 class="font-mono text-emerald-300">|h_L – h_R| ≤ 1
 зывают фактором баланса; из условия следует вращение
 авка и удаление за O(log n) даже в худшем случае
 <pre class="bg-slate-950 border border-slate-700 text-slate-300">
 Малый поворот (цепочка вправо-вправо стрелочками):
```

```

 z x
 L> x => / \
 L> y z y

```

Зигзаг (вправо-влево): сначала малый поворот нижнего из  
потом малый поворот верхнего – две перерисовки стрелок:

```

 z z x
 L> x => L> x => / \
 / y L> y z y

```

```
<p class="text-slate-300 text-sm mb-4">Поворот вокруг
 родителя y среднего звена и перевесить одно из поддерев
 отов вдоль пути вставки, и условие
 <details class="bg-slate-900 border border-slate-700 text-slate-300">
 <summary class="cursor-pointer px-4 py-2 text-slate-300">
 <div class="px-4 pb-4 text-slate-300 text-sm">
 x = right(z)

 right(z) = left(x) # внутреннее поддеревое
 left(x) = z # z становится левым ребенком
 высоты пересчитываются только у z и x – остальные не
 </div>
 </details>
```

```

 ь, не нужно переигрывать все матчи. Снял по
 тающихся.</p>
 </div>
 </div>
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-indigo-400 mb
 <p class="text-slate-300 text-sm leading-relaxe
 База закрыта — теперь ею пользуются остальные
 Кратчайшие пути живут в билетах 14–16 (<a data-
 derline font-semibold cursor-pointer">Дейкс
 300 hover:text-white underline font-semibol
 "text-indigo-300 hover:text-white underline
 остовные деревья — в билетах 18–20 (<a data-g
 semibold cursor-pointer">Краскал, Прим, Бор
 обходы и связность — в билетах 6–13 (<a data-
 white underline font-semibold cursor-pointe
 indigo-300 hover:text-white underline font-
 dges-code" class="text-indigo-300 hover:tex
 строки — в билетах 21–23 (<a data-goto="string
 ont-semibold cursor-pointer">КМП, <a da
 xt-white underline font-semibold cursor-poi
 text-indigo-300 hover:text-white underline
 Начинать удобно с <a data-goto="graph-dfs-bfs
 t-semibold cursor-pointer font-bold">билета
 </p>
 </div>
</div>

</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-

```



```

 <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-
 </p>
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="flex items-center mb-6">
 <div class="bg-blue-600 text-white px-4 py-1 rounded">
 <h2 class="text-3xl font-bold text-white">Форд-Беллман
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">Утверждение

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg

```

```

<h3 class="text-xl font-bold text-indigo-400">Аналогия 1: Автомагистрали</h3>

<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Аналогия 1: Автомагистрали</p>
 <p class="text-xl font-mono text-white">Аналогия 1: Автомагистрали</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Сколько времени долететь из Москвы в Париж, если мы садимся на самолет?
 </p>
 <div id="slot-chapter-image-floyd" class="h-400px">
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Сколько времени долететь из Москвы в Париж, если мы садимся на самолет?
 </p>
 </div>
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",

```

```

 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мос
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 outl
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скры
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>
 ссылку <code>term_link</code> (ближайшая мен
 <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
 map<char, int> go; // Предвычисленные пер
 int link = 0; // Суффиксная ссылка: куда
 int term_link = 0; // Терминальная ссылка: матр
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {

```

```
 </div>
 </div>
</section>`
 }
};
```

---

ФАЙЛ 23/111: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 200 | РАЗМЕР: 1000

---

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "sparse-table": [
 {
 question: "Сколько памяти занимает двумерная разрезка матрицы?",
 options: [
 "N × M – столько же, сколько сама матрица",
 "N × M × log N × log M: на каждом уровне почти по N × M",
 "N × M × 4 – по одной ячейке на каждый угол записи",
],
 correctIndex: 1,
 explanation: "Уровней log N по строкам и log M по столбцам. Если N и M о примерно 10 × 10 = 100 копий – ровно такова цена хранения.",
 },
 {
 question: "Можно ли считать сумму на прямоугольнике за O(1)?",
 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие нельзя",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 },
],
```

```

 explanation: "Сумма накапливается: каждая клетка
 ная здесь как раз разрезанная таблица (билет 2)
 },
 {
 question: "Запрос суммы прямоугольника (r1,c1)...(r2,c2)",
 options: [
 "S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]",
 "S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]",
 "S[r2+1][c2+1] - S[r1][c1]"
],
 correctIndex: 1,
 explanation: "С рамкой из нулей S[i][j] - это сумма
 ариант без +1 даёт ответ со сдвигом на клетку -1",
 },
 {
 question: "Почему левый верхний угол в формуле при
 options: [
 "Он входит и в левую, и в верхнюю полосу - его
 "Он всегда равен нулю, поэтому прибавление безоп
 "Так компенсируется рамка из нулей"
],
 correctIndex: 0,
 explanation: "Отрезая полосу слева и полосу сверху
 аланс: это и есть формула включений-исключений.",
 },
 {
 question: "Можно ли тем же приёмом считать минимум
 options: [
 "Да, если заменить вычитание на min",
 "Нет: у минимума нет обратной операции, нужна р
 "Да, но только для неотрицательных чисел"
],
 correctIndex: 1,
 explanation: "Весь приём держится на вычитании. З
 ной операции нет. Для min, max и gcd берут разр
 },
 {
 question: "Изменили один элемент матрицы. Что ста

```

```

 "Эйлера пути нет – нужно удалять лишние рёбра
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерово
 ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В нашем
 },
 {
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа в
 поиска мостов и точек сочленения."
 },
 {
 question: "Какова временная сложность алгоритма поиска
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
 }
]

```

ФАЙЛ 24/111: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 292 | РАЗМ

---

```
import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";
```

```
/**
 * Прямая связь «компилятор ↔ открытая визуализация».
 *
 * Компилятор передаёт не «магический номер строки эталонного
 * выполненного Python-кода. Визуализация читает захардкоженные
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие строки.
 * любой пользовательский код работает, если в нём использованы
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
 chapterId: string;
 /** Строка и функция реально выполненного пользовательского кода */
 line?: number;
 func?: string;
 /** Машинно-читаемые globals + locals: числа остаются строками */
 variables?: Record<string, DebugValue>;
 /** Имена, изменённые выделенной строкой. */
 changed?: string[];
}
```

```
/** Последний снимок хранится, чтобы поздно смонтировать визуализацию */
const latestByChapter = new Map<string, VizStateEventDetail>();
```

```
/** Транслировать реальные переменные визуализациям текста */
export function emitVizState(
 chapterId: string,
```

```

export type VizStepSelector = (variables: Record<string
```

```
/**
```

```
 * Подписка старых покадровых демонстраций. Если переда
```

```
 * номер кадра выбирается по именам/значениям переменны
```

```
 */
```

```
export function useVizStepSync(
 currentStep: number,
 goToStep: (n: number) => void,
 maxStep: number,
 selectByVariables?: VizStepSelector
) {
 const chapterId = useContext(VizChapterContext);
```

```
 useEffect(() => {
 if (!chapterId) return;
```

```
 const apply = (detail?: VizStateEventDetail) => {
 if (!detail || detail.chapterId !== chapterId) re
 // Без явного селектора компонент либо рисует use
 // либо остаётся ручным. Номер Python-строки нико
 const requested =
 selectByVariables && detail.variables ? selectB
 // null означает: компонент рисует runtime-перемен
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(requested))
 const target = Math.max(0, Math.min(Math.max(0, m
 if (target !== currentStep) goToStep(target);
 };
 });
```

```
 apply(latestByChapter.get(chapterId));
 const handler = (e: Event) => apply((e as CustomEvent)
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
}, [chapterId, currentStep, maxStep, goToStep, selectByVariables]);
```

```
/** Утилиты без небезопасных cast-ов для визуализаторов
```



```

}

const MAX_MATRIX_SIDE = 64;

const toCellNumber = (value: DebugValue | undefined): number | null {
 if (typeof value === "number" && Number.isFinite(value)) return value;
 // float("inf") снапшотится строкой "inf" – показываем NaN
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "+inf" || t === "infinity" || t === "-inf" || t === "-infinity") return Number.NaN;
 const n = Number(t);
 return Number.isFinite(n) ? n : null;
 }
 return null;
};

/** Ключ-пара вида "{0, 3}" или "0, 3" → [0, 3]. */
const parsePairKey = (key: string): [number, number] | null {
 const m = key.match(/^\(?\s*(-?\d+)\s*,\s*(-?\d+)\s*\)/);
 if (!m) return null;
 return [Number(m[1]), Number(m[2])];
};

export function liveMatrix(value: DebugValue | undefined): Map<string, number> {
 if (value === undefined || value === null) return new Map();

 const cells = new Map<string, number>();
 let rows = 0;
 let cols = 0;
 const put = (i: number, j: number, v: DebugValue | undefined) => {
 if (!Number.isInteger(i) || !Number.isInteger(j) || i < 0 || j < 0 || i >= MAX_MATRIX_SIDE || j >= MAX_MATRIX_SIDE) return;
 const n = toCellNumber(v);
 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
 };

```

```

 if (Number.isInteger(j)) put(i, j, v2);
 }
}
}

if (rows === 0 || cols === 0) return null;
return {
 rows,
 cols,
 at: (i, j) => (i >= 0 && j >= 0 && i < rows && j < cols),
 sizeLabel: `${rows}x${cols}`,
 hasNumbers: cells.size > 0,
};
}

/** Первое имя из списка, которое реально есть в снимке
export function pickLiveVar(
 vars: Record<string, DebugValue> | undefined,
 names: string[]
): { name: string; value: DebugValue } | null {
 if (!vars) return null;
 for (const name of names) {
 if (Object.prototype.hasOwnProperty.call(vars, name)) {
 return { name, value: vars[name] };
 }
 }
 return null;
}

/** Индекс, ограниченный реальной размерностью данных
export const clampIndex = (value: number | null, size: number): number | null => {
 if (value === null || !Number.isFinite(value) || size <= 0)
 return null;
 return Math.max(0, Math.min(size - 1, Math.trunc(value)));
}

/* — Выбор демонстрации внутри страницы (вкладки ID / chapter)

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

```

```
* Захардкоженная «карта синхронизации» компилятора с ви
*
* Для каждой страницы (главы) здесь вручную перечислен
* которые понимает её визуализация: i, j, k, v, dist, l
* Компилятор выполняет любой Python-код и передаёт реа
* имён напрямую; привязки к номерам или тексту строк э
*/
```

```
export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k,
 name: string;
 /** Роль в алгоритме (показывается во всплывающей под
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}
```

```
export interface PageSync {
 /** Заголовок демонстрации на странице (если интеракт
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тул
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет
 variables: SyncVariable[];
 /** Полный референсный Python-код, который кнопка-гла
 code?: string;
}
```

```
/** Справочные значения, указанные внутри самих симулято
```

```
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем tr
 code: `n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
```

```

 stack[-1].r = cur
 stack.append(cur)
 return stack[0] if stack else None

def split(t, m):
 """(L, R): в L всё с $x \leq m$, в R — с $x > m$."""
 global step
 if t is None:
 return (None, None)
 step += 1
 if t.x <= m:
 a, b = split(t.r, m)
 t.r = a
 return (t, b)
 a, b = split(t.l, m)
 t.l = b
 return (a, t)

def merge(a, b):
 """Слияние: все x в a меньше всех x в b ."""
 global step
 if a is None: return b
 if b is None: return a
 step += 1
 if a.y > b.y:
 a.r = merge(a.r, b)
 return a
 b.l = merge(a, b.l)
 return b

def erase(t, k):
 """Удалить точку с $x = k$: два split и один merge."""
 a, b = split(t, k - 1) # a: $x \leq k-1$, b: $x \geq k$
 mid_t, c = split(b, k) # mid_t: ровно $x = k$
 return merge(a, c)

def inorder(t, acc):
 if t:

```

## Универсальное пособие • полный исходный код

```

xk, pk, gk = 0, 0, 0 # их ключи-числа: 20 / 40
case = "zig" # какой случай сработал:
step = 0 # номер поворота
root = None # корень текущего дерева

--- тело алгоритма ---
class N:
 def __init__(s, k):
 s.k = k; s.l = None; s.r = None; s.p = None

def rot(x): # один поворот: поднять
 p = x.p
 g = p.p if p else None
 if p and p.l is x:
 p.l = x.r
 if x.r: x.r.p = p
 x.r = p
 elif p:
 p.r = x.l
 if x.l: x.l.p = p
 x.l = p
 if p: p.p = x
 x.p = g
 if g:
 if g.l is p: g.l = x
 else: g.r = x

def inorder(n, acc): # обход слева направо —
 if n:
 inorder(n.l, acc); acc.append(n.k); inorder(n.r)
 return acc

def splay(x):
 global case, step, p, g, root, xk, pk, gk
 while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
```

## Универсальное пособие • полный исходный код

```

code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])

for row in st:
 print(row) # готовая таблица: None там, где не посчитали

запрос min на отрезке за O(1): отрезок покрывают ДВА блока
L, R = 1, 6
K = (R - L + 1).bit_length() - 1
print("min на [" + str(L) + ", " + str(R) + "] =",
 min(st[L][K], st[R - (1 << K) + 1][K]),
 "| блоки длины", 1 << K)`
variables: [
 { name: "a", role: "исходный массив, из которого берем элементы"},
 { name: "n", role: "длина массива a", range: "n = 8"},
 { name: "LOG", role: "число уровней таблицы: log2 n"},
 { name: "st", role: "разреженная таблица: st[i][j] хранит min на отрезке [i, i + 2^j - 1]"},
 { name: "i", role: "начало блока, за который отвечает ячейка"},
 { name: "j", role: "уровень таблицы: ячейка хранит min на отрезке длины 2^j"},
 { name: "row", role: "очередной элемент a, из которого берем элементы"},
 { name: "L / R", role: "границы отрезка запроса — от L до R"},
 { name: "K", role: "степень двойки: длина блока 2^K"},
],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = заполнение очередного P[i] = P[i-1] + a[i]",
 code: `a = [3, 1, 4, 1, 5, 9, 2, 6] # исходный массив
n = len(a) # длина: префиксов будет n+1
i = 0 # текущая граница префиксов`

```

```

vizTitle: "Куча (priority queue)",
stepNote: "Шаг = просеивание: вставленный элемент в
 .",
code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
variables: [
 { name: "n", role: "текущее число элементов в куче" },
 { name: "i", role: "индекс элемента, который сейчас просеиваем" },
 { name: "(i-1)//2", role: "индекс родителя вершины" },
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка" },
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
v = st[-1] # вершина на вершине стека:
top = st.pop() # pop — сняли её со стека
print("v =", v, "| pop →", top, "| стек:", st)`,
 variables: [
 { name: "top", role: "вершина стека — то, откуда пришли" },
 { name: "v", role: "текущая вершина графа, которую сейчас просеиваем" },
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди и push соседей",
 code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущей вершины

```

```

v = stack.pop() # шаг: достали вершину с вер
if v in visited:
 continue
visited.add(v); order.append(v)
for u in adj[v]: # соседи v → на стек
 stack.append(u)
print("DFS:", order)

visited, order = set(), []
q = deque(["A"]) # BFS: очередь, обход по сл
while q:
 v = q.popleft() # шаг: достали из начала оч
 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → в конец очереди
 q.append(u)
print("BFS:", order)`
variables: [
 { name: "v", role: "текущая вершина обхода", group: "graph-components"},
 { name: "to / u", role: "очередной сосед вершины", group: "graph-components"},
 { name: "visited", role: "множество уже посещённых вершин", group: "graph-components"},
 { name: "order", role: "порядок, в котором обход", group: "graph-components"},
],
},
"graph-components": {
 vizTitle: "Компоненты связности: DFS-покраска",
 stepNote: "Шаг = вершина v достаётся из стека/очереди",
 code: `n = 10 # вершин в графе
adj = [[1, 3], [0, 2, 3], [1, 3], [0, 1, 2], [5, 6], [4, 6]]
comp = [-1] * n # -1 = вершина ещё не покрашена
used = [False] * n # то же самое булевым массивом
comps = 0 # счётчик найденных компонент
c = 0 # номер текущей компоненты
v = -1 # вершина, в которой обход не закончился
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то элементу adj

```



```
def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v) # выход из v!

for v in adj:
 if v not in used:
 dfs(v)

answer = order[::-1] # разворот — топологический
for i, v in enumerate(answer):
 print(f"позиция {i}: вершина {v}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = строка алгоритма: в фазе 1 DFS кла
 рованному графу и красит свою SCC.",
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4:
n = 5

фаза 1: DFS по обычному графу, запоминаем ПОРЯДОК ВЫХ
used = {}
for v in adj:
 used[v] = False
order = []

def dfs1(v):
 used[v] = True
 for to in adj[v]:
```

```

 { name: "adj / adjT", role: "граф и его транспозиция" },
 { name: "v", role: "текущая вершина DFS (в фазе 1)" },
 { name: "to", role: "сосед по ребру: в фазе 1 прямо" },
 { name: "used", role: "помеченные первым проходом" },
 { name: "order", role: "порядок ВЫХОДА из DFS первого прохода" },
 { name: "i", role: "индекс прохода по order на втором проходе" },
 { name: "stack", role: "стек DFS второго прохода: вершины, которые ещё не обработаны" },
 { name: "u", role: "вершина, снятая со стека во втором проходе" },
 { name: "comp", role: "номер компоненты сильной связности" },
 { name: "c", role: "счётчик компонент: каждая новая компонента увеличивает счётчик" },
 { name: "n", role: "число вершин", range: "5" },
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершина v",
 code: `tin, low, timer = {}, {}, 0

i = 0 # номер шага DFS
v, to = 0, 1 # текущее ребро DFS: v → to
tin[v] = tin.get(v, timer); low[v] = tin[v]

шаг: возврат из to в v
i += 1
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v] # → v — точка сочленения
print(f"i={i} v={v} to={to} critical={critical}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: "5" },
 { name: "to", role: "сосед: либо ребёнок в DFS-дереве, либо родитель" },
 { name: "tin[v]", role: "время входа в вершину (по времени)" },
 { name: "low[v]", role: "минимальный tin, достигнутый в subtree" },
 { name: "i", role: "перебор стартовых вершин для DFS" },
],
},
"bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v, to)",
 code: `
tin, low, timer = {}, {}, 0
i = 0
v, to = 0, 1
tin[v] = tin.get(v, timer); low[v] = tin[v]

шаг: возврат из to в v
i += 1
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v]
print(f"i={i} v={v} to={to} critical={critical}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: "5" },
 { name: "to", role: "сосед: либо ребёнок в DFS-дереве, либо родитель" },
 { name: "tin[v]", role: "время входа в вершину (по времени)" },
 { name: "low[v]", role: "минимальный tin, достигнутый в subtree" },
 { name: "i", role: "перебор стартовых вершин для DFS" },
],
},

```

```

for to, i in adj[v]:
 if not used[i]:
 used[i] = True
 stack.append(to)
 path.append(to)
 last = to
 step += 1
 moved = True
 break
if not moved:
 stack.pop() # из v все рёбра пройдены
print("степени:", deg)
print("нечётных вершин:", len(odd), odd, "->", "цикл" if len(odd) > 1 else "нет")
print("маршрут:", path, "| рёбра пройдено:", sum(used), "шагов")
variables: [
 { name: "path", role: "маршрут – последовательность вершин"},
 { name: "last / v", role: "вершина, где обход начался"},
 { name: "used", role: "какие рёбра уже пройдены (True/False)"},
 { name: "deg", role: "степени вершин: чётные всюду, нечётные в цикле"},
 { name: "odd", role: "список нечётных вершин – по порядку"},
 { name: "stack", role: "стек отката алгоритма Иерархического поиска"},
 { name: "step / i", role: "номер шага обхода", range: 1, n}
],
},
"planarity-euler-formula": {
 vizTitle: "Покраска вершин: $\chi \leq \Delta + 1$ ",
 stepNote: "Шаг = одна вершина v: смотрим цвета её соседей",
 code: `
n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 5]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежности
F = 2 - V + E # граней: из V - E + 2
 `
}

```

```

done = set()
pq = [(0, "A")]
while pq:
 du, u = heapq.heappop(pq)
 if u in done:
 continue
 done.add(u)
 print(f"обрабатываем {u} (dist={du})")
 for v, w in g[u]:
 if v in done:
 continue
 print(f"смотрим ребро {u}→{v} (w={w})")
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)\,
 variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в",
 code: `INF = float("inf")

демо-граф страницы: 7 вершин, 9 односторонних рёбер;
E = [("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A",
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E",
 ("D", "A", -2)] # замените -2 на -6 – появится

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер

```

```

code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF], [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, 0, INF], [INF, INF, INF, INF, 0, INF], [INF, INF, INF, INF, 0, INF]]
n = 7
for k in range(n):
 print(f"посредник k={k}")
 for i in range(n):
 for j in range(n):
 if i != j and i != k and j != k and d[i][k] < INF and d[k][j] < INF:
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
variables: [
 { name: "n", role: "число вершин – размер квадрата матрицы" },
 { name: "k", role: "промежуточная вершина – ВНЕШНЯЯ" },
 { name: "i", role: "начало пути – строка матрицы" },
 { name: "j", role: "конец пути – столбец матрицы" },
 { name: "dist[i][j]", role: "кратчайший путь i → j" },
],
},
"johnson-algo": {
 vizTitle: "Алгоритм Джонсона (перевзвешивание)",
 stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
 code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

u, v, w = "A", "B", -3 # ребро с отрицательным весом
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] ≥ 0
print(f"w'({u}→{v}) = {w_new}")
после Дейкстры: ответ = d[v] - h[start] + h[v]`,
variables: [
 { name: "h[v]", role: "потенциал вершины – кратчайшее расстояние от фиктивной вершины до v" },
 { name: "u, v", role: "ребро (u, v) перевзвешивается" },
 { name: "d[v]", role: "расстояние Дейкстры в новой матрице" },
],

```

```

 { name: "i", role: "номер текущего ребра в отсорти
 { name: "u, v, w", role: "концы и вес ребра, кото
 { name: "parent", role: "DSU: представитель множе
 { name: "rank", role: "высота дерева DSU – union
 { name: "mst", role: "взятые рёбра остова – рисуе
 { name: "total", role: "вес остова; на этом графе
],
},
"string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по
 code: `s = "aabaabaaa"

n = len(s)
i, j = 1, 0
pi = [None] * n # пустые клетки: значения п
pi[0] = 0

for i in range(1, n): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
 if s[i] == s[j]:
 j += 1
 pi[i] = j
print(pi)`,
 variables: [
 { name: "n", role: "длина строки s, для которой с
 { name: "i", role: "индекс текущего символа – счи
 { name: "j", role: "длина текущего наилучшего сов
 { name: "π[i]", role: "префикс-функция: длина наи
 ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l
 code: `s = "abacaba"

n = len(s)

```

```

term = {} # term[v] = слово, которое заканчивает

def add_word(w):
 """Добавить образец в бор: недостающие вершины полу
 global nodes
 v = 0
 for ch in w:
 if (v, ch) not in nxt:
 nxt[(v, ch)] = nodes
 nodes += 1
 v = nxt[(v, ch)]
 term[v] = w

for w in words:
 add_word(w)

BFS по уровням: fail-ссылка ребенка = куда приведёт f
fail[0] = 0
queue = sorted(v for (p, ch), v in nxt.items() if p == 0)
for v in queue:
 fail[v] = 0
qi = 0
while qi < len(queue):
 v = queue[qi]
 qi += 1
 for (p, ch), u in sorted(nxt.items()):
 if p != v:
 continue
 f = fail[v]
 while f != 0 and (f, ch) not in nxt:
 f = fail[f]
 fail[u] = nxt.get((f, ch), 0)
 if fail[u] == u:
 fail[u] = 0
 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

```

## Универсальное пособие • полный исходный код

```

cert = [] # сам сертификат – предполагаем
ok = True # проверился ли сертификат э

--- тело алгоритма ---
Полиномиальный алгоритм: перебор всех пар – $\Theta(n^2)$.
Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
 step = 1 # запустили алгоритм на вход
 ops = 0
 for i in range(n):
 for j in range(i, n):
 ops += 1
 step = 2 # ответ готов
 cls = "P" # $n^2/2$ операций – полином, э
 print("n =", n, "| ops =", ops, "|", cls)

Сертификат NP-задачи (3-SAT): решение угадано, провер
step = 3
cert = [1, 0, 1, 0] # значения переменных x1...x4
k = len(cert) # проверяем все k бит
ok = all(bit in (0, 1) for bit in cert)
cls = "NP" # угадать + быстро проверить
print("сертификат:", cert, "| k =", k, "| принят:", ok,
 variables: [
 { name: "n", role: "размер входа – горизонтальная
 { name: "ops", role: "сколько операций выполнил а
 " },
 { name: "cls", role: "имя класса (P / NP / PSPACE
 { name: "step / i", role: "кадр схемы (0 вход → 1
 i 0...9" },
 { name: "k", role: "размер сертификата: сколько э
 { name: "cert", role: "сертификат – назначение пер
 { name: "ok", role: "вердикт верификатора: принят
],
 },
intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
```



```

 stepNote: "Шаг = сосед и получает цвет 3 - color[v]
 code: `n = 8
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4], [3, 4, 5]]
color = [0] * n # 0 = не покрашена, иначе 1 или 2
queue = [] # очередь BFS
v = -1 # вершина, из которой смотрим
u = -1 # сосед, которого красим сейчас
ok = True # двудольный ли граф
i = 0

--- тело алгоритма ---
for start in range(n):
 if color[start] != 0:
 continue
 color[start] = 1
 queue = [start]
 while queue and ok:
 v = queue.pop(0)
 for u in adj[v]:
 if color[u] == 0:
 color[u] = 3 - color[v] # противоположный цвет
 queue.append(u)
 i += 1
 elif color[u] == color[v]:
 ok = False # нечётный цикл
 print("конфликт на ребре", (v, u))
print("двудольный:", ok, "| разметка:", color)`
variables: [
 { name: "color", role: "разметка на две доли: 1 и 2" },
 { name: "v", role: "вершина, из которой BFS смотрит" },
 { name: "u", role: "сосед, которого красим (или смотрим)" },
 { name: "queue", role: "очередь BFS", range: "≤ n" },
 { name: "ok", role: "вердикт: граф двудольный (не планарный)" },
 { name: "i / step", role: "номер кадра", range: "0..n-1" }
],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)"
}

```

## Универсальное пособие • полный исходный код

```

used = [False] * n # то же самое булевым массивом
comps = 0 # счётчик найденных компонент
c = 0 # номер текущей компоненты
v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS, очередь для BFS)
i = 0 # номер шага — двигает по кадру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "comp", role: "номер компоненты каждой вершины", type: "array", size: 1000, value: [-1]*1000 },
 { name: "v", role: "текущая вершина обхода", type: "int", size: 1, value: -1 },
 { name: "stack", role: "стек обхода (для BFS заметьте, что для DFS это очередь)", type: "array", size: 1000, value: [] },
 { name: "comps / c", role: "счётчик компонент и номер текущей компоненты", type: "int", size: 1, value: 0 },
 { name: "i", role: "номер шага — перематывает кадр", type: "int", size: 1, value: 0 }
],
},
"graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает"
```

```

 { name: "rank", role: "высота дерева DSU – union",
 { name: "e / step", role: "номер обрабатываемого ребра",
 { name: "u, v", role: "концы текущего ребра – подмножества",
 { name: "comps", role: "сколько множеств осталось",
 },
},
"complexity-classes#hierarchy": {
 vizTitle: "Иерархия классов $P \subseteq NP \subseteq PSPACE \subseteq EXP$ ",
 stepNote: "Шаг не нужен: схема подсвечивает тот овал, который",
 code: `cls = "NP" # имя класса: P, NP, PSPACE, EXP
n = 10 # размер входа
ops = 0 # счётчик операций (нужен вк. в алгоритм)

--- тело алгоритма ---
P: решается за полином – сортировка
a = [5, 2, 8, 1, 9, 3]
for i in range(len(a)):
 for j in range(len(a) - 1):
 ops += 1
 if a[j] > a[j + 1]:
 a[j], a[j + 1] = a[j + 1], a[j]
print(cls, "отсортировано за", ops, "операций:", a)`
 variables: [
 { name: "cls", role: "имя класса – P, NP, PSPACE, EXP",
 { name: "n", role: "размер входа задачи, про которую",
 { name: "ops", role: "сколько операций потребовалось",
],
},
"complexity-classes#reduce": {
 vizTitle: "Сведение $A \leq_p B$ (3-SAT $\rightarrow$ КЛИКА)",
 stepNote: "Шаг = один кадр схемы сведения: вход x $\rightarrow$ формула",
 code: `cls = "NP" # класс задачи A, B
step = 0 # кадр сведения A $\leq_p$ B: 0...6
n = 9 # размер входа x (литералов в дизъюнкте)
k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT $\leq_p$ КЛИКА: вершина = {номер дизъюнкта, номер литерала,

```



## Универсальное пособие • полный исходный код

```

 "E": [{"D": 3}, {"B": 6}, {"F": 7}, {"H": 11}],
 "G": [{"D": 8}, {"H": 10}], "H": [{"G": 10}, {"E": 11}],
 "I": [{"H": 12}, {"F": 13}]
start = "E" # из какой вершины растим дерево
heap = [(0, start, start)] # (вес, вершина, откуда пришла)
taken = [] # вершины уже в остовае – краевые
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
u, v, w = None, None, None # текущее ребро

--- тело алгоритма ---
while heap:
 w, x, frm = heapq.heappop(heap) # самое лёгкое ребро
 if x in taken: # устаревшая запись
 continue
 taken.append(x)
 u, v = frm, x
 if frm != x:
 mst.append((w, frm, x))
 total += w
 for to, wt in adj[x]:
 if to not in taken:
 heapq.heappush(heap, (wt, to, x))
print("вес остова:", total, "| вершин захвачено:", len(taken))
print(mst)

variables: [
 { name: "start", role: "вершина, из которой растим дерево" },
 { name: "heap", role: "куча рёбер-кандидатов (вес, вершина, откуда пришла)" },
 { name: "taken", role: "вершины, уже вошедшие в остовае" },
 { name: "u, v, w", role: "ребро, которое только что взяли" },
 { name: "mst / total", role: "рёбра остова и его вес" }
],
},
"mst#boruvka": {
 vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)",
 stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРШИННА",
 code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C`,

```

```

print(mst)`,
 variables: [
 { name: "comp", role: "DSU: корень компоненты вер",
 { name: "cheapest", role: "корень компоненты -> с",
 { name: "phase", role: "номер фазы; на этом графе",
 { name: "u, v, w", role: "ребро-кандидат, которое",
 { name: "mst / total", role: "взятые рёбра и вес "
],
},
"sparse-table#2d-build": {
 vizTitle: "Разреженная таблица 2D: построение",
 stepNote: "Шаг = новый уровень k: блоки $2^k \times 2^k$ доп",
 code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
 88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
 1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 — это
Записываем её одним шагом, а не проигрываем 64 одноти
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(0

for k in range(1, 4):
 print(f"уровень k={k}: блоки $\{1 \ll k\} \times \{1 \ll k\}$ ")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k
 + half, c + half, k - 1, k - 1)])`,
 variables: [
 { name: "r, c", role: "левый верхний угол блока,
 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[(r,c,k,k)]", role: "минимум квадратн
],
},

```



```

* Страница написана по канону остальных билетов (эталон
* одна главная секция, строгое правило по центру, анало
* явная строка сложности, весь код спрятан в <details>
*
* Визуализация – src/components/SegmentTreeVisualizer.
* построение / запросы / теория), синхронизация с комп
* (переменные n, i, v, l, r, m) и её подрежимы #build
*/
export const ticket1Segtree: Chapter[] = [
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description:
 "Структура для запросов на отрезке и обновлений э
 category: "Продвинутые структуры",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
ree[2v+1]), где m = (l + r) / 2. Запрос и о
 </div>

 <p class="text-slate-300 text-sm mb-4">Масси
 [0, n

 – это одиночные элементы массива. В узле л
 чество единиц, И/ИЛИ – любая операция, кото
 `
 }
]

```



## Аналогия 2: Матрёшки-коробки

```

</div>
<p class="text-slate-300 text-sm">М
Нужно покрасить половину массива в синий? Н
ь спускаемся только тогда, когда нас спроси
(push).</p>
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg l
 <p class="text-sm font-bold text-white m
 <p class="text-slate-300 text-sm mb-3">
на [L, R]» – всё это делается одним спуском
ss="font-mono text-emerald-300">tree[v] += 1
: перед любым спуском в детей вызываем p
 <table class="w-full text-xs text-left l
 <thead>
 <tr class="text-slate-500 border
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300 font-m
 <tr class="border-b border-slate
x · len</td><td class="py-1.5">lazy[v] += 1
 <tr class="border-b border-slate
c · len</td><td class="py-1.5">lazy[v] = c
 <tr class="border-b border-slate
len - tree[v]</td><td class="py-1.5">lazy[
 <tr><td class="py-1.5 pr-2">min
-1.5">lazy[v] = min(lazy[v], c)</td></tr>
 </tbody>
 </table>
 <p class="text-slate-500 text-xs mt-3">П
кивают в строгом порядке: сначала присваива
</div>

```

```

void build(int v, int l, int r) {
 if (l == r) { tree[v] = a[l]; return; } // лист
 int m = (l + r) / 2;
 build(2 * v, l, m); // левая
 build(2 * v + 1, m + 1, r); // правая
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // merge
}

int sum(int v, int l, int r, int L, int R) { // сумма
 if (r < L || R < l) return 0; //
 if (L <= l && r <= R) return tree[v]; //
 int m = (l + r) / 2;
 return sum(2 * v, l, m, L, R) // части
 + sum(2 * v + 1, m + 1, r, L, R);
}

void update(int v, int l, int r, int i, int x) { // a[i]
 if (l == r) { tree[v] += x; return; }
 int m = (l + r) / 2;
 if (i <= m) update(2 * v, l, m, i, x);
 else update(2 * v + 1, m + 1, r, i, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // перес
}

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-300">  
 -b border-slate-700/50">

Код: массовое обновление, lazy propagation

</summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">  
 y[4 \* MAXN]; // «обещание»: прибавит

```

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;

```

```

tree[v] = tree[2 * v] + tree[2 * v + 1]

def query(l, r): # сумма на полуинтервале [l, r)
 res = 0
 l += n; r += n
 while l < r:
 if l & 1: res += tree[l]; l += 1 # l – правый
 if r & 1: r -= 1; res += tree[r] # r – правый
 l //= 2; r //= 2
 return res

```

<p class="text-slate-400 text-xs">И  
 ней заметно хитрее – на экзамене безопаснее  
 </div>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate-  
 -b border-slate-700/50">

Вопросы, которые любят задавать (и  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <p>• Почему  $4n$ , а не  $2n$ ? – при  $n$ , н  
 пример:  $n = 6$ ).</p>  
 <p>• Сколько вершин посещает запрос  
 ва.</p>  
 <p>• Почему построение  $O(n)$ , а не  $O$   
 <p>• Какие операции подходят? – люб  
 количество различных на отрезке» обычное де  
 <p>• Зачем push в запросе, если нич  
 й; push делает данные детей достоверными.</p>  
 </div>

</div>  
 </div>  
 </section>  
 ` ,  
 } ,  
 ];

ext-emerald-300">u</span> обязан выйти из о  
x</span>, а у неё <span class="font-mono te  
бавляет</b> неотрицательные веса, значит кор  
ся механика: <b>извлечь минимум → зафиксиро  
ниже.</p>

```
<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Декстер Дуглас и Фр
 </div>
 <div class="my-3 rounded-lg overflow

 </div>
 <p class="text-slate-300 text-sm mb
елевизор). Ярко-синий электрический вирус <
а переходит от текущего узла к самым близким
ext-emerald-300">w ≥ 0), зафиксирован
ию расстояния!</p>
 </div>
```

```

 Повторять, пока куча не пуста
 .


```

```

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 font-bold"
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate"
 <td>Пиццерия A (0)</td><td class="py-2 font"
 <tr class="border-b border-slate"
 <td>Парк C (2)</td><td class="py-2 font-mono"
 <tr class="border-b border-slate"
 <td>Дом 1 B (3)</td><td class="py-2 font-mono"
 <tr class="border-b border-slate"
 <td>Метро G (5)</td><td class="py-2 font-mono"
 <tr class="border-b border-slate"
 <td>Офис D (6)</td><td class="py-2 font-mono"
 <tr class="border-b border-slate"
 <td>Дом 2 E (7)</td><td class="py-2 font-mono"
 <tr class="align-top"><td class="
 = "py-2 font-mono">исходящих рёбер нет</td><
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
 F 8. Из 11 проверок рёбер 10 улучши
 /span>: A → C → G → E → F = 2 + 3 + 2 + 1 =
 </div>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"

```



```

def dijkstra(graph, start):
 """graph[v] = [(сосед, вес), ...]; веса >= 0."""
 dist = {v: INF for v in graph}
 prev = {v: None for v in graph}
 dist[start] = 0
 done = set()
 pq = [(0, start)] # (расстояние,

 while pq:
 du, u = heapq.heappop(pq)
 if u in done: # устаревшая п
 continue
 done.add(u) # dist[u] окон
 for v, w in graph[u]:
 if du + w < dist[v]: # релаксаци
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
 return dist, prev

def restore(prev, start, target):
 if prev[target] is None and target != start:
 return None # вершина недо
 path = [target]
 while path[-1] != start:
 path.append(prev[path[-1]])
 return path[::-1]

демо-граф страницы (доставка пиццы, рёбра односторонни
g = {
 "A": [("B", 4), ("C", 2)],
 "B": [("G", 4), ("D", 5)],
 "C": [("B", 1), ("G", 3), ("E", 8)],
 "D": [("F", 3)],
 "E": [("F", 1)],
 "F": [],
 "G": [("D", 1), ("E", 2)],
}

```

```

 }
 }
}

vector<int> path(int s, int t) { // восста
 vector<int> res;
 if (d[t] == INF) return res; // недостижим
 for (int v = t; v != -1; v = p[v]) res.push_back(v)
 reverse(res.begin(), res.end());
 return res;
}

```

<p class="text-slate-400 text-xs">0  
 уть из  $10^5$  рёбер веса  $10^9$  даёт  $10^{14}$  – <span  
 </div>  
 </details>

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Код: $O(V^2)$ без куд
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 ти $n \times n$, $w[i][j] = \text{INF}$ если ребра нет
 d = [INF] * n
 used = [False] * n
 d[s] = 0

 for _ in range(n):
 u = -1
 for v in range(n): # ищем минимум
 if not used[v] and (u == -1 or d[v] < d[u]):
 u = v
 if u == -1 or d[u] == INF:
 break # остались тол
 used[u] = True # фиксируем d[u]
 for v in range(n): # релаксируем
 if d[u] + w[u][v] < d[v]:
 d[v] = d[u] + w[u][v]

```





```

 </div>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-500">
 <h3 class="text-xl font-bold text-blue-400">Демонстрация
 <p class="text-slate-300 text-sm mb-4">Демонстрация (включая 2 D, Лес Е, Склад F) и 9 односторонних
 В → С = -2 (сплав по течению – б
 >, которая в обычном режиме весит -6. В компиляторе
 внутри

<div class="bg-slate-800/70 p-5 rounded-lg border-1 border-slate-500">
 <p class="text-sm font-bold text-white">Таблица 1
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b border-slate-500">
 <th class="py-1.5 pr-2 font-normal">Вход
 <th class="py-1.5 pr-2 font-normal">Выход
 <th class="py-1.5 font-bold">Комментарий
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500">
 <td class="py-2 text-emerald-300">да</td><td class="py-2 text-emerald-300">да</td>
 <tr class="border-b border-slate-500">
 <td class="py-2 text-emerald-300">нет</td><td class="py-2 text-emerald-300">без изменений</td>
 <tr class="border-b border-slate-500">
 <td class="py-2 text-emerald-300">нет</td><td class="py-2 text-emerald-300">гарантия V –
 <tr class="align-top"><td class="py-2 text-emerald-300">отрицательный
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">В обычном порядке: S→A,
 порядок рёбер местами – и понадобится до ше

```



```

 for u, v, w in edges:
 if d[u] + w < d[v]:
 d[v] = d[u] + w

На n-ю ночь проверяем отрицательный цикл ("бесконечность")
for u, v, w in edges:
 if d[u] + w < d[v]:
 return True # Бэтмен нашел бесконечную кормушку
return False # Всё тихо, отрицательных циклов нет</div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Код: Форд–Беллман
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
def bellman_ford(n, edges, s):
 """edges = [(u, v, w), ...] – ориентированные рёбра
 dist = [INF] * n
 prev = [-1] * n
 dist[s] = 0
 changed_vertex = -1

 for it in range(n):
 changed_vertex = -1
 for u, v, w in edges:
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 prev[v] = u
 changed_vertex = v
 if changed_vertex == -1:
 break

 if changed_vertex == -1:
 return dist, prev, None

changed vertex улучшен на n-й итерации:

```

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-white">
 t;
```

```
#include <iostream>
using namespace std;
const long long INF = 4e18;
```

```
struct Edge { int u, v; long long w; };
```

```
int n; // число вершин
vector<Edge> edges; // плоский список рёбер
vector<long long> d;
vector<int> p;
```

```
// возвращает вершину, улучшенную на «лишней» итерации,
```

```
int bellman_ford(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 int x = -1;
 for (int it = 0; it < n; ++it) { // n-1
 x = -1;
 for (const auto &e : edges) {
 if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
 d[e.v] = d[e.u] + e.w;
 p[e.v] = e.u;
 x = e.v;
 }
 }
 if (x == -1) break; // ранний выход
 }
 return x;
}
```

```
vector<int> negative_cycle(int x) {
 int y = x;
 for (int i = 0; i < n; ++i) y = p[y]; // заходим в цикл
 vector<int> cycle;
 for (int v = y; v = p[v];) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 return cycle;
}
```

```

 if dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 if not in_queue[v]:
 q.append(v)
 in_queue[v] = True
 counter[v] += 1
 if counter[v] >= n:
 return None # отрицательный цикл
 return dist

```

```

 <ul class="list-disc pl-5 space-y-2">
 Ранний выход. Если з
рафах экономит кратно.
 SPFA (Shortest Path
, у которых только что улучшился <span clas
an>, но худший случай остаётся <span cla
кас» из рёбер) SPFA ловит TL. Критерий цикл
 Порядок рёбер. Форма
итерации, против течения – за <span class="
ция даёт финальные
 DAG. Если граф ацикл
0(V + E), и

 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
<h3 class="text-xl font-bold text-amber-400

```

```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-300">
 <ul class="list-disc pl-5 space-y-2 tex
 V – 1, а не V. Рабочих и
ont-mono text-emerald-300">V-я – тол
отрицательным циклом <span class="font-mono
 Переполнение INF. <span c
INT_MAX превращается в отрицательно
запасом и long lon

```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Флой,
ext-emerald-300">0(V^3) и <span class="
(но не с отрицательными циклами) и пишется
an> размера $V \times V$ </
вующих рёбер, <span class="font-mono text-em
n> в остальных клетках.</p>
```

```
<div class="my-5 p-5 rounded-xl bg-slate-900">
 <div class="mb-4 rounded-xl overflow-hidden">

 </div>
 <div class="flex items-center gap-3">

 <div>
 <h4 class="text-purple-300 font-mono">
 «Все против Все» (Все против
 </h4>
 <p class="text-slate-300 text-sm">
 Флойд–Уоршелл – это «Королева»
т кратчайший путь к абсолютно КАЖДОЙ другой
 </p>
 </div>
 </div>
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900
border-emerald-500 shadow-md">
 Суть фокуса (по шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
```

плакат окончательный. Порядок «разрешений»

```
an>.</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Аналогия 2: Почему k обязан быть

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4"
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400
```

```
<p class="text-slate-300 text-sm mb-4">Демо
вершины 0..6, <span class="font-mono text-em
merald-300">(i, j), которая улучшает
ляторе переменные <span class="font-mono te
class="font-mono text-emerald-300">j
напрямую: измените <span class="font-mono"
```

```
<div class="bg-slate-800/70 p-5 rounded-lg
```

```
<p class="text-sm font-bold text-white
```

```
<table class="w-full text-xs text-left
```

```
<thead>
```

```
<tr class="text-slate-500 borde
```

```
<th class="py-1.5 pr-2 font
```

```
<th class="py-1.5 pr-2 font
```

```
<th class="py-1.5 font-bold
```

```
</tr>
```

```
</thead>
```



```

 <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
 <tr class="align-top"><td class=
se-300">< 0</td><td class="py-2 text-rose-400">
делены</td></tr>
 </tbody>
 </table>
</div>
</div>
<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-700">
 <h3 class="text-xl font-bold text-indigo-400">Динамика
 <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Формальная динамика
 <div class="p-5 text-sm text-slate-300">
 <p>Обозначим i → j, все промежуточные вершины
 i и j
 <p>База: {0...k}
 <p>Переход: путь из {0...k} либо не использует
 {i}[j], – либо использует {i}[j]
 <p class="text-center font-mono text-slate-300">
 <p>Почему хватает одной матрицы.
 <p>и столбец k
 + d(k-1)[k][k], а d[i][j]
 мое не меньше первого. Значит, читая k-й итерации
 d[i][j] без
 <p>Следствие: если в графе е
 d[k][k] < 0 и рассуждение
 </div>
 </details>

```

```
<h3 class="text-xl font-bold text-emerald-400">
 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">> Код: Флойд–Уоршелл
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
```

```
def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ обнуляем

 for u, v, w in edges:
 if w < d[u][v]: # кратчайшее расстояние от u до v
 d[u][v] = w
 nxt[u][v] = v
 if not directed and w < d[v][u]:
 d[v][u] = w
 nxt[v][u] = u

 for k in range(n): # k – ВНЕШНИЙ цикл
 for i in range(n):
 if d[i][k] == INF:
 continue
 for j in range(n):
 if d[k][j] == INF:
 continue
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 nxt[i][j] = nxt[i][k]

 negative = [i for i in range(n) if d[i][i] < 0]
 return d, nxt, negative

def path(nxt, i, j):
 if nxt[i][j] == -1:
```

```

 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return false;
}

long long diameter() { // самая длинная дистанция
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (d[i][j] < INF) res = max(res, d[i][j]);
 return res;
}

int center() { // вершина с минимальной максимальной дистанцией
 int best = -1; long long bestEcc = INF;
 for (int i = 0; i < n; ++i) {
 long long ecc = 0;
 for (int j = 0; j < n; ++j) {
 if (d[i][j] == INF) { ecc = INF; break; }
 ecc = max(ecc, d[i][j]);
 }
 if (ecc < bestEcc) { bestEcc = ecc; best = i; }
 }
 return best;
}

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg p-4">

<summary class="p-4 font-bold text-slate-300">Динамика алгоритма  
-b border-slate-700/50">» Та же динамика для

<div class="p-5 text-sm text-slate-300">

```

] = 999 вместо 4).
 Диагональ.
«починит» её только при наличии цикла, а по
 Переполнение INF.
уходит в минус и «улучшает» всё подряд. Пр
no">long long.
 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Формулы
ых с циклом, ответ
t-monospace">d[i][j] = -∞, если существует
d[i][k] < ∞.
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память.
ачений (≈100 МБ на
источка берите Дейкстру.
 Сам путь. Матрица nxt[i][j] (как

</div>

<p class="text-slate-300 text-sm mt-4">Сравнение с другими алгоритмами
от одного источника и дешевле, если все пары не
n class="font-monospace">V Дейкстра быстрее
). Алгоритм Джонсона (билет 17) комбинирует
/span> Дейкстра на перевзвешенном графе. Традиционная
достижимость проверяется по DAG.</p>
</div>
</div>
</section>`
},
{
 id: "johnson-algo",
 title: "17. Графы. Поиск кратчайшего пути. Ускорение Дейкстры",
 type: "html",
 description:
 "Джонсон (перевзвешивание потенциалов), встречный

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
пускают две: от А и от В. Каждая зона роста
я не в момент встречи, а когда сумма
– иначе можно пропустить более дешёвый обхо
ирует.</p>
 </div>
</div>

```

```

<div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-bolder">Вход
 <th class="py-2 pr-3 font-bolder">Выход
 <th class="py-2 pr-3 font-bolder">Путь
 <th class="py-2 font-bolder">Описание
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">разрешает Дейкстра
 <td class="py-2 pr-3">g V)</td>
 <td class="py-2">нет отрицательных
 <td class="py-2">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">не обходим весь граф раз
 <td class="py-2">нужен путь до одной цели</td>
 <td class="py-2">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">две маленькие волны
 <td class="py-2 pr-3">е вдвое меньше раскрытий</td>
 <td class="py-2">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">раскрывает вершины в сторону цели
 <td class="py-2 pr-3">-2">есть допустимая эвристика h(v)</td>
 <td class="py-2">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">ss="py-2 pr-3">убирает кучу целиком</td>
 <td class="py-2 pr-3">лько 0 и 1</td>
 <td class="py-2">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3"><td class="py-2 pr-3">куча → массив корзинок</td>
 <td class="py-2 pr-3">С небольшое</td>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tr>
 </tbody>
 </table>
 </div>

```

```

 Запускаем быструю Дейкстру
class="font-mono text-emerald-300">d(u, v) =

 <p class="text-slate-400 text-xs mt-1">
 еженном графе ($E \approx V$) это $O(V^2 \log V)$ против
 ойд — для маленьких или очень плотных.</p>
 </div>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-1">
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны r
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; на
огда
более короткий путь.</p>

```

```

<h3 class="text-lg font-bold text-white mt-1">
<p class="text-slate-300 text-sm mb-3">Дейк
по g(v) (уже
 $h(v) = g(v) + h(v)$, где $h(v)$ — оценка
а требования: допустимость ($h(v) \leq w(v)$) и
монотонность/согласованность ($h(u) \leq w(u, v)$)
вырождается ровно в Дейкстру — это хороший

```

```

<h3 class="text-lg font-bold text-white mt-1">
<p class="text-slate-300 text-sm mb-3">Если
кучу заменяют деком: ребро веса 0 к
новременно живут максимум два уровня), поэто
>. Обобщение — алгоритм Дила (корзины) (или
 $E + V \cdot C$). Оба приёма — стандартный ответ на

```

```

<h3 class="text-lg font-bold text-white mt-1">
<p class="text-slate-300 text-sm mb-3">Форд
оторых расстояние только что уменьшилось
ческого прогона, но асимптотика та же, $O(V \cdot E)$
пользуют осторожно, с эвристиками SLF/LLL).
) уменьшает высоту и на практике быстрее, к

```



```

 mu = min(mu, nd + df[to])
 return mu

--- A*: приоритет f = g + h ---
def astar(g, s, t, h): # h(v) – допустимая оценка
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход: g
 if v in closed:
 continue
 closed.add(v)
 for to, w in g[v]:
 nd = gv + w
 if nd < gdist.get(to, INF):
 gdist[to] = nd
 heapq.heappush(heap, (nd + h(to), nd, to))
 return INF

--- 0-1 BFS: веса 0 и 1, дека вместо кучи ---
from collections import deque
def bfs01(g, s, n): # g[v] = [(to, w)]
 INF = float("inf")
 d = [INF] * n; d[s] = 0
 dq = deque([s])
 while dq:
 v = dq.popleft()
 for to, w in g[v]:
 if d[v] + w < d[to]:
 d[to] = d[v] + w
 (dq.appendleft(to) if w == 0 else dq.append(to))
 return d

```



```

* KruskalSimulator, три алгоритма живут на одном графе
* переключаются вкладками «Краскал / Прима / Борувка».
*
* Синхронизация с компилятором: PAGE_SYNC["mst"] + под
* "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
* через emitVizDemo, код в панели следует за ней).
*/
export const tickets18to20Mst: Chapter[] = [
 {
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, Прима",
 type: "html",
 description:
 "Три способа построить минимальное остовное дерево",
 category: "Графы. Остовные",
 content: `
<section id="mst" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 он связан. MST – такое остовное дерево, у
 </div>

 <p class="text-slate-300 text-sm mb-4">Все
 реза (cut property): разрежем вершины на
 MST. Алгоритмы отличаются только тем, К
 Прим – «выросшее дерево против остального г
 </div>
 <div class="bg-slate-800 p-5 rounded-lg border">

```

```

 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-b<
 <td class="py-2 pr-3">Все к
 <td class="py-2 pr-3 font-m<
 <td class="py-2 pr-3 font-m<
 <td class="py-2">параллельны
 </tr>
 </tbody>
</table>
</div>

<div class="grid grid-cols-1 xl:grid-cols-3
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
der border-indigo-500 shadow-md">
 Аналогия к билету 18 • Краск
 </div>
 <p class="text-slate-300 text-sm">С
» . По всей карте растут независимые кусочки
дерево. Порядок рёбер важен, порядок верши
 </div>

 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 * Аналогия к билету 19 • Прим:
 </div>
 <p class="text-slate-300 text-sm">Пр
а захватывает ближайший непосещённый город.
ва» против «снаружи» .</p>
 </div>

 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
 Аналогия к билету 20 • Боровк
 </div>

```

```

 lt;pair<int, pair<int,int>>>>
vector<int> p, rnk;

int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false; // уже в одной компоненте
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
}

sort(edges.begin(), edges.end()); // O(E log E)
long long cost = 0; int taken = 0;
for (auto [w, uv] : edges) {
 if (unite(uv.first, uv.second)) { // не замыкает цикл
 cost += w;
 if (++taken == n - 1) break; // V - 1 ребро
 }
}
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-300">  
 -b border-slate-700/50">

Код 19: Прим с кучей (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300">  
 <pre class="bg-slate-950 p-4 rounded">

heapq

```

INF = float("inf")
key = [INF] * n # минимальный вес ребра, ведущего к вершине
used = [False] * n
key[0] = 0
heap = [(0, 0)] # (вес ребра, вершина) ← вот

```

```

cu, cv = find(u), find(v)
if cu == cv:
 continue # уже внутри
if cu not in best or w < best[cu][0]:
 best[cu] = (w, u, v)
if cv not in best or w < best[cv][0]:
 best[cv] = (w, u, v)
if not best:
 break # граф несвя
for w, u, v in best.values():
 if find(u) == find(v):
 continue # дубль: два
 parent[find(u)] = find(v)
 comps -= 1
 total += w
print(total)</pre>

```

<p class="text-slate-400 text-xs">3.  
 ну), значит фаз не больше  $\log_2 V$ .</p>
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">

Вопросы, которые любят задавать (C  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <p>• Почему жадность здесь работает  
 меном ребра в предполагаемом MST.</p>  
 <p>• Чем Прим отличается от Дейкстр  
 ного веса против дерева кратчайших путей).</p>  
 <p>• Какой алгоритм лучше на плотной  
 <p>• Зачем нужна Борувка, если есть  
 независимо, поэтому её любят на GPU и в рас  
 <p>• Что если граф несвязный? – все  
 p>

<p>• Как найти второе по величине о  
 ребро на образовавшемся цикле (LCA + максим

```

<h4 class="text-indigo-300 font-bold text-sm">
 Префикс-функция $\pi[i]$ – смотрим $\pi[i-1]$
</h4>
<p class="text-slate-300 text-sm mb-4">
 Чтобы узнать значение $\pi[i]$ – в уже вычисленный префикс $\pi[i-1]$
 добавим символ $s[i]$
</p>
<ol class="list-decimal pl-5 space-y-1.5">
 Считаем значение $\pi[i]$
 Сравниваем следующий символ префикса
 – $\pi[i-1]$ с $s[i]$

<div class="bg-slate-950 p-3 rounded-lg">
 <div class="inline-flex items-center justify-center">
 |<
 |<
 |<
 </div>
 <p class="text-xs text-indigo-300 margin-0">
 – Рамка смотрит ВЛЕВО: проверяет, не совпадает ли
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. $\pi[1] = 0$
 ab: Из начала ("a") не получилось
 abc: Опять мимо. $\pi[2] = 0$

 </div>

```

```

ждения), делаем $j = \pi[j - 1]$.</p>
<p class="text-slate-300 text-sm mb-4">П
ждый откат j уменьшает. Значит суммарно отк
классическая амортизационная оценка, её люб
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
ый алгоритм.</p>
 <p class="text-slate-400 text-sm mb-1">
т онлайн (символы могут приходить потоком),
 <p class="text-slate-400 text-sm">• I
5] = 4, период $6 - 4 = 2$ («ab»)</p>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: поиск подстроки КМР (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
fix_function(s):
n = len(s)
pi = [0] * n
for i in range(1, n):
 j = pi[i - 1]
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат по уже п
 if s[i] == s[j]:
 j += 1
 pi[i] = j
return pi

def kmp_search(text, pattern):
 m = len(pattern)
 pi = prefix_function(pattern) # O(m) один раз
 j = 0

```

```
<h3 class="text-xl font-bold text-emerald-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
ca 0) и её суффиксом, начинающимся с i.</p>
</div>
```

```
<div class="bg-slate-900 border border-emerald-500">
 <h4 class="text-emerald-300 font-bold text-sm">
 Z-функция $Z[i]$ – смотрим ВПРАВО ($i+1$)
 </h4>
 <p class="text-slate-300 text-sm mb-4">
 Буква Z запоминается как Z
 i и подглядываем ВПРАВО (ПОЗАДИ)
 s[0...].
 </p>
 <div class="bg-slate-950 p-3 rounded-lg">
 <div class="inline-flex items-center justify-center">
 [0
 |
 |<
 |
 </div>
 <p class="text-xs text-emerald-300">
 - Рамка смотрит ВПРАВО: измеряет
 </p>
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcbcd</p>
 <ul class="list-disc pl-5 space-y-2">
```

```

<h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>
```

```
<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo">
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("1
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2">
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border b
```



```

 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 ат). Сведение A $\leq$ р B – преобразовать вход A
 .</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Клас
 машина Тьюринга) с выбранным бюджетом ресурс
 про худший случай, а не про «в средн

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (Р против
 </div>
 <p class="text-slate-300 text-sm">К
 ий путь, MST). Класс NP – когда реша
 проверишь, что там нет нарушений правил. Во
 и решить?». Открыт с 1971 года и стоит мил

 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Переводчик (сведе
 </div>
 <p class="text-slate-300 text-sm">С
 ский (сведение за полином) и друг-англичанин
 Отсюда главный инструмент: если к B сводитс
 ь не стоит.</p>
 </div>
 </div>
 </div>

```

```

 й на полиномиальном поле</td></tr>
 <tr class="align-top"><td class="
за $2^{(n^k)}$ </td><td class="py-2">обобщённые
 </tbody>
 </table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Стандартный сертификат полиномиального размера и проблема
 – «все вершины по разу и каждая соседняя пара образует
ь полиномиальное сведение $A \leq_p B$. Тогда B – полиномиально
<p class="text-slate-300 text-sm mb-3">Теорема 1.1. Проблема
мул SAT NP-полна. Доказывается «в лоб»: выводится универсальн
ею формулой размера $\text{poly}(n)$, где переменные соответствуют
ильна принятию входа. Дальше всё катится целиком к сведению
гамильтонов цикл $\rightarrow$ TSP.</p>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg border">
 <p class="text-sm font-bold text-white">
 <ul class="list-disc list-inside text-slate-300">
 Точный перебор с отсечениями (meet-in-the-middle).
 Приближённые алгоритмы:
et cover – $\ln n$.
 Эвристики и метаэвристики:
ке.
 Параметризованная сложность
n).
 Специальные классы входов:
полиномиальными (независимое множество на д
 Готовые решатели: SAT/ILP,
воего алгоритма.

</div>

```

```

<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">

```

```
def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)]
 edges = []
 for i in range(len(verts)):
 for j in range(i + 1, len(verts)):
 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # х и н
 edges.append((i, j))
 return verts, edges, len(clauses) # клика
```

# Проверка задачи из P для контраста: двудольность = 2-р

```
def is_bipartite(n, graph):
 from collections import deque
 col = [-1] * n
 for s in range(n):
 if col[s] != -1:
 continue
 col[s] = 0
 dq = deque([s])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1
 dq.append(to)
 elif col[to] == col[v]:
 return False
 return True
```

<p class="text-slate-400 text-xs">ОТ  
ТОВЫЙ ОТВЕТ, НО НЕ НАХОДИТЬ ЕГО; ДЛЯ ЗАДАЧИ

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

```

category: "Графы. База",
content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>
 {/* DFS vs BFS */}
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

```
йкстра).
```

```

```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

```
Код обходов C++ (Основа)
```

```
</summary>
```

```
<div class="grid grid-cols-1 md:grid-co
```

```
<pre class="bg-slate-950 p-3 rounde
```

```
// DFS (Рекурсия с возвратом)
```

```
vector<bool> vis;
```

```
vector<vector<int>>> adj;
```

```
void dfs(int v) {
```

```
 vis[v] = true;
```

```
 for (int u : adj[v]) {
```

```
 if (!vis[u]) dfs(u);
```

```
 }
```

```
}</pre>
```

```
<pre class="bg-slate-950 p-3 rounde
```

```
// BFS (Очередь FIFO)
```

```
queue<int> q;
```

```
q.push(start_node);
```

```
vis[start_node] = true;
```

```
while (!q.empty()) {
```

```
 int v = q.front(); q.pop();
```

```
 for (int u : adj[v]) {
```

```
 if (!vis[u]) {
```

```
 vis[u] = true;
```

```
 q.push(u);
```

```
 }
```

```
 }
```



```
</div>
```

</ul>

<div class="bg-slate-950 p-4 rounded-lg

<p class="text-amber-300 font-bold

<p class="text-slate-300 text-xs le

1. <b> $K_5$  (полный граф на 5 вершин  
class="font-mono"> $10 \leq 3(5) - 6 = 9$ </span> -

2. <b> $K_{3,3}$  (полный двудольный  $3 \times 3$   
ss="font-mono"> $9 \leq 12$ </span> – формально пр  
 $2V - 4$ </span>: <span class="font-mono"> $9 \leq$

</p>

</div>

<p><b>Теорема Куратовского (1930) / Вагн  
фа, являющегося <b>подразделением (subdivis  
</p>

</div>

</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-purple-400

<div class="bg-slate-900 p-4 rounded-lg mb-1

<p class="text-lg text-purple-300 italic

<p class="text-xl font-mono text-white"

</div>

<div class="space-y-3 text-slate-300 text-s

<p><b>Сложность задачи раскраски:</b></p>

<ul class="list-disc list-inside space-y-

<li><b> $k = 2$  (2-раскрашиваемость):</b>  
</b> обычным BFS/DFS обходом.</li>

<li><b> $k \geq 3$  ( $k$ -colorability):</b>  
</li>

</ul>

<p class="mt-3"><b>Теоремы о раскраске:</b>

<ul class="list-disc list-inside space-y-

<li><b>Жадная раскраска:</b> дает в

каждое ребро делаем `union(u, v)`, в конце читаем `count` и после каждого добавления спрашиваем

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
```

Аналогия 1: Острова

```
 </div>
 <p class="text-slate-300 text-sm">Теперь появились серые (неисследованные) зоны? Прыгаешь по островам и компонентам. Краска — это comp[v], прыжок ч
```

```
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
```

Аналогия 2: Группы друзей (DSU)

```
 </div>
 <p class="text-slate-300 text-sm">Каждый человек принадлежит к одной компании в одну (union). Кто представитель компании? Кто не принадлежит к компании, и вопрос «в одной ли компании два человека», только если концы в разных компонентах
```

```
 </div>
</div>
```

```
<h3 class="text-lg font-bold text-white mt-6">Таблица компонент
<div class="overflow-x-auto">
```

```
 <table class="w-full text-xs text-left">
 <thead>
```

```
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-bold">ID
 <th class="py-2 pr-3 font-bold">Parent
 <th class="py-2 pr-3 font-bold">Size
 <th class="py-2 font-bold">Color
```

```
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
```

```
<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
```

Аналогия 1: Учеба в вузе

```
</div>
```

```
<p class="text-slate-300 text-sm mb-0">
 Аналоговая сортировка – это расписание, которое
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
```

Ограничения: Циклы

```
</div>
```

```
<p class="text-slate-300 text-sm mb-0">
 Чтобы получить опыт, нужна работа". Алгоритм выявляет
</p>
```

```
</div>
```

```
</div>
```

```
<div class="bg-rose-950/40 p-4 rounded-lg border border-rose-500">
```

```
<p class="text-rose-300 font-bold text-sm">
```

```
<p class="text-slate-300 text-xs leading-5">
```

Обычный массив `<span class="font-mono">` ли в графе есть ориентированный цикл, код с

`<b>Правильно:</b>` используем раскраску

`<br>• <span class="font-mono text-slate-300">`

`<br>• <span class="font-mono text-amber-300">`

`<br>• <span class="font-mono text-emerald-300">`

`<br>`Если во время обхода из серой вершины `<b>гарантированный цикл</b>`!

```
</p>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
```

```
<summary class="p-4 font-bold text-slate-300">
 Проверка корректности кода C++ с детекцией циклов
</summary>
```

Корректный код C++ с детекцией циклов

```
</summary>
```



```

</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарайю",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700/80">
 <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Вспомогательная теорема: Если в орграфе G есть цикл, то в нем обязательно есть цикл из вершин, принадлежащих к одной и той же компоненте сильной связности.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты сильной связности (SCC) — это максимальные по включению подграфы орграфа, в которых каждая вершина принадлежит к одному и тому же циклу.
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» — Нет!
 </p>

 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код: Алгоритм Косарайю (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю — это алгоритм нахождения компонент сильной связности (SCC) в орграфе. Он основан на двух проходах по графу: первом — для нахождения компонент сильной связности, и втором — для их упорядочивания.
 </p>
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю — это алгоритм нахождения компонент сильной связности (SCC) в орграфе. Он основан на двух проходах по графу: первом — для нахождения компонент сильной связности, и втором — для их упорядочивания.
 </p>
 </div>
 </div>
 </div>
`
 }
]

```





```


 </div>
</div>

<h3 class="text-lg font-bold text-white mt-1">Исполнение
<p class="text-slate-300 text-sm mb-3">Исполнение
 → рекурсивный переход. При возврате (на выходе)
ты:
 <details class="bg-slate-900/40 rounded-lg p-4"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код построения Эйлера цикла C++
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
vector<unordered_multiset<int>>> adj;
vector<int> path;

void find_euler(int v) {
 while (!adj[v].empty()) {
 int u = *adj[v].begin();
 adj[v].erase(adj[v].find(u));
 adj[u].erase(adj[u].find(v)); // Убираем ребро
 find_euler(u);
 }
 path.push_back(v); // Добавляем в путь на выходе!
}</pre>
 </div>
</details>
</div>
</div>
</section>`,
 },
];
=====

```

```

 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

 // Adjacency list
 const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
 edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
 });

 interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
 }

 export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }
));
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];

```

```

 currentLow[v] = Math.min(currentLow[v], currentTin[v]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);

 if (currentLow[to] >= currentTin[v] && p !== ap.add(v)) {
 recordStep(
 `Найдена точка сочленения: вершина ${v}`,
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это`,
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он`,
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки сочленения.");

```



```

<circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
/>
<text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
>
 {node.label}
</text>

{isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}

```



```
 Посещена
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border">
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded ${
 idx === currentStepIndex ? 'bg-rose-500' : 'bg-slate-500'
 }`}
 >
 {step.desc}
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition-all"
 style={{
 width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
 }}
 </div>
 </div>
 </div>
 </div>
</div>
```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));

```

```

 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 ${updated ? `Релаксация успешна! dist[${edge.v}] = ${uDist + edge.w}` : ` `}`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: `Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10,
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v} образует цикл!`,
 activeCodeLine: 9,
 });
 }
 }
}

```

```

const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node.id, 0])),
 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-3">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white rounded-lg">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-blue-300">Полный цикл
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border border-slate-800">
 <button onClick={() => setHasCycle(false)} className="p-1 text-blue-400">
 <button onClick={() => setHasCycle(true)} className="p-1 text-blue-400">
 </div>
 <div gap-1 ${hasCycle ? 'bg-rose-600 text-white' : 'bg-slate-800 text-white'}>
 <button onClick={() => { setIsPlaying(false); TEXT : "Вернуться к первому шагу"} } className="p-1 text-white rounded">
 <button onClick={() => setHasCycle(true)} className="p-1 text-blue-400">
 <button onClick={() => setHasCycle(false)} className="p-1 text-blue-400">
 </div>
 </div>
 </div>
)

```

```

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u);
 const vNode = nodes.find((n) => n.id === v);
 const isChecking = currentStep.checkingEdges;

 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; }
 else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={
 isChecking && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} stroke={strokeColor} strokeWidth={
 isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={edge.w} fontColor={strokeColor}
 fontSize="11" fontWeight="bold" />
 </g>
);
 })
);
}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNodes;

 return (
 <g key={node.id} className="transition-all duration-200">
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? 15 : 10} stroke={isCheckingNode ? '#7dd3fc' : '#64748b'}
 strokeWidth={isCheckingNode ? 2 : 1} />
 <text x={node.x} y={node.y + 5} fill={isCheckingNode ? '#7dd3fc' : '#64748b'} />
 <text x={node.x} y={node.y - 28} fill={isCheckingNode ? '#7dd3fc' : '#64748b'} />
 </g>
);
})
}

```

```

 { /* Таблица расстояний */ }
 <div className="w-full lg:w-1/2 bg-rose-950/10" >
 <div>
 <h4 className="text-lg font-bold text-rose-950" >
 <div className="grid grid-cols-4 gap-2 text-rose-950" >
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge === n.id;
 return (
 <div key={n.id} className={`p-2 rounded-lg border-slate-700/60 bg-slate-800/40`} >
 <div className="text-xs text-slate-400" >
 <div className={`font-mono font-bold`} >
 {d}
 </div>
 </div>
 </div>
);
 })}
 </div>
 </div>
 </div>

 { /* Код алгоритма */ }
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-400" >
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-2`} >
 {item.line === currentStep.start ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}

 {item.code}

 </div>
))}
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0).",
 });

 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по",
 });

 while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`,
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
```

```

 }
 }

 steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 }, [stepIdx, isPlaying]);
}
```



```

<div className="grid grid-cols-1 lg:grid-cols-2 g
 { /* Визуал графа */ }
 <div className="bg-slate-800 border border-slate
 50px]">
 <svg className="w-full h-full min-h-[350px]"
 { /* Рёбра */ }
 { EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return {
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500
 `} `}
 />
 }
)
 }
 }
 { /* Вершины */ }
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600

```

```

<div className="bg-slate-800 text-slate-400
se tracking-wider">
 bfs(graph, start)
 ⏮ar {step
</div>
<div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp
 { line: 5, code: ' node = queue.pop()'
 { line: 6, code: ' for neighbor in grap
 { line: 7, code: ' if neighbor not in
 { line: 8, code: ' visited.add(ne
 { line: 9, code: ' queue.push(nei
]}.map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5' : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
);
 })}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (

```

```
 src: bellmanImg,
 alt: 'Бэтмен на машине Генри Форда',
 caption: ' Бэтмен на машине Форда – n-1 ночей патр',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Fortnite Все против Всех (Флойд–Уоршелл)',
 caption: ' Fortnite: Все против Всех – Флойд-Уорше',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor}`}>

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}`}>
 {info.caption}
 </p>
 </div>
);
}
```

---

ФАЙЛ 35/111: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```

 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}
 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);

```

```

 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);

 if (!list || list.length === 0) return null;

 const q = list[idx];
 const answered = picked !== null;
 const correct = answered && picked === q.correctIndex

 const pick = (i: number) => {
 if (answered) return;
 setPicked(i);
 if (i === q.correctIndex) setScore((s) => s + 1);
 };

 const next = () => {
 if (idx + 1 < list.length) {
 setIdx(idx + 1);
 setPicked(null);
 } else {
 setFinished(true);
 }
 };

 const restart = () => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 };

 return (
 <section aria-label="Блиц по теме" className="mt-8"
 <div className="bg-slate-700/50 p-5 rounded-xl border
 <div className="flex items-center justify-between

```

```

 ? "border-slate-600 bg-slate-800/70 to
 : isRight
 ? "border-emerald-500 bg-emerald-500
 : chosen
 ? "border-rose-500 bg-rose-500/15
 : "border-slate-700 bg-slate-800/70
 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex items-center justify-between disabled:cursor-default ${tone}`}
 >
 {answered && isRight && <CheckCircle/>&&
 {answered && chosen && !isRight && <X/>&&
 {option}
 </button>
);
 }
}
</div>

{answered && (
 <div role="status" className="mt-3 rounded-lg p-4">
 <p className={`text-xs font-bold mb-1 ${correct ? "text-emerald-500" : "text-rose-500"}}`>
 {correct ? "Верно" : "Неверно"}
 </p>
 <p className="text-[13px] leading-relaxed">
 {feedback}
 </p>
 </div>
)}

<div className="mt-3 flex justify-end">
 <button
 type="button"
 onClick={next}
 disabled={!answered}
 className="inline-flex items-center gap-2"
 >
 Далее
 </button>
</div>

```

```

 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}

const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;

export const ChapterView: React.FC<Props> = ({ chapter, ...props }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);
 /**
 * Упоминания «билет 15» становятся переходами на стр. 15
 * Разметка идёт ПОСЛЕ терминов: подсказки глоссария
 */
 useTicketLinks(contentRef, [chapter.id], { selfId: chapter.id });

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
 }, [cancelHide]);

```

```

 } catch {
 setSaving("idle");
 }
 }, [chapter]));

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 // ссылка на другой билет: переходим внутри приложения
 const link = (e.target as HTMLElement).closest<HTMLElement>(".link");
 if (link) {
 e.preventDefault();
 onGo(link.dataset.goto ?? "");
 return;
 }
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (!el) return;
 e.preventDefault();
};

```



```

 >
 <Printer className="w-3.5 h-3.5" /> PDF это
 </button>
 <a
 href={`?lite=1&topic=${encodeURIComponent(chapter.id)}&print=1`}
 target="_blank"
 rel="noreferrer"
 title="Текстовая версия темы без навигации,
 еры"
 className="flex items-center gap-1.5 text-x-3
 300 hover:text-white hover:border-sky-500 t
 >
 <FileText className="w-3.5 h-3.5" /> Текст

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.content
/>

{chapter.id === "intro" && (
 <div className="my-8 print:hidden">
 <h3 className="text-xl font-bold text-white">
 Мнемонические карточки кра
 </h3>
 <MnemonicCards />
 </div>
)}

[["dijkstra", "bellman-ford", "floyd"].includes(chapter.id) && (
 <div className="my-6 print:hidden">

```

```
 </VizChapterContext.Provider>
 </section>
 })

 {/* прочитал → посмотрел, как работает → провер
 <ChapterQuiz chapterId={chapter.id} />

 <ChapterNav prev={prev} next={next} index={inde
 </article>

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
</>
);
};
```

---

ФАЙЛ 38/111: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useContext, useEffect, useMemo, useState
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { VizChapterContext, emitVizDemo, useVizRuntime,
```

```
/**
```

```
 * Билет 24 – «Классы сложности, сведение задач».
```

```
 *
```

```
 * Четыре вкладки:
```

-----  
/\* ----- Сведение -----

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма
 – то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x – конкретный экземпляр
 "x]", active: 1 },
 { title: "Строим $f(x) = y$ за полиномиальное время", note: "Полиномиальное
 шинами из разных дизъюнктов, если литералы не противоречат.
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм B
 – но допустим, что он есть.", boxes: ["A", "x", "f"], active: 3 },
 { title: "Переводим ответ обратно: $A(x) = B(f(x))$ ", note: "Поскольку B
 $\Rightarrow$ формула выполнима. Полиномиальное время сохранилось.
 5 },
 { title: "Вывод: B не проще A", note: "Раз $A \leq_p B$ и A NP-полна.
 -полна.", boxes: ["A", "x", "f", "y", "B", "ans"], active: 6 },
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
 A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
 x: { t: "вход x\n|x| = n", c: "#64748b" },
 f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
 y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
 B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
 ans: { t: "ответ\nA(x) = B(f(x))", c: "#10b981" },
};
```

-----  
/\* ----- Верификатор -----

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные 1..4
const CLAUSES: [number, number, number][] = [
 [1, 2, -3],
 [-1, 3, 4],
 [2, -3, -4],
 [-2, 1, 3],
];
/** Сертификат, который проверяем: x = (1, 0, 1, 0) – он удовлетворяет
```

```

 setStep(0);
 setPlaying(false);
}, [tab]));

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNum
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArr
const liveOk = vars ? (vars.ok === undefined ? null :
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNum

/** Копим точки (n, ops): код пользователя крутит цикл
useEffect(() => {
 if (liveN === null || liveOps === null) return;
 if (!Number.isFinite(liveN) || !Number.isFinite(live
 setSamples((prev) => {
 const last = prev[prev.length - 1];

```

```

const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "log"
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 }
return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length - 1)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-purple-600 text-white rounded">
 <Boxes className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">
 <p className="text-xs text-purple-300">P · I
 </h3>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => (
 <button

```

```

 {on ? 3 : 1.5} strokeDasharray={on ? undefi
 <text x={c.x} y={c.y - c.ry + 18} t
 <text x={c.x} y={c.y - c.ry + 32} t
 </g>
);
 }}}
 /* примеры задач – внутри своих овалов */
 {CLASSES.map((c) =>
 c.tasks.map((t, i) => {
 <text key={t} x={c.x} y={c.y + (c.id :
 tAnchor="middle" fill="#e2e8f0" fontSize="8
 . {t}
 </text>
 })
 })
 /* NP-полные = пересечение */
 <text x={250} y={352} textAnchor="middle"
 <text x={250} y={368} textAnchor="middle"
 рхии времени)</text>
 </svg>
</div>
<p className="text-xs text-slate-400 leading-
<p className="text-[11px] text-slate-500">
 Подсказка: выполни в компиляторе <code clas
 </p>
</div>
})

```

```

/* ----- СВЕДЕНИЕ -----
{tab === "reduce" && {
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <div className="flex flex-wrap items-center
 {reduceFrame.bboxes.map((b, i) => {
 const meta = BOX_LABEL[b];
 const active = i === reduceFrame.active
 return {
 <React.Fragment key={b + i}>

```

```

 {liveCert && cert из кода}
 </div>
 <div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < liveK : false;
 const current = liveK !== null ? i === liveK : false;
 const st = VERIFY_STEPS[i];
 return (
 <div
 key={i}
 className={
 "flex items-center gap-3 p-3 rounded-sm border border-amber-950/20" : checked ? "border-rose-600/40 bg-slate-800 bg-slate-900/40"
 }
 >
 {current ? "Текущий шаг" : "Шаг"}
 {st}
 {st.ok ? "OK" : "Error"}
 <span className={
 "text-[11px] font-normal" : "text-[11px] font-bold"
 }>{checked ? (st.ok ? "✓ истинно" : "✗ ложно") : ""}
 </div>
);
 })}
 </div>
 {(verifyDone || liveOk !== null) && (
 <div className={
 "mt-3 rounded-lg border p-3 text-emerald-300" : "border-rose-600/40 bg-slate-800 bg-slate-900/40"
 }>
 {(liveOk ?? allOk) ? "Сертификат принят" : "Сертификат не принят: хотя бы один дизъюнкт ложен."}
 </div>
)}
 </div>
 <div className="bg-slate-900/70 border border-rose-600/40 p-3">
 <p className="text-sm font-bold text-white">

```

```

 const x = 44 + (Math.log(Math.max(1, s.n)) * 50);
 const y = 200 - (Math.log(Math.max(1, s.n)) * 50);
 return <circle key={`${s.n}`} cx={Math.min(x, 300)} cy={y}
strokeWidth={2} />;
 }}}
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + (i % 3) * 16} width={58} height={16} />
 <text x={343 + (i % 3) * 58} y={24 + (i % 3) * 16} />
 </g>
))}
 </svg>
 {samples.length === 0 && (
 <p className="text-xs text-slate-500 text-center">
 Пусто. Запусти в компиляторе код с переменными t-mono text-purple-300 — например,
 </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " + (samples.length > 0 ? "text-white" : "text-slate-500")} />
 <p className="text-sm font-bold text-white">
 {verdict ? `Похуже на ${verdict.name}` : "Пусто"}
 </p>
 <p className="text-xs text-slate-400 leading-relaxed">
 {verdict ? `Наклон в лог-лог координатах ≈ ${verdict.slopeTail.toFixed(2)} — кривая загибается` : "Как только в коде появятся n и ops, они будут расти с n)."}
 </p>
</div>
</div>
)}

```

```

{/* живые переменные */}
<div className="mt-3 flex flex-wrap items-center">


```



```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}
```

```
const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
```

```

 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро ",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:3},
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:3,4:2},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{

```

```

 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм
 activeCodeLine: []
 },
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length -
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v
 const rawTo = vizString(vars.to) ?? (vizNumber(vars
 const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.l
 return byLabel ?? (Number.isFinite(Number(value))
 };
 const v = toId(rawV);
 const to = toId(rawTo);
 if (v === null) return null;
 const candidates = DFS_STEPS
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => step.currentNode === v && (
 return candidates.at(-1)?.index ?? null;
 });
 const [dfsAuto, setDfsAuto] = useState(false);

```

```

</button>
<button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>
<div className="grid grid-cols-1 md:grid-cols-5 g
 {/* Graph visualization */}
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.key
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
));
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u
 return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
 }}}
 {step.currentNode !== null && step.currentN
 const n = GRAPH_NODES.find(x => x.id ===

```

```

 { /* Code panel */ }
 <div className="bg-slate-950 rounded-lg border
 <div className="bg-slate-900 px-3 py-1.5 te
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono lea
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includ
 return <div key={i}
 className={`px-1 transition-all dur
 hl ? "bg-indigo-600/20 text-indig
 }`}>
 <span className="text-slate-600 mr-1
 {cl.line || ' '}
 </div>;
 })}
 </pre>
 </div>
 </div>

 { /* Stack */ }
 <div className="bg-slate-900/50 rounded-xl bo
 <div className="text-[10px] font-bold text-
 Стек DFS
 {step.s
 </div>
 <div className="flex flex-col-reverse gap-1
 {step.stack.length === 0
 ? <div className="text-[10px] text-slat
 : step.stack.map(({id, i}) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emera
 : "bg-slate-900 text-slate-400
 }`}>Вершина {GRAPH_NODES.find(n=>n.
 <span className="text-slate-500 m
 </div>
 })
 })
 </div>
 </div>
 </div>
 </div>

```

```

 log: string;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 ы листать вручную.";

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

 const adjList: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });
}

```

```

 minD = dist[n.id];
 u = n.id;
 }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
}

```

```

 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
 distances: Object.fromEntries(nodes.map((node) => [node, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node, null])),
 visited: Object.fromEntries(nodes.map((node) => [node, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}\n`
 }
 : baseStep;
 }
 : baseStep;

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-1px border-slate-700" >
 { /* Шапка симулятора */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white rounded" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Полный код
 </p>
 </div>
 </div>

 <div className="flex flex-wrap items-center justify-center" >
 <button
 onClick={() => { setIsPlaying(false); setCompilerLinked(false); }}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Вернуть к началу"}
 className="flex items-center gap-1 bg-slate-700 text-white px-3 py-2 rounded"
 >
 ↶
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Продолжить"}
 className={`flex items-center gap-1 px-4 py-2 rounded`}
 >
 ▶
 Продолжить
 </button>
 </div>
 </div>
 </div>
);

```



```

 <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="6" markerHeight="6" />
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" />
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 { /* Рёбра */
 edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);
 const isOptimal = currentStep.previous[edge.u] === edge.v;

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 2 : 1}
 />
 </g>
);
 });
 }
 </script>

```

```

 </g>
);
 }}}
 </svg>
 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
 </div>

 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/60 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2.5 flex-1 overflow-y-hidden">
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u;
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 <div className="flex items-center gap-2">

 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v;
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 {edge.v}


```

```

 <span className={`w-2 h-2 rounded
 {n.name}
 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
);
 }}}
</div>
</div>

```

```

const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState<boolean>(false);
const [speed] = useState<number>(800);
const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в кэше`,
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✓ Кэш-хит! fibМемо(${n}) уже посчитано`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвращаем n`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 memoMap[n] = n;
 return n;
 }

 const n1 = simulateFib(n - 1);
 const n2 = simulateFib(n - 2);
 const result = n1 + n2;
 memoMap[n] = result;
 return result;
 }

 simulateFib(10);
}, [runtime]);

```

```

 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 ([key, value]) => {
 const number = vizNumber(value);
 return number === null ? [] : [[Number(key), number]];
 }
))
 : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
 ? {
 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица м

```

```

 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
 : 'bg-emerald-600 text-white hover:bg-emerald-700'}
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
 </div>
</div>

```

```

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"

```

```

 })
 </div>

 { /* Tabs */
 <div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Eye className="w-4 h-4" />
 Стен-бай-стен состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

 { /* Tab Content */
 <div className="min-h-[320px] flex flex-col justi
 {activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl
 <h5 className="font-bold text-white text-sm

 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ п

```

```

 </div>
 </div>
 })
 </div>

 {/* Progress Bar */}
 <div className="mt-8 pt-6 border-t border-slate-800" >
 <div className="flex items-center justify-between" >
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 42/111: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useContext, useEffect, useMemo, useState } from "react";
import { Undo, Terminal, Route } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "../context";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

```



```

const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
};

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери");
 return;
 }
 const key = ekey(last, vId);
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 if (nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
 не цикл.`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $

```

```

 complete: keys.length >= C1_EDGES.length,
 });
 }, [vars]));

const path = live && live.path.length ? live.path : c
const visitedKeys = live ? live.keys : c1VisitedEdges
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[
const done = live ? live.complete : c1Done && visited
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёб
 шин ${odd.length} → это ${odd.length === 0 ? "ц
 : `Маршрут из кода: ${live.path.length ? live.pat
 ys.length}/${C1_EDGES.length}`
 : c1Msg;

const verdict =
 odd.length === 0
 ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт
 }
 : odd.length === 2
 ? { text: `2 нечётные (${odd.map((id) => C1_NODES
 в другой`, cls: "bg-amber-950/40 border-amber-
 : { text: `${odd.length} нечётных → эйлерова марш

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between
 <h4 className="text-base font-bold text-white f
 все рёбра ровно по одному разу</h4>
 <button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

```

```

 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="middle">deg {deg[n.id]}{isOdd ? " · нечёт" : ""}
 </g>;
 }}}
</svg>
<div className="absolute top-2 left-2 bg-slate-500 p-2">
 Ребёр: {visitedKeys.length}/{C1_EDGES.length}
</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done && !live
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"}`}>
 {msg}
</div>

{/* Критерий Эйлера */}
<div className={`p-3 rounded-xl border text-xs font-mono ${
 verdict ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : "bg-rose-950/40 border-rose-500/30 text-rose-50"}`}>
 Критерий Эйлера (для связного графа): {verdict}

 Степени: {C1_NODES.map((n) => `${n.label}=${deg[n.id]}`).join(" ")}
 две нечётны. Алгоритм Иера строит маршрут эйлерова графа.

</div>

<div className="flex flex-wrap items-center gap-2">

 /> из компилятора:
 {live ? (
 <>

 {live.path.map((id) => C1_NODES[id].label).join(" ")}


```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
// k/i/j/d приходят напрямую; номер строки произвольн
const [isPlaying, setIsPlaying] = useState(false);

const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
];

const codeLines = [

```

```

 log: ` Улучшение пути [${i}]→[${j}] чере
 ∞'.`,
 activeCodeLine: 7,
));
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ` Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
/**

```



```

</text>
</g>
);
}}
</svg>

```

```
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>
```

```

{ /* Список смежности */ }
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 `}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
rentStep.j === edge.v);
 return (
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

)
 })}
 </div>
)
 })}
 </div>
 </div>
</div>

```

```

 let bg = 'bg-slate-900/40 text-slate-100'
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-100'
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-100'
 else if (currentStep.i === -1 && !currentStep.j)
 bg = 'bg-indigo-950/60 text-indigo-100'
 else if (i === j) bg = 'bg-slate-900/40 text-slate-100'
 const shown = val === null ? '.' : val
 return (<td key={j} className={`p-2 ${bg} text-sm`}
))}
 </tr>
 </tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-100">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-2 ${
 item.e === item.l ? 'bg-blue-900/80 text-amber-400' :
 'bg-slate-900/40 text-slate-100'}`}>
 {item.l}
 {item.e}
 </div>
))}
 </div>
 </div>
</div>
</div>
</div>
);
}

```



```

 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

 const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

 const ADJ: number[][] = NODES.map(() => []);
 EDGES.forEach(([u, v]) => {
 ADJ[u].push(v);
 ADJ[v].push(u);
 });
 ADJ.forEach((a) => a.sort((x, y) => x - y));

 /** Палитра компонент. */
 const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

 interface DfsFrame {
 title: string;
 note: string;
 comp: number[]; // 0 = не покрашена, иначе н
 cur: number | null; // текущая вершина
 frontier: number[]; // стек/очередь
 comps: number; // счётчик компонент
 edge: [number, number] | null;
 }

 /** Покадровая раскладка DFS-обхода: ровно тот порядок,
```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v ? v : find(parent[v]));
 let comps = NODES.length;
 const push = (title: string, note: string, edge: [number, number] | null, merged: boolean) => {
 frames.push({ title, note, parent: [...parent], comps, edge, merged });
 push("Старт: каждая вершина сама по себе", `parent[v] = v`, null, false);
 };
 EDGES.forEach(([u, v], i) => {
 const ru = find(u);
 const rv = find(v);
 if (ru === rv) {
 push(`Ребро ${i}: (${u}, ${v}) – корни совпали`, `union ничего не меняет, comps остаётся ${comps}`, null, false);
 return;
 }
 parent[rv] = ru;
 comps -= 1;
 push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) = ${ru}, find(${v}) = ${rv}, comps стало ${comps}`, [u, v], true);
 });
 push(`Готово: comps = ${comps}`, `Все ${EDGES.length} ребра обработаны, связности`, null, false);
 return frames;
}

const DFS FRAMES = buildDfsFrames();

```

```

 }
 return s + 1;
 });
}, 900);
return () => {
 if (timer.current) window.clearInterval(timer.cur
};
}, [playing, maxStep]));

const goTo = useCallback((n: number) => setStep(Math.r

/* — переменные компилятора, которые рисуются НАПРЯМ
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р
const liveComp = useMemo(() => {
 const arr = vars ? (vizArray(vars.comp) ?? vizArray
 if (!arr) return null;
 return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
 const arr = vars ? (vizArray(vars.parent) ?? vizArr
 if (!arr || arr.length < NODES.length) return null;
 return arr.slice(0, NODES.length).map((v) => vizNum
}, [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumbe
const liveComps = vars ? (vizNumber(vars.comps) ?? vi
const liveFrontier = useMemo(() => {
 if (!vars) return null;
 const arr = vizArray(vars.stack) ?? vizArray(vars.d
 if (!arr) return null;
 return arr.map((v) => vizNumber(v)).filter((v): v i
}, [vars]);
/** Шаг из кода: i / e / step — двигает покадровую де
const liveStep = vars ? (vizNumber(vars.step) ?? vizN

useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))

```

```

 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">10
 <p className="text-xs text-emerald-300">10
 </div>
 </div>
 <div className="flex items-center bg-slate-950" >
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 py-3
 600 text-white shadow" : "text-slate-400 hover:
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (D
 </button>
 <button
 onClick={() => switchMode("dsu")}
 className={"flex items-center gap-1.5 px-3 py-3
 00 text-white shadow" : "text-slate-400 hover:
 >
 <Network className="w-3.5 h-3.5" /> DSU (pë
 </button>
 </div>
 </div>

 {/* панель управления */}
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setStep(0); setPlaying
 late-700 text-slate-300 hover:text-white tran
 <RotateCcw className="w-4 h-4" />
 </button>
 <button onClick={() => { setPlaying(false); goT
 g-slate-900 border border-slate-700 text-slate
 <ChevronLeft className="w-4 h-4" />
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1

```

```

const color = c > 0 ? COLORS[(c - 1) % COLORS.length];
const isCur = cur === n.id;
const inFrontier = frontier.includes(n.id);
return (
 <g key={n.id} onClick={() => goTo(step)} >
 {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />}
 {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="white" />}
 </g>
 <circle cx={n.x} cy={n.y} r={13} fill={color} />
 <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">{c}</text>}
</g>
);
}}}
{/* стек/очередь и DSU-таблица справа */}
<g>
 <text x={470} y={24} textAnchor="end" fill="black">
 {mode === "dfs" ? "стек DFS" : "parent[]"}
 </text>
 {mode === "dfs"
 ? frontier.slice(-8).map((v, i) => (
 <g key={i}>
 <rect x={440} y={32 + i * 20} width={20} height={10} fill={color} />
 <text x={453} y={44 + i * 20} textAnchor="left">{v}</text>
 </g>
))
 : (parentNow ?? []).slice(0, 10).map((pv, i) => (
 const color = COLORS[(setOf(parentNow).get(pv) - 1) % COLORS.length];
 return (
 <g key={i}>
 <rect x={432} y={32 + i * 15} width={10} height={10} fill={color} />
 <text x={436} y={41.5 + i * 15} textAnchor="left">{pv}</text>
 </g>
);
))
 }
</g>
);
}}}
```

---

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`, path: Array.from(vis) });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Добавляем узел ${v} в очередь`, path: Array.from(vis) });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посещаем узел ${u}`, path: Array.from(vis) });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Завершили обход графа`, path: Array.from(vis) });

 return steps;
}

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
```





```

 </svg>
 </div>

 { /* Structure View (Stack / Queue) */ }
 <div className="bg-slate-900 border border-
r">
 <div className={`absolute -top-3 left
 ${mode === 'dfs' ? 'border-
ld-300 border border-emerald-500'}`}>
 {mode === 'dfs' ? 'Стек вызовов' : ''}
 </div>

 <div className="flex flex-col gap-2 w-full"
 {step.queueOrStack && step.queueOrStack.length > 0}
 >
 <div key={idx} className={`w-full
 ${mode === 'dfs' ? 'bg-slate-800
text-emerald-200'}
 ${idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? 'border border-emerald-500'}
 `}>
 {mode === 'dfs' ? 'Вызов' : 'Элемент'}
 {idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? 'Закрыть' : ''}

 {step.callStack[idx] ? step.callStack[idx] : step.queueOrStack[idx]}

 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-2"
 { /* Controls */ }
 >
 <div className="flex bg-slate-900 border border-slate-800"
 >
 <button onClick={() => {setAutoPlay(!autoPlay)}}
 {"Назад" title="Назад" className="p-2 text-slate-400"}
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';

interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[idx], a[largest]] = [a[largest], a[idx]];
 idx = largest;
 } else break;
 }
 return a;
}
```

```

 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шме
 { name: 'Оля (Порез)', symptom: ' Глубокий порез',
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emo
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS
 });
 return arr;
})());

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? v
 const liveI = vizNumber(runtime?.variables?.i);
 const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNum
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : Str
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string>('');
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({

```

```
}, [busy, heap]));
```

```
const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * 10)]
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift))
 insertPatient(preset.name, prio);
};
```

```
const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};
```

```
// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);
```

```
 addLog(`    СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);
```

```
 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));
```

```
 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'healed' } : x));

 setTimeout(() => {
```

```

 res.push(
 <line key={`el${idx}`}
 x1={`${p.x}%`} y1={`${p.y + 4}%`}
 x2={`${lp.x}%`} y2={`${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={`${p.x}%`} y1={`${p.y + 4}%`}
 x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>
 </div>
)

```

```

 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:cursor-not-allowed text-white flex items-center px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white flex items-center transition-all cursor-pointer"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
 <div className="grid grid-cols-1 lg:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl p-4 min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-pulse' : ''}`}>
 <div className="absolute top-4 left-4 text-[1.2em] font-medium">
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

 <div className="text-emerald-400">
 </div>
 </div>
</div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

 { /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
 <div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-
 {healingPatient ? (
 <div className="flex flex-col items-cente
 <div className="relative">
 <span className="text-6xl block anim-l
 <span className="absolute -top-4 -right
 <span className="absolute -bottom-2 -
 </div>
 <div>
 <h4 className="text-white font-black
 Пациент: {healingPatient.name
 </h4>
 <p className="text-emerald-400 text-x
 Состояние: Стабилизация ({healingPa
 </p>
 <div className="flex justify-center g
 <span className="w-2 h-2 rounded-fu
 <Heart className="w-4 h-4 text-rose
 </div>
 </div>
 </div>
 </div>

```

```
/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу — мож
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
 const [step, setStep] = useState(0);
 const paused = useContext(DemoPauseContext);
 const period = Math.round(ms * SPEED);

 useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
 return () => clearInterval(t);
 }, [steps, period, paused]);

 return step;
}

export const C = {
 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
```



```

 {label}
 </text>
 })
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

---

ФАЙЛ 48/111: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7

```

```

 [{"D", "E"}],
 [{"F", "G"}, {"G", "H"}, {"F", "H"}],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) => e === v);

 return (
 <Svg caption={step === 0 ? "Граф распался на острова" : ""}>
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={`${ci}-${i}`} a={pos[u]} b={pos[v]} color={colors[ci]} />
))
)}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k} color={colors[ci]} />
 })}
 </Svg>
);
};

```

/\*\* Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
 const edges: [string, string, number][] = [
 ["A", "B", 1], ["B", "C", 1], ["C", "D", 1], ["D", "E", 1], ["E", "F", 1], ["F", "G", 1], ["G", "H", 1], ["H", "I", 1], ["I", "J", 1],
 ["A", "F", 2], ["B", "G", 2], ["C", "H", 2], ["D", "I", 2], ["E", "J", 2],
 ["A", "D", 3], ["B", "E", 3], ["C", "F", 3], ["D", "G", 3], ["E", "H", 3], ["F", "I", 3], ["G", "J", 3],
 ["A", "I", 4], ["B", "J", 4], ["C", "J", 4], ["D", "J", 4], ["E", "J", 4], ["F", "J", 4], ["G", "J", 4], ["H", "J", 4], ["I", "J", 4], ["J", "J", 4],
];
 return (
 <Svg>
 <caption>
 {step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи (дороги, слова, переходы)"
 : "Направление и вес"}
 </caption>
 {edges.map(([u, v, w]) => (
 <Edge key={`${u}-${v}`} a={pos[u]} b={pos[v]} color={C.active} weight={w} />
))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} r={12} label={k} color={C.active} />
))}
 </Svg>
);
};

```

```

const startX = (260 - raw.length * (boxW + 4)) / 2;

return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>

```

```

 {id}
 </text>
 </g>
)))
</Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["р – корень, х под ним", "Один поворот: "]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p":
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p":
]}
 />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верш
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
 />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, х вправо", "Поворот НИ
 frames={[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]

```

```
};
```

```
export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k === head ? "red" : "gray"} />
))}
 </Svg>
);
};
```

```
export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"], ["труссы", "носки"], ["штаны", "боты"], ["носки", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, { x: n.x, y: n.y }]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.includes(u) && placed.includes(v) ? "red" : "gray"} />
))}
 {nodes.map((n) => (
 <Node key={n.id} x={pos[n.id].x} y={pos[n.id].y} label={n.id} fill={placed.includes(n.id) ? "red" : "gray"} />
))}
 </Svg>
);
};
```

```

const step = useLoop(2, 1300);
const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
};
const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
const cut = step === 1;
return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "blue"} />
))}
 </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 cut && k === "C" ? null : (
 <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "blue"} />
)
))}
 </Svg>
);
};

```

```

const edges: [string, string][] = [["A", "B"], ["B", "C"], ["C", "A"]];
const step = useLoop(2, 1400);
const scc = ["A", "B", "C"];
const inScc = (u: string, v: string) => step === 1 && scc.includes(v);
return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из каждого вершины можно попасть в любую другую"}
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={inScc(u, v)} />
))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step === 1 ? "#000" : "#fff"} />
))}
 </Svg>
);
};

```

ФАЙЛ 50/111: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit";

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156], [130, 200], [80, 244]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине"
 {moving > 0 ? [0, 1, 2, 3].map(i => (
 <Edge a={pos[i]} b={pos[i + 1]} />
)) : []}
 {moving > 0 ? [0, 1, 2, 3].map(i => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={heap[i]} fill={step === 0 ? "#000" : "#fff"} />
)) : []}
 </Svg>
);
};

```

```

 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={40}>
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
)}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d} />
</Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f50"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },

```



```

return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.
)};
)}}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}}
 </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n
 fill={n.lvl === step ? C.active : C.idle} s

```

```
export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18} />
 <text x={25 + i * 30} y={29} textAnchor="middle" />
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
 })}
 </Svg>
);
};
```

```
export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
 })}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" fontSize={14} />
 </Svg>
);
};
```

```

 $\pi[i]$ — длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : fill}
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} text={C.dp[idx]}
 fontSize={10} fontWeight="700" fill="white" />
)}
 </g>
);
)}
)}
 </Svg>
);
};

<text x={130} y={122} textAnchor="middle" fontSize={14}>
 каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

```

 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

/** Список работающих мини-демо — его проверяет `npm run`
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```

```

 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint && <p className="text-[11px] text-white">
 <VizLiveStatus demoTitle={viz.title} className="text-white">
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 {/* Тот же id главы: развёрнутая демонстрация */}
 <VizChapterContext.Provider value={vizId}>
 <Component />
 </VizChapterContext.Provider>
 </div>
 </div>
 </div>,
 document.body
);
};

```

ФАЙЛ 53/111: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";

```

```

 setPos({ top: Math.max(8, Math.min(top, vh - height
 }, [anchor]));

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "unde
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo
 <div className="flex items-start gap-2 px-3 pt-

```

```
 </div>,
 document.body
);
};
```

---

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];
}
```

```

 return "stroke-slate-500";
 };

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a";
 if (e.id === 'AB' && step > 4) return "url(#arr";
 if (e.id === 'BC' && step === 5) return "url(#a";
 if (e.id === 'BC' && step > 5) return "url(#arr";
 if (e.id === 'CA' && step === 6) return "url(#a";
 if (e.id === 'CA' && step > 6) return "url(#arr";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}";
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}";
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}";
 }
};

```



Запускаем медленный, но мощный алгоритм  
 ных вершин:

```


 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра?
 </div>
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес: 0
 Вычисляем: $-2 + 0 - (-2) =$ -2
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ -1
 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ 1
 </div>
);
case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы не добавили.
 Все новые веса ребер в графе стали:
 </div>
);

```

```

 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 </defs>

```

```

 { /* Edges */
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) return null;

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 x2={v.x} y2={v.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(isS)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${edgeColorClass}`}
 style={{textHigh, textEmer}}
 {isS ? 'font-weight: bold;' : ''}
 >{getEdgeContent(e)}
 </text>
 </g>
);
 })}

```

```

 </div>

 <div className="w-full md:w-[320px] flex justify-center leading-relaxed text-sm text-slate-400" >
 { /* Log Box */
 <div className="bg-[#0f172a] p-5 rounded" >
 {getDesc()}
 </div>

 { /* Controls */
 <div className="flex bg-slate-900 border" >
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border-r border-slate-400 transition-colors" title="Сбросить" >
 <RotateCcw strokeWidth={3} />
 </button>
 <button onClick={() => setStep(2)}
 className="p-3 bg-slate-800 border-r border-slate-400 transition-colors" title="Вар на 10 мин" >
 <Undo strokeWidth={3} size={24} />
 </button>
 <button onClick={() => setStep(3)}
 className={`flex-1 font-bold text-white`}
 ${step === MAX_STEP ? 'bg-emerald-600/50' : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "Варить"}
 {step !== MAX_STEP && <Skip />}
 </button>
 </div>

 { /* Progress Dots */
 <div className="flex justify-center gap-2" >
 {Array.from({length: MAX_STEP + 1}, (_, i) => {
 <div key={i} className={`w-10 h-10 rounded-full ${
 i < step ? 'bg-indigo-900' : 'bg-slate-700'
 }`} />
 })}
 </div>
 }
 </div>

```

```

 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];
```

```
const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];
```

```
interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}
```

```
export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode === v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);
```

```

const adj: number[][] = Array.from({ length: 5 }, (
 initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();

```

```

 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
 }

 recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент
 null,
 true,
 sccId,
);

 setSteps(generatedSteps);
 setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,

```

```

<div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-2 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center gap-2"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"

```

```

const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f468c";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 />

```



```

 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && {
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 }}
 </g>
);
 }}}
</svg>

{step.transposed && {
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
 }}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

```

```

 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)))
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>

```

```

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([

```

```
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
}
```

```
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

 // --- PRIM STEPS ---
 const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)', 'F-H (9)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-7', '4-8', '4-9', '5-6', '6-7', '7-8', '8-9', '9-10', '10-11', '11-12', '12-13', '13-14', '14-15', '15-16', '16-17', '17-18', '18-19', '19-20', '20-21', '21-22', '22-23', '23-24', '24-25', '25-26', '26-27', '27-28', '28-29', '29-30', '30-31', '31-32', '32-33', '33-34', '34-35', '35-36', '36-37', '37-38', '38-39', '39-40', '40-41', '41-42', '42-43', '43-44', '44-45', '45-46', '46-47', '47-48', '48-49', '49-50', '50-51', '51-52', '52-53', '53-54', '54-55', '55-56', '56-57', '57-58', '58-59', '59-60', '60-61', '61-62', '62-63', '63-64', '64-65', '65-66', '66-67', '67-68', '68-69', '69-70', '70-71', '71-72', '72-73', '73-74', '74-75', '75-76', '76-77', '77-78', '78-79', '79-80', '80-81', '81-82', '82-83', '83-84', '84-85', '85-86', '86-87', '87-88', '88-89', '89-90', '90-91', '91-92', '92-93', '93-94', '94-95', '95-96', '96-97', '97-98', '98-99', '99-100', '100-101', '101-102', '102-103', '103-104', '104-105', '105-106', '106-107', '107-108', '108-109', '109-110', '110-111', '111-112', '112-113', '113-114', '114-115', '115-116', '116-117', '117-118', '118-119', '119-120', '120-121', '121-122', '122-123', '123-124', '124-125', '125-126', '126-127', '127-128', '128-129', '129-130', '130-131', '131-132', '132-133', '133-134', '134-135', '135-136', '136-137', '137-138', '138-139', '139-140', '140-141', '141-142', '142-143', '143-144', '144-145', '145-146', '146-147', '147-148', '148-149', '149-150', '150-151', '151-152', '152-153', '153-154', '154-155', '155-156', '156-157', '157-158', '158-159', '159-160', '160-161', '161-162', '162-163', '163-164', '164-165', '165-166', '166-167', '167-168', '168-169', '169-170', '170-171', '171-172', '172-173', '173-174', '174-175', '175-176', '176-177', '177-178', '178-179', '179-180', '180-181', '181-182', '182-183', '183-184', '184-185', '185-186', '186-187', '187-188', '188-189', '189-190', '190-191', '191-192', '192-193', '193-194', '194-195', '195-196', '196-197', '197-198', '198-199', '199-200', '200-201', '201-202', '202-203', '203-204', '204-205', '205-206', '206-207', '207-208', '208-209', '209-210', '210-211', '211-212', '212-213', '213-214', '214-215', '215-216', '216-217', '217-218', '218-219', '219-220', '220-221', '221-222', '222-223', '223-224', '224-225', '225-226', '226-227', '227-228', '228-229', '229-230', '230-231', '231-232', '232-233', '233-234', '234-235', '235-236', '236-237', '237-238', '238-239', '239-240', '240-241', '241-242', '242-243', '243-244', '244-245', '245-246', '246-247', '247-248', '248-249', '249-250', '250-251', '251-252', '252-253', '253-254', '254-255', '255-256', '256-257', '257-258', '258-259', '259-260', '260-261', '261-262', '262-263', '263-264', '264-265', '265-266', '266-267', '267-268', '268-269', '269-270', '270-271', '271-272', '272-273', '273-274', '274-275', '275-276', '276-277', '277-278', '278-279', '279-280', '280-281', '281-282', '282-283', '283-284', '284-285', '285-286', '286-287', '287-288', '288-289', '289-290', '290-291', '291-292', '292-293', '293-294', '294-295', '295-296', '296-297', '297-298', '298-299', '299-300', '300-301', '301-302', '302-303', '303-304', '304-305', '305-306', '306-307', '307-308', '308-309', '309-310', '310-311', '311-312', '312-313', '313-314', '314-315', '315-316', '316-317', '317-318', '318-319', '319-320', '320-321', '321-322', '322-323', '323-324', '324-325', '325-326', '326-327', '327-328', '328-329', '329-330', '330-331', '331-332', '332-333', '333-334', '334-335', '335-336', '336-337', '337-338', '338-339', '339-340', '340-341', '341-342', '342-343', '343-344', '344-345', '345-346', '346-347', '347-348', '348-349', '349-350', '350-351', '351-352', '352-353', '353-354', '354-355', '355-356', '356-357', '357-358', '358-359', '359-360', '360-361', '361-362', '362-363', '363-364', '364-365', '365-366', '366-367', '367-368', '368-369', '369-370', '370-371', '371-372', '372-373', '373-374', '374-375', '375-376', '376-377', '377-378', '378-379', '379-380', '380-381', '381-382', '382-383', '383-384', '384-385', '385-386', '386-387', '387-388', '388-389', '389-390', '390-391', '391-392', '392-393', '393-394', '394-395', '395-396', '396-397', '397-398', '398-399', '399-400', '400-401', '401-402', '402-403', '403-404', '404-405', '405-406', '406-407', '407-408', '408-409', '409-410', '410-411', '411-412', '412-413', '413-414', '414-415', '415-416', '416-417', '417-418', '418-419', '419-420', '420-421', '421-422', '422-423', '423-424', '424-425', '425-426', '426-427', '427-428', '428-429', '429-430', '430-431', '431-432', '432-433', '433-434', '434-435', '435-436', '436-437', '437-438', '438-439', '439-440', '440-441', '441-442', '442-443', '443-444', '444-445', '445-446', '446-447', '447-448', '448-449', '449-450', '450-451', '451-452', '452-453', '453-454', '454-455', '455-456', '456-457', '457-458', '458-459', '459-460', '460-461', '461-462', '462-463', '463-464', '464-465', '465-466', '466-467', '467-468', '468-469', '469-470', '470-471', '471-472', '472-473', '473-474', '474-475', '475-476', '476-477', '477-478', '478-479', '479-480', '480-481', '481-482', '482-483', '483-484', '484-485', '485-486', '486-487', '487-488', '488-489', '489-490', '490-491', '491-492', '492-493', '493-494', '494-495', '495-496', '496-497', '497-498', '498-499', '499-500', '500-501', '501-502', '502-503', '503-504', '504-505', '505-506', '506-507', '507-508', '508-509', '509-510', '510-511', '511-512', '512-513', '513-514', '514-515', '515-516', '516-517', '517-518', '518-519', '519-520', '520-521', '521-522', '522-523', '523-524', '524-525', '525-526', '526-527', '527-528', '528-529', '529-530', '530-531', '531-532', '532-533', '533-534', '534-535', '535-536', '536-537', '537-538', '538-539', '539-540', '540-541', '541-542', '542-543', '543-544', '544-545', '545-546', '546-547', '547-548', '548-549', '549-550', '550-551', '551-552', '552-553', '553-554', '554-555', '555-556', '556-557', '557-558', '558-559', '559-560', '560-561', '561-562', '562-563', '563-564', '564-565', '565-566', '566-567', '567-568', '568-569', '569-570', '570-571', '571-572', '572-573', '573-574', '574-575', '575-576', '576-577', '577-578', '578-579', '579-580', '580-581', '581-582', '582-583', '583-584', '584-585', '585-586', '586-587', '587-588', '588-589', '589-590', '590-591', '591-592', '592-593', '593-594', '594-595', '595-596', '596-597', '597-598', '598-599', '599-600', '600-601', '601-602', '602-603', '603-604', '604-605', '605-606', '606-607', '607-608', '608-609', '609-610', '610-611', '611-612', '612-613', '613-614', '614-615', '615-616', '616-617', '617-618', '618-619', '619-620', '620-621', '621-622', '622-623', '623-624', '624-625', '625-626', '626-627', '627-628', '628-629', '629-630', '630-631', '631-632', '632-633', '633-634', '634-635', '635-636', '636-637', '637-638', '638-639', '639-640', '640-641', '641-642', '642-643', '643-644', '644-645', '645-646', '646-647', '647-648', '648-649', '649-650', '650-651', '651-652', '652-653', '653-654', '654-655', '655-656', '656-657', '657-658', '658-659', '659-660', '660-661', '661-662', '662-663', '663-664', '664-665', '665-666', '666-667', '667-668', '668-669', '669-670', '670-671', '671-672', '672-673', '673-674', '674-675', '675-676', '676-677', '677-678', '678-679', '679-680', '680-681', '681-682', '682-683', '683-684', '684-685', '685-686', '686-687', '687-688', '688-689', '689-690', '690-691', '691-692', '692-693', '693-694', '694-695', '695-696', '696-697', '697-698', '698-699', '699-700', '700-701', '701-702', '702-703', '703-704', '704-705', '705-706', '706-707', '707-708', '708-709', '709-710', '710-711', '711-712', '712-713', '713-714', '714-715', '715-716', '716-717', '717-718', '718-719', '719-720', '720-721', '721-722', '722-723', '723-724', '724-725', '725-726', '726-727', '727-728', '728-729', '729-730', '730-731', '731-732', '732-733', '733-734', '734-735', '735-736', '736-737', '737-738', '738-739', '739-740', '740-741', '741-742', '742-743', '743-744', '744-745', '745-746', '746-747', '747-748', '748-749', '749-750', '750-751', '751-752', '752-753', '753-754', '754-755', '755-756', '756-757', '757-758', '758-759', '759-760', '760-761', '761-762', '762-763', '763-764', '764-765', '765-766', '766-767', '767-768', '768-769', '769-770', '770-771', '771-772', '772-773', '773-774', '774-775', '775-776', '776-777', '777-778', '778-779', '779-780', '780-781', '781-782', '782-783', '783-784', '784-785', '785-786', '786-787', '787-788', '788-789', '789-790', '790-791', '791-792', '792-793', '793-794', '794-795', '795-796', '796-797', '797-798', '798-799', '799-800', '800-801', '801-802', '802-803', '803-804', '804-805', '805-806', '806-807', '807-808', '808-809', '809-810', '810-811', '811-812', '812-813', '813-814', '814-815', '815-816', '816-817', '817-818', '818-819', '819-820', '820-821', '821-822', '822-823', '823-824', '824-825', '825-826', '826-827', '827-828', '828-829', '829-830', '830-831', '831-832', '832-833', '833-834', '834-835', '835-836', '836-837', '837-838', '838-839', '839-840', '840-841', '841-842', '842-843', '843-844', '844-845', '845-846', '846-847', '847-848', '848-849', '849-850', '850-851', '851-852', '852-853', '853-854', '854-855', '855-856', '856-857', '857-858', '858-859', '859-860', '860-861', '861-862', '862-863', '863-864', '864-865', '865-866', '866-867', '867-868', '868-869', '869-870', '870-871', '871-872', '872-873', '873-874', '874-875', '875-876', '876-877', '877-878', '878-879', '879-880', '880-881', '881-882', '882-883', '883-884', '884-885', '885-886', '886-887', '887-888', '888-889', '889-890', '890-891', '891-892', '892-893', '893-894', '894-895', '895-896', '896-897', '897-898', '898-899', '899-900', '900-901', '901-902', '902-903', '903-904', '904-905', '905-906', '906-907', '907-908', '908-909', '909-910', '910-911', '911-912', '912-913', '913-914', '914-915', '915-916', '916-917', '917-918', '918-919', '919-920', '920-921', '921-922', '922-923', '923-924', '924-925', '925-926', '926-927', '927-928', '928-929', '929-930', '930-931', '931-932', '932-933', '933-934', '934-935', '935-936', '936-937', '937-938', '938-939', '939-940', '940-941', '941-942', '942-943', '943-944', '944-945', '945-946', '946-947', '947-948', '948-949', '949-950', '950-951', '951-952', '952-953', '953-954', '954-955', '955-956', '956-957', '957-958', '958-959', '959-960', '960-961', '961-962', '962-963', '963-964', '964-965', '965-966', '966-967', '967-968', '968-969', '969-970', '970-971', '971-972', '972-973', '973-974', '974-975', '975-976', '976-977', '977-978', '978-979', '979-980', '980-981', '981-982', '982-983', '983-984', '984-985', '985-986', '986-987', '987-988', '988-989', '989-990', '990-991', '991-992', '992-993', '993-994', '994-995', '995-996', '996-997', '997-998', '998-999', '999-1000', '1000-1001', '1001-1002', '1002-1003', '1003-1004', '1004-1005', '1005-1006', '1006-1007', '1007-1008', '1008-1009', '1009-1010', '1010-1011', '1011-1012', '1012-1013', '1013-1014', '1014-1015', '1015-1016', '1016-1017', '1017-1018', '1018-1019', '1019-1020', '1020-1021', '1021-1022', '1022-1023', '1023-1024', '1024-1025', '1025-1026', '1026-1027', '1027-1028', '1028-1029', '1029-1030', '1030-1031', '1031-1032', '1032-1033', '1033-1034', '1034-1035', '1035-1036', '1036-1037', '1037-1038', '1038-1039', '1039-1040', '1040-1041', '1041-1042', '1042-1043', '1043-1044', '1044-1045', '1045-1046', '1046-1047', '1047-1048', '1048-1049', '1049-1050', '1050-1051', '1051-1052', '1052-1053', '1053-1054', '1054-1055', '1055-1056', '1056-1057', '1057-1058', '1058-1059', '1059-1060', '1060-1061', '1061-1062', '1062-1063', '1063-1064', '1064-1065', '1065-1066', '1066-1067', '1067-1068', '1068-1069', '1069-1070', '1070-1071', '1071-1072', '1072-1073', '1073-1074', '1074-1075', '1075-1076', '1076-1077', '1077-1078', '1078-1079', '1079-1080', '1080-1081', '1081-1082', '1082-1083', '1083-1084', '1084-1085', '1085-1086', '1086-1087', '1087-1088', '1088-1089', '1089-1090', '1090-1091', '1091-1092', '1092-1093', '1093-1094', '1094-1095', '1095-1096', '1096-1097', '1097-1098', '1098-1099', '1099-1100', '1100-1101', '1101-1102', '1102-1103', '1103-1104', '1104-1105', '1105-1106', '1106-1107', '1107-1108', '1108-1109', '1109-1110', '1110-1111', '1111-1112', '1112-1113', '1113-1114', '1114-1115', '1115-1116', '1116-1117', '1117-1118', '1118-1119', '1119-1120', '1120-1121', '1121-1122', '1122-1123', '1123-1124', '1124-1125', '1125-1126', '1126-1127', '1127-1128', '1128-1129', '1129-1130', '1130-1131', '1131-1132', '1132-1133', '1133-1134', '1134-1135', '1135-1136', '1136-1137', '1137-1138', '1138-1139', '1139-1140', '1140-1141', '1141-1142', '1142-1143', '1143-1144', '1144-1145', '1145-1146', '1146-1147', '1147-1148', '1148-1149', '1149-1150', '1150-1151', '1151-1152', '1152-1153', '1153-1154', '1154-1155', '1155-1156', '1156-1157', '1157-1158', '1158-1159', '1159-1160', '1160-1161', '1161-1162', '1162-1163', '1163-1164', '1164-1165', '1165-1166', '1166-1167', '1167-1168', '1168-1169', '1169-1170', '1170-1171', '1171-1172', '1172-1173', '1173-1174', '1174-1175', '1175-1176', '1176-1177', '1177-1178', '1178-1179', '1179-1180', '1180-1181', '1181-1182', '1182-1183', '1183-1184', '1184-1185', '1185-1186', '1186-1187', '1187-1188', '1188-1189', '1189-1190', '1190-1191', '1191-1192', '1192-1193', '1193-1194', '1194-1195', '1195-1196', '1196-1197', '1197-1198', '1198-1199', '1199-1200', '1200-1201', '1201-1202', '1202-1203', '1203-1204', '1204-1205', '1205-1206', '1206-1207', '1207-1208', '1208-1209', '1209-1210', '1210-1211', '1211-1212', '1212-1213', '1213-1214', '1214-1215', '1215-1216', '1216-1217', '1217-1218', '1218-1219', '1219-1220', '1220-1221', '1221-1222', '1222-1223', '1223-1224', '1224-1225', '1225-1226', '1226-1227', '1227-1228', '1228-1229', '1229-1230', '1230-1231', '1231-1232', '1232-1233', '1233-1234', '1234-1235', '1235-1236', '1236-1237', '1237-1238', '1238-1239', '1239-1240', '1240-1241', '1241-1242', '1242-1243', '1243-1244', '1244-1245', '1245-1246', '1246-1247', '1247-1248', '1248-1249', '1249-1250', '1250-1251', '1251-1252', '1252-1253', '1253-1254', '1254-1255', '1255-1256', '1256-1257', '1257-1258', '1258-1259', '1259-1260', '1260-1261', '1261-1262', '1262-1263', '1263-1264', '1264-1265', '1265-1266', '1266-1267', '1267-1268', '1268-1269', '1269-1270', '1270-1271', '1271-1272', '1272-1273', '1273-1274', '1274-1275', '1275-1276', '1276-1277', '1277-1278',
```

```

setVertices(prev => prev.map(v => v.id === 0 ? {
setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
}, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) - цикл',
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},

```

```

 description: 'Плесень из центра Токио (Е) тянет',
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
```





```

 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between two villages)
 () => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}.
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ...e, status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мосты',
 description: 'Теперь Красный и Синий колхозы одобрят мосты A-D с весом 5. Остальные мосты B-E(6) и C-F(9) будут отклонены.',
 type: 'warning'
 }, ...prev]);
 },
 // Step 4: Concurrently merge into single Kolkhoz
 () => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'green' })));
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { ...e, status: 'included' } : e));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский колхоз'
 }, ...prev]);
 }
);

```

```

 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стартовый (Heap) и захватываем соседей!',
 type: 'info',
 }]);
}
return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 деревень объединиться в колхоз.',
 type: 'info',
}]);
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 setLogs(introLog(algo));
};

/* ————— связь с Python-компилятором ————— */
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

/** Перемотка на кадр n: сброс и повтор первых n шагов */
const goToStep = useCallback((n: number) => {
 const target = Math.max(0, Math.min(currentSteps.length, n));
 setVertices(initialVertices);
 setEdges(initialEdges);

```

```

 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[0] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → одинаковый цвет */
const colorOfComp = (root: string | undefined): string | undefined => {
 if (!root) return 'none';
 let h = 0;
 for (let i = 0; i < root.length; i++) h = (h * 31 + root.charCodeAt(i)) % PALETTE.length;
 return PALETTE[h];
};

```

```

/** Всё, что код пользователя рассказал про граф. */
const live = useMemo(() => {
 if (!vars) return null;
 const parentRec = vizRecord(vars.parent) ?? vizRecord(0);
 const parentArr = vizArray(vars.parent) ?? vizArray(0);
 /** Метка множества вершины, как её записал код (будет использоваться для подсчёта) */
 const rawComp = new Map<number, string>();
 /** То же, но только если метка – вершина графа: по метке можно найти вершину */
 const parentIdx = new Map<number, number>();

```

```

 compOfVertex.set(id, String.fromCharCode(65 + cur
 });
 const takenRaw = vizArray(vars.mst) ?? vizArray(var
 const takenEdges = (takenRaw ?? []).map(parseEdge).
 const takenVertices = (vizArray(vars.taken) ?? vizA
 .map((x) => idOf(typeof x === 'string' ? x : vizN
 .filter((x): x is number => x !== null);
 const curU = idOf(vizString(vars.u) ?? vizNumber(va
 const curV = idOf(vizString(vars.v) ?? vizNumber(va
 const curW = vizNumber(vars.w);
 const heapRaw = vizArray(vars.heap) ?? vizArray(var
 const heapEdges = (heapRaw ?? []).map(parseEdge).fi
 const cheapestRaw = vizRecord(vars.cheapest) ?? viz
 const cheapestEdges = cheapestRaw ? Object.values(c
 l } => e !== null) : [];
 const total = vizNumber(vars.total) ?? vizNumber(va
 const any = compOfVertex.size > 0 || takenEdges.leng
 (curU !== null && curV !== null) || heapEdges.leng
 return any ? { compOfVertex, takenEdges, takenVerti
 // eslint-disable-next-line react-hooks/exhaustive-c
}, [vars]);

```

/\*\* Рёбра, которые рисуем: код пользователя важнее ка

```

const effEdges: Edge[] = useMemo(() => {
 if (!live) return edges;
 const key = (a: number, b: number) => `${Math.min(a
 const takenKey = new Set(live.takenEdges.map((e) =>
 const heapKey = new Set(live.heapEdges.map((e) => k
 const cheapKey = new Set(live.cheapestEdges.map((e)
 return edges.map((e) => {
 const k = key(e.u, e.v);
 const isCur = (live.curU === e.u && live.curV ===
 if (isCur) return { ...e, status: 'inspecting' as
 if (takenKey.has(k)) return { ...e, status: 'incl
 if (heapKey.has(k)) return { ...e, status: 'in_he
 if (cheapKey.has(k)) return { ...e, status: 'insp
 return { ...e, status: 'pending' as const };
 });

```

```

 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800/50" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 }
 >
 <GitGraph className="w-3.5 h-3.5" />

```

```

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
: activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
: activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
: "bg-rose-600 hover:bg-rose-500 text
)
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
 </div>
 </div>

 /* Что компилятор сообщил графу прямо сейчас */
 <div className="mb-6 flex flex-wrap items-center" >

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {live ? (
 <>
 {live.curU !== null && live.curV !== null && (

 ребро {String.fromCharCode(65 + live.curU)} - {String.fromCharCode(65 + live.curV)}

) : ''}
 </>
) : ''}
 {live.takenEdges.length > 0 && (

 в осто́ве: {live.takenEdges.map((e) => {
 {String.fromCharCode(65 + e.u)} - {String.fromCharCode(65 + e.v)}
 })}

)}
 </div>
)}

```

```

<div className="grid grid-cols-1 lg:grid-cols-3
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 ro
 active min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-sl
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5
 <div className="w-3 h-3 rounded-full bg
 </div>
 {activeAlgo === 'kruskal' ? (
 <
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 </>
) : activeAlgo === 'prim' ? (
 <
 <div className="flex items-center gap
 <div className="w-3 h-1 bg-sky-600
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 </>
) : (
 <
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>

```



```

 <text
 x={{u.x + v.x} / 2 + "%"}
 y={{u.y + v.y} / 2 + "%"}
 fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
 {effVertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 >
 </div>
)}}
</div>
</div>

```

```

 </div>
 </div>
 })

 <div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCircle c
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
 })}
 </div>
 </div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">

```

```

 </div>
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Award className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-r
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
ожении:</p>
 <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Code className="w-3.5 h-3.5 text-ind
 </p>
 <pre className="text-xs text-emerald-40
80 flex-1">

```

```
{`# 1. Инициализируем DSU
```

```
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]
```

```
def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False
```

```
2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
```

```

<pre className="text-xs text-emerald-400" style="font-family: monospace; font-size: 0.8em; margin: 0;">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
</pre>
</div>
</div>
</div>
}}

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style="display: flex; flex-direction: column; align-items: center; padding: 10px;">
 <div className="lg:col-span-1 space-y-4" style="width: 100%;">
 <div className="bg-slate-950 p-4 rounded-2xl" style="background-color: #0f172a; color: white;">
 <p className="text-slate-400 text-xs font-mono" style="margin: 0;">
 <Clock className="w-3.5 h-3.5 text-rose-500" style="display: block; margin: 0 auto;"/>
 </p>
 <p className="text-2xl font-black text-white" style="margin: 0;">
 Boruvka
 </p>
 <p className="text-slate-400 text-xs font-mono" style="margin: 0;">
 mst = []
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-2xl" style="background-color: #0f172a; color: white;">
 <p className="text-slate-400 text-xs font-mono" style="margin: 0;">
 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 </p>
 </div>
 </div>
 </div>
 </div>
)

```

```
 }}
 </div>

</div>
);
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
```

```
 * панели компилятора – только сам текст тем, чтобы:
```

```
 * 1) любую страницу можно было распечатать/сохранить
```

```
 * 2) внешний парсер (ридер, «версия для слабовидящих»
```

```
 * LLM-индексатор) получал чистый текст без служебных
```

```
 *
```

```
 * Все спойлеры <details> раскрыты, визуализация описания
```

```
 * показывает, какие у неё вкладки и какие переменные к
```

```
 */
```

```
const LIGHT_CSS = `
```

```
 :root { color-scheme: light; }
```

```
 body { background: #ffffff !important; color: #111827;
```

```
 .lite-wrap { max-width: 46rem; margin: 0 auto; padding:
```

```

 .replace(/<button[\s\S]*?</button>/gi, "")
 .replace(/<svg[\s\S]*?</svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/\\sdata-term-id="[^"]*" /g, "")
 .replace(/class="term-hint"[^"]*" /g, "")
 // ссылки на другие билеты должны вести в текстовую
 .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
 .replace(/class="ticket-link"/g, "");
}

```

```

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

```

```

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
 const list = useMemo(() => {
 if (!topicId) return chapters;
 const one = chapters.filter((c) => c.id === topicId);
 return one.length ? one : chapters;
 }, [topicId]);
}

```

```

return (
 <div className="lite-wrap">
 <style dangerouslySetInnerHTML={{ __html: LIGHT_GUIDE }} />

 <h1>Универсальное пособие: алгоритмы и структуры</h1>
 <p className="lite-meta">
 Билеты 1–24 • текстовая версия (без навигации,
 {topicId ? `одна тема: ${list[0]?.title} ?? topicId` : ''}
 полная интерактивная версия
 </p>

```

```

 {!topicId && (
 <nav className="lite-toc">
 <h3>Содержание</h3>

 {list.map((c) => (

```

```

 <p>
 {demo ? `Вкладка «${demo}»`
 {sync.vizTitle ? ` – ${sync.vizTitle}`
 {sync.stepNote ? <> Шаг демонстрации
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => (
 <li key={v.name}>
 <code>{v.name}</code> – {v.value}
 {v.range ? ` (${v.range})`

))}

)}
 </div>
);
}})
<p className="lite-meta" style={{ margin: 0; padding: 0; }}>
 В интерактивной версии эта демонстрация
 переменных перерисовывают её напрямую
</p>
</div>
)}

{
 ((quizzes[chapter.id] ?? []).length > 0 &&
 <div className="lite-quiz">
 <h3>Блиц: вопросы по теме</h3>

 {
 ((quizzes[chapter.id] ?? []).map((q, i) => (
 <li key={i}>
 <p>
 {q.question}
 </p>

 {
 q.options.map((option, j) => (
 <li key={j}>
 {j === q.correctIndex ? Правильно : }

```

```
const fmt = (value: DebugValue | undefined): string =>
 if (value === undefined || value === null) return "-";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "-∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "-∞";
 return value;
 }
 return String(value);
};
```

/\*\*

\* Таблица «ваших» расстояний, когда имена вершин в коде  
 \*  
 \* Схема демонстрации нарисована на фиксированном наборе  
 \* S...F у Форда–Беллмана). Если в коде пользователя верши  
 \* {0...6, v1...v9}, картинка физически не может их показат  
 \* просто пропадали, и казалось, что визуализация живёт  
 \* выводятся рядом, а подсказка объясняет, как связать  
 \*/

```
export function LiveDistTable({ dist, prev, done, demoIds }) {
 if (!dist) return null;
```

```
 const keys = Object.keys(dist);
 if (keys.length === 0) return null;
```

```
 const demoSet = new Set(demoIds);
 const unknown = keys.filter((key) => !demoSet.has(key));
 // Все вершины совпали со схемой – таблица не нужна,
 if (unknown.length === 0) return null;
```

```
 const doneSet = new Set((done ?? []).map((value) => S
```



```

 +{keys.length - shown.length} ещё

) : null}
 </div>
 </div>
);
}
```

---

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.png';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
 title: '× Фриказоид и Декстер Дуглас',
 desc: 'Декстер смотрит в CRT-монитор; синий электрический ток',
 highlight: 'неотрицательными',
 highlightColor: 'text-emerald-400',
 rest: 'весами ($w \geq 0$). Подсунь отрицательное ребро',
 complexity: ' $O((V+E) \log V)$ ',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Бэтмен на машине Форда (Форд-Беллман)',
 title: 'Бэтмен на машине Форда',
 }
];
```

```
 <div className={`mt-3 text-xs font-mono px-3`} >
 </div>
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 });
}
```

```
 className="p-2 -mr-2 text-slate-400 hover:cursor-pointer"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
 </>
);
};
```

---

ФАЙЛ 61/111: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Sparkles, FileDown, FileText, Code2, Loader2, } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}

/**
```

```

 shadow-indigo-600/30 whitespace-nowrap">
 <BookOpen className="w-4 h-4" /> Учебное по

 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 aria-label={compilerOpen ? "Скрыть панель P
 className={`shrink-0 flex items-center justify
 duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white bg-s
 }`}
 >
 <Terminal className="w-4 h-4" /> Python
 </button>

 <div id="download-menu" className="flex items
 <a
 id="download-guide-pdf-btn"
 href={`${import.meta.env.BASE_URL}export/
 download="guide_lite.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex-1 sm:flex-none flex items
 bg-indigo-600 hover:bg-indigo-500 text-whit
 n-200 whitespace-nowrap"
 title="Скачать PDF пособия: компактный ко
 >
 <FileDown className="w-4 h-4 text-indigo-

 <details className="relative shrink-0">
 <summary
 className="flex items-center justify-ce

```

```

 title="Скачать всё пособие одним авто
 >
 {busy ? <Loader2 className="w-3.5 h-3
 </button>
 </div>
 </details>
 </div>
 </div>
 </div>
 </header>
);
};
```

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useCallback, useContext, useEffect, use
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { PlanarityDemo } from "./PlanarityDemo";
import { VizChapterContext, emitVizDemo, useVizRuntime,
```

```
/**
```

```
 * Билет 7 – «Графы. Планарные. Покраска».
```

```
 *
```

```
 * Четыре вкладки:
```

```
 * 1. «Планарность» – K5 и K3,3 с подсвеченными пересече
```

```
 * 2. «Жадная покраска» – $\chi \leq \Delta + 1$: вершины красятся
```

```
 * достаётся наименьший цвет, не занятый соседями;
```

```
 * 3. «Двудольность» – проверка 2-раскрашиваемости BFS
```

```
 * нечётный цикл = двумя красками не обойтись;
```

```
 * 4. «Покраска рёбер» – $\chi' \leq \Delta + 1$ (Визинг), а для дв
```

```
 * $\chi' = \Delta$ (теорема Кёнига).
```

```
 *
```

```
 * Два графа на выбор: «основной» (8 вершин, $\Delta = 3$, ест
```

```
 * и «двудольный» (6 вершин, $\Delta = 3$, $\chi = 2$, $\chi' = 3$).
```

```

 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y:
];
 const g: Graph = { key, label: "Основной (8 вершин, Δ
 ", "5", "6", "7"] };
 g.adj = buildAdj(g);
 return g;
}

```

```

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[]
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]))
 adj.forEach((a) => a.sort((x, y) => x - y));
 return adj;
}

```

```

interface Frame {
 title: string;
 note: string;
 color: number[]; // 0 = не покрашена
 ecolor?: number[]; // 0 = не покрашено
 v: number | null;
 u: number | null;
 e: number | null;
 used?: number[];
 bad?: [number, number] | null;
}

```

```

/** Жадная покраска вершин: $\chi \leq \Delta + 1$. */
function greedyFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const delta = Math.max(...g.adj.map((a) => a.length))
 frames.push({
 title: "Старт: ни одна вершина не покрашена",
 note: `color = [0]·n. Порядок обхода – просто по н

```

```

 color: [...color], v: null, u: null, e: null,
 });
 for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.` ,
 color: [...color], v: start, u: null, e: null,
 });
 while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {
 if (color[u] === 0) {
 color[u] = 3 - color[v];
 queue.push(u);
 frames.push({
 title: `${g.names[v]} → ${g.names[u]}: цвет
 note: `Сосед не покрашен: color[${g.names[u]
 x}).join(", ") || "пуста"}].`,
 color: [...color], v, u, e: null,
 });
 } else if (color[u] === color[v]) {
 conflict = [v, u];
 frames.push({
 title: `Конфликт: ${g.names[v]} и ${g.names
 note: `Ребро (${g.names[v]}, ${g.names[u]})
 покрасить: $x \geq 3$.`,
 color: [...color], v, u, e: null, bad: conf
 });
 }
 }
 }
 }
 frames.push({
 title: conflict ? "Граф НЕ двудольный" : "Граф двуд

```

```

 });
 const chiE = Math.max(...ecolor);
 frames.push({
 title: `Готово:  $\chi' = \chi_E$  при  $\Delta = \Delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф:  $\chi' = \Delta = \Delta$  – это теорема 1 по Визингу.`
 : ` $\chi' = \chi_E$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$.

 $\chi' = \Delta + 1$ – классом 2 (например, K3 и любой граф с нечётным числом вершин).`,
 color: new Array(g.n).fill(0), ecolor: [...ecolor],
 });
 return frames;
 }

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
 { id: "planar", label: "1 · Планарность: K5 и K3,3", icon: null },
 { id: "greedy", label: "2 · Покраска вершин ($\chi \leq \Delta + 1$)", icon: null },
 { id: "bipartite", label: "3 · Двудольность (2 краски)", icon: null },
 { id: "edges", label: "4 · Покраска рёбер (Визинг, Кёниг)", icon: null },
];

export const PlanarColoringViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("greedy");
 const [graphKey, setGraphKey] = useState<GraphKey>("m");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 const graph = useMemo(() => makeGraph(graphKey), [graphKey]);
 const frames = useMemo(() => {
 if (tab === "greedy") return greedyFrames(graph);
 }, [tab, graph]);

```



```

const linked = Boolean(liveColor || liveEcolor || (liveV || liveU || liveE));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: {}, ecol: {} });
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number")
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV);
const curU = liveU !== null && Number.isFinite(liveU);
const curE = liveE !== null && Number.isFinite(liveE);
const delta = Math.max(...graph.adj.map((a) => a.length));
const usedColors = Array.from(new Set(graph.pos.map((v) => liveColor[v])));
const usedEcolors = Array.from(new Set(graph.edges.map((e) => liveEcolor[e])));

const go = useCallback((d: number) => {
 setPlaying(false);
 setStep((s) => Math.max(0, Math.min(maxStep, s + d)));
}, [maxStep]);

if (tab === "planar") {
 return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <Header />
 <Tabs tab={tab} setTab={setTab} />
 <PlanarityDemo />
 <p className="text-[11px] text-slate-500 mt-2">
 Формула Эйлера $V - E + F = 2$ и оценка $E \leq 3V - 6$
 </p>
 </div>
);
}

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 round
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((fram
)));
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a]
 stroke={isBad ? "#f43f5e" : ec > 0
 strokeWidth={isCur ? 6 : isBad ? 5
 strokeDasharray={ec === 0 && !isBad
 className="transition-all duration-
 opacity={isCur || isBad ? 1 : 0.85}
 />
 {tab === "edges" && ec > 0 && (
 <text x={{graph.pos[a].x + graph.po
 y={colorOf(ec)} fontSize="10" fontWeight="bo
 }}
 </g>
);
)}}
 {graph.pos.map((p, id) => {
 const c = nodeColor(id);
 const isCur = id === curV;
 const isNbr = id === curU;
 return (
 <g key={id}>
 <circle cx={p.x} cy={p.y} r={isCur ?
 "#38bdf8" : c > 0 ? colorOf(c) : "#475569"}
 <text x={p.x} y={p.y + 4} textAnchor=
 names[id]}</text>
 {c > 0 && tab !== "edges" && <text x=
```

```

 <div className="mt-3 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python: массив <code classNam
 igo-300">ecolor</code> для рёбер) и вершина

)}
 </div>
 </div>
);
};

const Header: React.FC = () => {
 <div className="flex flex-wrap items-center justify-b
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-fuchsia-600 text-white r
 <Palette className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold tex
 <p className="text-xs text-fuchsia-300">K5/K3,3
 </div>
 </div>
 </div>
 </div>
);

const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => vo

```

```

 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
 }
 const labels = ["A","B","C","D","E"];
 for(const p of pts) {

```

```

 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 z-20">
 $V=6, E=9 \Rightarrow 2V-4 = 8$ (двудольный)
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950/40 p-10">
 ▲ Запомни: K_5 и $K_{3,3}$ – два кита непланарности.
 Гомеоморфный K_5 или $K_{3,3}$ – он непланарен. Теорема Кюри.
</div>
</div>

```

```

import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохран
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;
 /** Перейти к странице пособия (посмотреть визуализац
 /** Кнопка «открыть страницу в пособии» – не нужна, е
 onOpenGuide?: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) =>
 setStderr: (options: { batched: (message: string) =>
 setStdin: (options: { stdin: () => string | number |

```



```

 tip: "LIFO, как рекурсия DFS: append = push, pop = push",
 code: 'st = []\nst.append("A") # push\nst.append(
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop",
 code: 'from collections import deque\nq = deque(
 t(q))\n',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь пр",
 code: 'import heapq\nh = []\nheapq.heappush(h,
 w, v)\n',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечн",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint
 },
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в стр",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}
 },
],
},
];

```



```

));
 /** «Глаз» – переключатель: эталонный код демо ↔ инициализация
 const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved.isRunning;
 });
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Код выполняется");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 /** Принудительный повтор (глаз / Ctrl+Enter), даже если код не готов
 const [traceRequest, setTraceRequest] = useState(0);
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);
 /**
 * Панель жива? Заккрытие (крестик, «глаз» в шапке, см. код выше)
 * компонент, но `runPythonAsync` к тому моменту может быть выполнен
 * этого флага доехавший до конца прогон публиковал состояние
 * после закрытия: панель исчезла, а визуализация прогона
 * кода, которого на экране больше нет.
 */
 const aliveRef = useRef(true);
 useEffect(() => {
 aliveRef.current = true;
 return () => {
 aliveRef.current = false;
 };
 }, []);

 // Код хранится по страницам, поэтому при открытии панели
 // бывает: рассинхрон возможен только после смены вкладки
 const [desyncedFrom, setDesyncedFrom] = useState<string>("");
 const prevTplRef = useRef({ base: initTemplate, full: initTemplate });

 /** Активная трасса. input нужен, чтобы изменение stdin

```

```

 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: te
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – бу
 }
}, [code]);

/* — Пошаговый отладчик —————

/** Связь больше не зависит от эталонных строк: совпа
const hasViz = sync.variables.length > 0;

/**
 * Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
 * (там уже виден append/st[i][j]=...), но индексы цикл
 * события: после st[i][j]= Python уже мог увеличить
 */

```

```

 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

```

```

 setOutput("В редакторе пусто – выполнять нечего.\n
 «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализацией");
 setOutputOpen(true);
 setRunning(false);
 return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Загружаю код");

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

 const raw = await runtime.runPythonAsync(buildDebugOptions());
 // Панель могли закрыть, пока Python считал: выходим
 if (!aliveRef.current) return;
 const result = JSON.parse(String(raw)) as DebugResult;
 const printed = clipOutput([...stdout, ...stderr]);

 if (result.steps.length === 0) {
 // Шагов нет (только комментарии, синтаксическая ошибка)
 // заменяется пустым листингом, а визуализация
 // показывает собственный встроенный сценарий (с подсветкой)
 setDebug({ result, idx: 0, src: source, input, output });
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(
 result.error
 ? `Ошибка:\n${result.error}`
 : printed || "В коде нет исполняемых строк"
);
 }
} catch (e) {
 console.error(e);
 setRuntimeReady(false);
 setOutputOpen(false);
 setOutput("");
 setRunning(false);
}

```

```

useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) retu
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
 };
 // capture-фаза: чтобы глобальный обработчик навига
 window.addEventListener("keydown", onKey, true);
 return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

```

```

 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) step
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, st
 const next = current.slice(0, start) + padded + c
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + p
 });
 return next;
 });
}, []));

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh
 -[520px] 2xl:w-[600px] flex flex-col bg-slate-9
 style={typeof window !== "undefined" ? {
 ...(width && window.innerWidth >= 1024 ? { width
 ...(height ? { height: Math.min(height, window.
 } : undefined)
 }
 >
 {/* Верхний край меняет высоту на телефоне и деск

```

```

 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={({e}) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 })
 >
 <div className="absolute left-1/2 top-1/2 -tr
 digo-500 transition-colors" />
 </div>
})
/* — Заголовок панели
<div className="shrink-0 flex flex-wrap items-cen
 <span className={`w-2 h-2 rounded-full shrink-0
 <Terminal className="w-4 h-4 text-emerald-400 s
 <span className="text-[13px] font-bold text-whi
 <span className="hidden sm:inline text-[10px] t

 <div className="ml-auto flex flex-wrap items-cen
 <Tooltip
 side="bottom"
 content={running ? "Python выполняется..." :
 д и запустить автопроход"}
 >
 <button

```

```

 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор стане
обственном встроенному демо. Кнопка ↺ верн
 : "Вставить полный эталонный код этого
удет заменён."
 }
 >
 <button
 type="button"
 onClick={() => {
 const next = !refOn;
 setRefOn(next);
 // «Убрать эталон» – значит убрать код
 // выполняется, поэтому демонстрация бо
 // значений и честно показывает встроенн
 // странице скажет об этом явно).
 setCode(next ? template : "");
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setTraceRequest((n) => n + 1);
 }}
 className={`p-1.5 rounded-lg transition-c
 refOn
 ? "bg-indigo-500/30 text-white ring-1

```



```

 stripTab === "vars" ? "bg-slate-800 text-white" :
 }` }
 >
 <Variable className="w-3.5 h-3.5" /> Переменная
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5`}
 stripTab === "hints" ? "bg-slate-800 text-white" :
 }` }
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказка
</button>
<div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный узел"
 : "На выбранной странице нет интерактивных элементов"
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center`}
 hasViz ? "text-emerald-400" : "text-slate-400"
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

</Tooltip>
</div>
</div>

{stripTab === "vars" ? {

```

```

<div className="flex flex-wrap items-center"
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bold"
 <code className="font-mono text-emerald-300"
 {v.name} />

 {v.role}
 {v.range && <span className="block text-emerald-300"
 {v.range} />

 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md font-bold text-emerald-300 hover:bg-emerald-300"
 >
 {v.name}
 </code>
 </Tooltip>
)
)}
 {sync.stepNote && (
 <Tooltip side="bottom" content={`⚠️ War ⚠️`}
 <span
 tabIndex={0}
 className="cursor-help inline-flex text-[10px] text-slate-400 hover:text-white"
 >
 <ArrowRightLeft className="w-3 h-3" />

 </Tooltip>
)
)}
</div>
)}

```

```

 <span
 className="inline-flex items-center gap-1
 ont-bold text-amber-300 shrink-0"
 title="Трасса ниже построена по предыдущей"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin" />

)}

 {code.trim() === "" ? "пусто – ничего не выведено" : ""}

</div>

<textarea
 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setRefOn(false);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-[2] min-h-[110px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-amber-300"
 placeholder="Напишите Python-код... (пустой редактор)"
/>

{showTrace && debug && (
 <div className={`flex flex-col border-t border-amber-300`} >
 {/* панель управления шагами */}
 </div>
)}

```

```

result.steps.length - 1); }} disabled={!curr
d text-slate-400 hover:text-white hover:bg-
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет — исправьте ошибку и начини
)}

 {debug.result.truncated && (
 <Tooltip content={`Выполнение остановлен
 <span tabIndex={0} className="cursor-l
rounded-full px-1.5 py-0.5">
 ЛИМИТ

 </Tooltip>
)}
 <Tooltip content={hasViz ? "Визуализация :
st... Эталонный текст не обязателен — пишите
нет имён, связанных с демонстрацией; Python
 <span
 tabIndex={0}
 className={`cursor-help inline-flex i
"border-emerald-500/40 bg-emerald-500/10 t
 >
 {hasViz ? <Link2 className="w-3 h-3" .
 {hasViz ? "имена → демо" : "свободный

</Tooltip>
 <Tooltip content={traceOpen ? "Свернуть л
и переменные текущего шага."}>
 <button

```

```

 className={`flex whitespace-pre px-1
 active ? "bg-emerald-500/15 text-emerald-300" : ""
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-emerald-500"
 }`}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 }}}
</div>

```

```

{/* панель переменных – «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden" >
 <div className="flex items-center gap-1.5" >
 <Variable className="w-3 h-3 text-indigo-500" />
 Переменные на этом шаге

 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap" />
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500" />
)}
</div>
)}
<div className="mt-1 flex flex-wrap gap-1" >
 {debugOrdered.map(([name, value]) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return (
 <span
 key={name}

```

```

 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono
 />
</div>
</div>

```

*/\* — Вывод*

```

<div className="shrink-0 border-t border-slate-800"
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2"
 >
 <span className="inline-flex items-center gap-2"
 <Terminal className="w-3.5 h-3.5 text-emerald-500"
 />
 <span className="inline-flex items-center gap-2"
 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500"
 {outputOpen ? "Закрыть" : "Открыть"}
 />
 />
 </button>
 {outputOpen && (
 <pre
 data-testid="compiler-output"
 className="flex-1 min-h-[70px] overflow-auto text-slate-300"
 >
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-slate-800"
</div>
</aside>
);
}

```

```
const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
 text: 'Я голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍝', color: 'blue',
 text: 'Пожалуйста, повар, дай мне жареную картошку и курицу, пожалуйста!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
 text: 'Пожалуйста, повар, дай мне котлету с картошкой и сметаной, пожалуйста.', animState: 'idle' },
]);
```

```
const runtime = useVizRuntime();
const liveQ = vizArray(runtime?.variables?.q);
const liveV = vizString(runtime?.variables?.v);
const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: "🐍",
 color: "from-emerald-500/20 to-emerald-600/20",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;
```

```
const [servedHistory, setServedHistory] = useState<string[]>([]);
const [isServing, setIsServing] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);
```

```
const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
// ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжело!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setQueue([...queue, ...liveQ]);
}, [liveQ, isServing, queue]);
```

```

// Step 2: Guest leaves with happy face, queue sh
setQueue(prev => prev.map((c, idx) => idx === 0 ?

setServedHistory(prev => [`${headGuest.emoji} ${h

setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
}, 400);
}, 900);
}, [queue, isServing]));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', col
 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ', col
 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', col
 без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState
 }, 500);
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-in
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 tex

```



```

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12"

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}
 <div className="absolute top-4 left-4 text-[1.2em] font-mono"
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && {liveQ}}
 </div>

 <div className="flex-1 flex flex-col justify-between"
 <div className="flex items-end justify-between"

 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center"
 <div className="relative"
 <div className="absolute -top-10 left-1/2"
 <div className="w-1.5 h-6 bg-slate-500 rounded"
 <div className="w-2 h-4 bg-slate-400 rounded"
 </div>
 {liveQ}
 </div>
 <div className="text-center">
 {liveQ}

 ПОВАР

 </div>
 </div>
 </div>

 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
 <div className="flex flex-col items-center"
 {liveQ}

 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>

```

```

 relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[10px]
 ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-slate-900">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">

```

```
 {idx === 0 ?
 {entry}
 </div>
)})
 </div>
</div>
);
};
```

---

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { useState, useEffect, useContext, useMem
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,
import { Play, RotateCcw, Plus, Volume2, Info } from 'l
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```
export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
```

```

 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[liveState]) {
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail).length : 0;
 setSimulationStep(liveStep <= 0 ? 0 : failReady ? failReady : total);
 }
 if (liveQueue) {
 const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
 if (q.length) setBfsQueue(q);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]);

const compilerLinked = Boolean(liveWordsSig || liveState);

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);

```

```

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].child);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова S, чтобы расставить fail-ссылки!`
);
 };

 useEffect(() => {
 buildInitialTrie(words);
 }, [words]);

```

```

 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на гл
 очке суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полны
 ь полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.ch

 for (const [char, u] of Object.entries(rNode.children)
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char]
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char]
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is terminal) {

```



```
<div className="absolute -left-3 top-1/2 -transform-rotate-90 border-b-[12px] border-b-transparent" />
```

```
<div className="flex items-center space-x-2 text-slate-800">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
 Прямое вещание из аквариума:
</div>
```

```
<p className="text-slate-200 text-base leading-relaxed">
 {speechText}
</p>
```

```
<div className="mt-4 pt-3 border-t border-slate-200">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Борется с вирусом!' : 'Восстановлен!'}
 </div>
</div>
```

```
<div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1 shadow-lg shadow-emerald-900/40 transition-colors" />
 <Play className="w-4 h-4" /> Запустить симуляцию
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-2 py-1 shadow-lg shadow-blue-900/40 transition-colors" />
 Шаг BFS

 {simulationStep}

)}
</div>
```



```

 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
</defs>

```

```

 { /* Draw Edges */ }
 { Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 })}
)}
}

```

```

 { /* Draw Fail Links */ }
 { simulationStep > 0 && Object.values(nodes)

```

```

 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
)}
 </g>
)
}

```

```

 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

```

```

 {/* Таблица Вершин и суффиксных ссылок */}
 <div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

```

```

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >
 <th className="py-2 px-3" >ID Вершины</th>
 <th className="py-2 px-3" >Символ</th>
 <th className="py-2 px-3" >Родитель</th>
 <th className="py-2 px-3" >fail-ссылка</th>
 <th className="py-2 px-3" >term_link</th>
 <th className="py-2 px-3" >Дети</th>

```

```

</td>
<td className="py-2.5 px-3 text-slate-500"
 {node.id === 0 ? '-' : `#${node.id}`}
</td>
<td className="py-2.5 px-3"
 {node.id === 0 ? (

 {node.id}

) : (
 <span className="bg-indigo-950 text-white"
 >#{node.fail}
)}
</td>
<td className="py-2.5 px-3"
 {node.id === 0 || node.term_link ? (
 <span className="text-slate-500"
 >#{node.term_link}
) : (
 <span className="bg-rose-950 text-white"
 >#{node.term_link}
)}
</td>
<td className="py-2.5 px-3 text-slate-500"
 {Object.keys(node.children).length === 0 ? '∅' : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
</td>
<td className="py-2.5 px-3"
 {node.words.length === 0 ? (

 {node.words}

) : (
 <span className="bg-emerald-900 text-white"
 >#{node.words.join(', ')}
)}
</td>

```

---

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber, vizArray } from '../viz-runtime';

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range in array
 result?: number; // partial result
}

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

```

 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 state: { ...tree }, highlightNodes: [v], mergeChildren:
 });
 go(1, 0, n - 1);
 steps.push({ id: id++, desc: `Дерево построено! Корень:
 Highlight: [0, n - 1] });
 return steps;
 }

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v],
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v],
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v],
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 mergeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v],
 return res;
 }

```

```

 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`
 , arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`
 , highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```
function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 treeState, v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
]

```

```
];
```

```
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",

```

```

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
const p = Player({ steps, color });
const runtime = useVizRuntime();
const liveV = vizNumber(runtime?.variables?.v);
const liveTree = vizArray(runtime?.variables?.tree);
const liveI = vizNumber(runtime?.variables?.i);

if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const fullTree = useMemo(() => buildTreeFull(arr), [a
const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
const isCompilerActive = liveV === nd.v;
const isHigh = step.highlightNodes.includes(nd.v) |
const isChild = step.mergeChildren?.includes(nd.v);

// Live compiler value or step value or pre-calcula
let displayVal: number | string = nd.val !== null ?
if (liveTree && liveTree[nd.v] !== undefined && typ
displayVal = liveTree[nd.v] as number;
}

```



```

 Исходный код
 </div>

```

```

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b border-slate-500">
 <div className="text-[11px] font-bold text-slate-400">
 Исходный массив

 </div>
 <div className="flex flex-wrap gap-1 justify-center">
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.arrayStart;
 const isLiveI = liveI === i;
 return (
 <div key={i} className={`flex flex-col items-center justify-center gap-1
 text-amber-400 ring-2 ring-amber-400/40 ${inRange ? 'bg-emerald-950 border-emerald-500' : ''}`}>
 {v}
 {i}
 </div>
);
 })}
 </div>
 </div>

```

```

 {/* Tree */}
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950 border border-slate-500">
 <div className="scale-[0.85] origin-top min-w-max">
 </div>
 </div>

```

```

 {/* Code */}
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (

```

```

 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 {/* Progress bar */}
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12
 const [customInput, setCustomInput] = useState('5, 8,
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => pars
 if (parsed.length >= 2 && parsed.length <= 16) {

```

```

 <button type="submit" className="bg-indigo-500 text-white px-4 py-2 rounded-md">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => {

 {v}

))}
 {arr.length}
 </div>
 </form>

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
 <label className="text-[10px] font-bold uppercase text-slate-400">Запрос</label>
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 От
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 До
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 />
 </div>
 </div>
 </div>
 </div>
)
}

```

```

 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
 })

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↑"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
 })

{view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin
 Каждый узел берёт <code className="bg-sla
 к получает <code className="bg-slate-900 px

```

```

 </div>
 <p className="text-xs text-slate-300 mb-3">
 Стартуем с двух указателей (l и r) на левом и правом кончике массива.
 Сдвигаем левый указатель вправо пока l < r. Если l < r, то мы нашли
 элемент, который больше или равен x. Если l > r, то мы нашли элемент,
 который меньше x. Если l == r, то мы нашли элемент, который равен x.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-top border-slate-200">

 <div className="flex justify-between border-bottom border-slate-200">

 <div className="flex justify-between">

 00 font-bold">x сложно</div>

 00 font-bold">x сложно</div>

 00 font-bold">x сложно</div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

 { /* Verdict */ }
 <div className="bg-slate-950 p-5 rounded-xl border border-slate-800">
 <h4 className="font-bold text-blue-400 mb-2">
 <p className="text-sm text-slate-300 leading-relaxed">
 <strong className="text-white">Итеративный способ
 <strong className="text-white">рекурсивный (сверху-вниз)
 </p>
 </div>
 </div>
}
</div>
);
}

```

```

 <h3 className={`text-xs font-bold text-slate-400`}
 >
 <Compass className="w-4 h-4 text-indigo-400"
 </h3>
 {!compact && (
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute left-2 top-1/2 -tran
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-bo
 e-500 focus:outline-none focus:border-indigo
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
)}
 {!compact && (
 <span className="text-[10px] text-slate-500 i
 Порядок чтения: билеты 1 → 24

)}
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4"
 {groups.length === 0 && <p className="text-sm t

 {groups.map(({ section, items }) => (
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5

```

```
 }}

 {!compact && (
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt
 isSelected ? "text-indigo-400" : "text-gray-400"}
 />
)}
 </button>
);
 }}}
</div>
</div>
)}
</div>
</aside>
);
};
```

---

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft }
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
}
```

```
// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '=' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты.");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};
```



```

const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(`  Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`  Выбран самый "тяжелый" кандидат:`);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.1 },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный пирог"', probability: 0.2 },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"', probability: 0.3 }
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

```

```

<button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Queue
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </button>

<button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-white'}
>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Heap
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>

 Beam Search

 </div>

 Beam Search

</button>

```

```

 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border border-
 imate-fade-in">
 <span className="inline-block w-2 h-2 round
 {popMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стор
 <p className="text-xs mt-1">Добавьте гр
 </div>
 } : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map((plate, ind
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop

```

```

 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{dequeueMessage && (
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
)}

```

```


 })
 {
 <span className={`font-bold text-
 {person.name}

 <span className={`text-[10px] font
 isFirst ? 'bg-indigo-500/30 tex
 }`}>
 {isFirst ? 'Получает сын' : `В

 </div>
 });
 }}}

 <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
</div>
)}
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
)}

```

```

{ /* HEAP TAB */ }
{ activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e

```

```

 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 text-sm font-medium text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

 {heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">

 {heapMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl">

```

```

 <span className="text-[10px
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

 }}
</div>
</div>

<div className="flex items-center
 {/* Custom progress bar */}
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
 }}}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```

```

 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Один
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {[
 { id: "cases" as Tab, label: "1 · Механика: Z",
 { id: "big" as Tab, label: "2 · Большое дерев
 { id: "play" as Tab, label: "3 · Собери сам
]).map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font
 tab === t.id ? "bg-indigo-600 text-white"
 }`>
 >
 {t.label}
 </button>
 })}
 </div>

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с пра
 </p>
 }}
</div>
);
};

export default SplayPlayground;

```



```

 const n2: TNode = { key: 2, left: n1, right: leaf(7) };
 const n4: TNode = { key: 4, left: n2, right: leaf(7) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right: leaf(7) };
 const n4: TNode = { key: 4, left: n2, right: leaf(7) };
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(7) }, right: leaf(7) }),
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: leaf(7) };
 return { key: 8, left: n4, right: leaf(9) };
 },
},
];

```

/\* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode | null => {

```

const p = path[path.length - 2];
const g = path.length >= 3 ? path[path.length - 3] : null;

if (!g) return { node: x.key, kind: "Zig" };

const xLeft = p.left === x;
const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
return { node: x.key, kind: "Zig-Zag (сначала нижняя ветвь)" };
}

/* ----- раскладка ----- */

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; edges: [number, number][] } {
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth + 1)));
 const colW = 46;

```

```

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.node === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold text-slate-400">
 Как играть
 <p className="text-[12px] text-slate-400 lead-4">
 Клик по узлу = один поворот с его родителем

 (1 поворот)

 поворотов выбираешь сам, разбор покажем в конце
 </p>
 </div>
 <div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESET.length)}
 className="flex items-center gap-1.5 px-3 py-2 text-sm font-bold transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>
 </div>
 </div>
 </div>
);

```

```

 style={{ transition: "all 700ms cubic-bezier(0.4, 0.6, 0.2, 0.8)"}
 }
);
 }
}

{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0.6, 0.2, 0.8)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 stroke-width={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
})}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button

```

```

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 `)}
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`
 }
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

)
)}

)}
 </div>
)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
 </div>
)}
 </div>
);
};

```

---

```

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",

```

```

label: "Zig-Zig",
when: "х и р — дети с ОДНОЙ стороны (оба слева или оба справа)",
rotations: "2 поворота, первым — верхний",
keys: { х: 20, р: 40, г: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 — 40",
 text: "Главная хитрость этого случая: первым крутит 60, а не 40",
 focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему по
убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 — 20",
 text: "А теперь обычный одиночный поворот — тот же самый",
 focus: ["p", "x"],
 },
]

```

```

 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 }

```



```

 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill="white">
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
)}
)}
}

```

```

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={`-${a}-${b}-${i}`}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 />
);
})}

```

```

 }}
 </g>
);
 }
}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizSt
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(
const lastSig = useRef<string | null>(null);

```

```

useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;
 setPlaying(false);

```

```

 >
 {c.label}
 </button>
)))
 </div>

<div className="mb-3 text-xs sm:text-[13px] text-
 Кор,
 ({active.rotat
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала
<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current

 <span className="text-sm font-bold text-white
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading
</div>

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

 {/* обход слева направо – одинаковый на всех ка
 <div className="mt-2 pt-2 border-t border-slate
 <div className="flex items-center gap-1.5 fle

```

```

 d: hover: text-slate-300 text-xs font-bold transition-colors
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2
 text-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 : {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-2
 slow
 ? "bg-emerald-600/20 border-emerald-500 transition-colors"
 : "bg-slate-800 border-slate-700 text-slate-300"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}

```

```

) : (

 запусти код из панели Python: переменная <с
 случай,
 а <code className="font-mono text-indigo-300">

)}
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры н
 </p>
</div>
);
};

```

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect, useMemo, useRef, useState }

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный пов
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, по
 * rot – поворот: зелёным подсвечены рёбра, которых н
 * Рядом с логом всегда висит врезка «Механика одного п
 * На каждом кадре – живая автопроверка in-order (сорти
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, col
 */

export interface SN {

```

```

 cur = cur.right;
 }
}

```

/\*\* Поднять child над его родителем (один атомарный пов

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" — для diff между кадрами. \*/

```

export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
}

```

```

const steps: SplayStep[] = [];
if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */

```

```

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key
);
 } else {

```



```

while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
}
return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...inorderKeys(t.left), t.key, ...inorderKeys(t.right)] : []);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # B" },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" },
 { line: 10, text: " rotate(x over g)" },
];

/* ----- врезка «механика одного поворота» ----- */

const MechanismInset: React.FC<{ step: SplayStep | null }> = ({ step }) => {
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border border-slate-700 p-4">
 {isCut ? "Вырезание" : isRot ? "Поворот" : ""}
 </div>
);
};

```

```

<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font-weight="bold">
 <line x1="70" y1="46" x2="45" y2="86" stroke="black" stroke-width="2" />
 <line x1="70" y1="46" x2="95" y2="86" stroke="black" stroke-width="2" />
 <line x1="45" y1="86" x2="27" y2="126" stroke="black" stroke-width="2" />
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669" />
 <text x="70" y="50" textAnchor="middle" fill="black" />
 <circle cx="45" cy="86" r="15" fill="#1e293b" />
 <text x="45" y="90" textAnchor="middle" fill="black" />
 <circle cx="27" cy="126" r="13" fill="#0f172a" />
 <text x="27" y="130" textAnchor="middle" fill="black" />
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill="black" />
 <circle cx="95" cy="126" r="13" fill="#0f172a" />
 <text x="95" y="130" textAnchor="middle" fill="black" />
 <text x="112" y="92" fill={isRot ? "#10b981" : "black"}
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize="11" />
</g>
</svg>
<p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p γ не меняется никог
 числа поворотов. Единственный, кто меняет хозяин
</p>
</div>
);
};

```

```

useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(([
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py

const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
const hBefore = useMemo(() => (steps ? heightOf(steps
const hNow = heightOf(viewRoot);

```

```

 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Шанка */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клики
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 </input>
 </div>
 </div>
 </div>
)

```

```

 В начало
 </button>
 <button onClick={() => { setPlaying(false); set
 -lg text-xs font-bold bg-slate-700 text-white
 ← Шаг
 </button>
 <button onClick={() => setPlaying(!playing)} cl
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); set
 className="px-3 py-1.5 rounded-lg text-xs font
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (
 <span className="px-2.5 py-1 rounded-full text
 00/60">
 {step.caseLabel}

)}
 </div>
)}

 {/* Дерево */}
 <div className="bg-indigo-950/20 rounded-xl p-4 b
 {!steps && (
 <div className="absolute top-3 left-3 bg-indi
 ont-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} classNam
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);

```

```

 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13">
 {n.key}
 </text>
 </g>
);
 }}}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400" >
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center" >
 {step ? step.note : "Клики по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "text-amber-400" : "text-slate-400"}`} >
 {checkOk ? " " : " "} проверка кадра: 0
 {inorderKeys(viewRoot).length}
 </p>
 }
 </div>
 <div className="flex-1" >

```

```

 <h4 className="text-sm font-bold text-slate-200">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig" &&
 const zz2 = step?.caseLabel === "Zig-Zig" &&
 const za1 = step?.caseLabel === "Zig-Zag" &&
 const za2 = step?.caseLabel === "Zig-Zag" &&
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 text-slate-400`} >

 {l.text}

 </div>
)
 })}
 </div>
 </div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {
 id: string;

```

```

)))
 : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.color}`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
 }, 500);
}, [dirtyStack, isWashing]);

```



```

 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', imState: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', mState: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 };

```

```

const topIndex = displayStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 disabled:cursor-not-allowed text-white"
 >
 Добавить тарелку
 </button>
 </div>
 </div>
)

```

```
 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && {
 <div className="absolute inset-0 pointer-events-none" >
 <div className="absolute w-6 h-6 rounded-full bg-white" >
 </div>
 <div className="absolute w-4 h-4 rounded-full bg-white" >
 </div>
 <div className="absolute w-5 h-5 rounded-full bg-white" >
 </div>
 <div className="absolute text-4xl anim-crash" >
 </div>
 </div>
 }}

 {displayStack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center" >

 <p className="text-white font-black text-2xl" >
 <p className="text-slate-500 text-xs mt-1" >
 </div>
 } : {
 <div className={`flex-1 flex flex-col justify-center`} >

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center justify-center"
 style={{ bottom: `${24 + topIndex * 44}px` }} >

 er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>
 </div>
 </div>
 </div>
 }
 }
```

```

<div className="absolute top-4 right-4 flex items-center justify-center"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
</div>

<div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col item-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </p>
 </div>
) : {
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full border
items-center justify-center gap-2 opacity-100"
 >
 {plate.name}

 </div>
 </div>
 })}
 </div>
 }}
</div>

{/* Столешница сушки */}
<div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-200" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-400" />

```

```

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 'Сходство уменьшается. Откат назад: откат j = pi[j - 1] = ${pi[j - 1]};
 j = pi[j - 1];
 });
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 'Сходство увеличивается.' });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 'Схождений нет.' });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записываем ${j} в pi[${i}].` });

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true });
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало. l=${l}, r=${r}, z=${z}` });

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=${i}, l=${l}, r=${r}, z=${z}` });
 }
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

 const { s: demoString, steps, patternLength: demoPatternLength } =
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);
 const runtime = useVizRuntime();
 const runtimeString = vizString(runtime?.variables?.runtimeString);
 const s = runtimeString ?? demoString;
 const patternLength = runtimeString ? runtimeString.length : demoPatternLength;

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 const timer = useRef<NodeJS.Timeout | null>(null);

 // Reset when inputs or mode change
 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

 // Timer loop for autoPlay
 useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setAutoPlay(false); }
 return prev + 1;
 });
 }, 1200);
 }
 }, [autoPlay, steps]);

```

```

 <button onClick={() => setMode("z")}
 className={`px-3 py-1.5 text-xs
 " : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2">
 <div className="bg-slate-800 p-1 flex items-center gap-2">

 <input value={pattern} onChange={setPattern} />

 </div>
 <div className="bg-slate-800 p-1 flex items-center gap-2">

 <input value={text} onChange={setText} />

 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-full">
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-400";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-slate-400" : "text-white";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(char)) {
 borderColor = "border-amber-400";
 bgColor = step.match === char ? "bg-amber-400" : "bg-slate-900/40";
 }
 }
 })}
 </div>
 </div>
 </div>
</div>

```

```

 {(mode == "kmp" ?
</div>

 /* KMP fingers */
 {mode == "kmp" && step
 <div className="abs
z-20">
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
20">
 <span className
 </div>
 }}

 /* Z fingers */
 {mode == "z" && step.i
 <div className="abs
e z-20">
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>

```





```

*/
export const Tooltip: React.FC<TooltipProps> = ({ content, side }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 transition duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 >
 {content}
 <span
 className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
 ${side === "top" ? "-bottom-[5px] border-b border-slate-900" : ""}
 >

);
};

```

```
];
```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
```

```
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];
```

```
 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
```

```

 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
 };
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)

```

```

) : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 "
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]"
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"

```

```

let fill = "#1e293b";
let stroke = "#334155";
let textColor = "text-slate-400";
void textColor;

if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
} else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
} else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
}

return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-500"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 />
 </g>
)

```

```


 РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded-2xl
pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>

 {/* Color key */}
 <div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}


```

## Универсальное пособие • полный исходный код

```

* build – сборка: ВИДИМАЯ сортировка точек (и чем с
* координатная плоскость НАВЕРХУ (ассоциаци
* чек-лист инвариантов (автопроверка, как у
* split – разрез по x с двумя корзинами (грамотная
* merge – склейка двух деревьев по y.
* erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
* layers – миф «дети в массиве по $2k/2k+1$ »: работает
* search – «отсортирую бинарным поиском»: поиск не с
* сортировка платит $\geq n \cdot \log n$ сравнений, со
*
* Канонический набор: все x и все y различны $\Rightarrow$ дерево
* Узел идентифицируется парой $id = "x:y"$.
*/
```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
```

```

}
export function splitTree(root: TNode | null, x: number) {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key} <= ${x})
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key} > ${x})
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null };
}
export function mergeTree(a: TNode | null, b: TNode | null) {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
 note: `о одно из деревьев целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
 note: `a выбран, потому что его правое поддерево с (${b.key}, ${b.grade})
 не содержит элементов с ключом <=${b.key}` });
 a.right = go(a.right, b);
 } else {
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
 note: `b выбран, потому что его левое поддерево с (${a.key}, ${a.grade})
 не содержит элементов с ключом >=${a.key}` });
 b.left = go(a, b.left);
 }
 return { l: go(a.left, b.left), r: go(a.right, b), grade: Math.max(a.grade, b.grade) };
 };
 const res = go(a, b);
 return { root: res, trace };
}

```



```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number) => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева ----- \*/

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 20 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id }); }
 if (n.right) { es.push({ from: n.id, to: n.right.id }); }
 walk(n.left);
 walk(n.right);
 };
 walk(root);
 return es;
}

```

```

 stroke={edgeColor ? edgeColor(e.from, e.to)}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 }
}
}
nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${p[0]}, ${p[1]})`}
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
 > strokeWidth={pulse ? 4 : 2} className="transition-all duration-500"
 <text y={-2} fill="#fff" fontSize="13" fontStyle="italic">{n.id}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{p[0]} {p[1]}</text>
 </g>
);
}
}
}
</svg>
{subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6) / 10);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

interface PlaneProps {

```

```

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key &&
 badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key &&
 badEdge.b.key === e.a.key));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 strokeDasharray={isBad ? "4,3" : "none"} />
})}

{/* временная линия игры */}
{interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]} />
)}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key))
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ? () => {}}
 onPointerUp={interactive && onPointUp ? () => {}}
 style={interactive ? { cursor: "pointer" } : {}}
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
 strokeWidth={2} />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12}
 >{p.key}</text>
 </g>
);
})}

{pending && (

```

```

 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur)).a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]) {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n.a)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === gpid(p.a)));
 const complete = edges.length === all.length - 1 && roots.length === 1;
 return { heap, oneParent, acyclic, bst, complete, win: heap && oneParent && acyclic && bst && complete };
}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {

```

```

 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);
 const ys = pairs.map((p) => p.grade);
 return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
 a: Pair;
 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: true };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 pid(e.a) === pid(y) && pid(e.b) === pid(x))
 return { ok: false, reason: "такая линия уже проведена" };
 }
 if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key});` };
 // цикл: родитель уже сидит в компоненте ребёнка (ребёнок не может быть своим родителем)
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {

```

```

 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))));
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет
 if (sorted[0].key <= p.key) left = sorted[0];

```

```

 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return;
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | nul

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.p

 const n = userPairs.length;

 const userTree = useMemo(() => (userPairs.length >= 1
 const canonEdges = useMemo(() => {

```

```

const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair), [userPair]);
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(gameEdges) : null), [gameEdges, gameWon]);

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 }
};

```



```

return (
 <div className="space-y-5">
 {/* Произвольный список */}
 <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-800">
 <div className="flex flex-wrap items-center justify-between">
 <h4 className="text-sm font-bold text-slate-300">Демо-набор
 <button onClick={() => { setRaw(DEMO_RAW); setRaw(DEMO_RAW); }}
 className="py-1 rounded-lg text-xs font-bold bg-slate-800 text-slate-200">
 демо-набор
 </button>
 </div>
 <textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); setRaw(e.target.value); }}
 rows={2}
 className="w-full bg-slate-950 border border-slate-800 border-emerald-500"
 placeholder="3 5, 45 8, 1 -4 ..."
 />
 {parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg-slate-900/60 rounded-xl p-4 border border-slate-800">
 {parsed.errors.map((e, i) => <div key={i}>{e.message}
 </div>
)}
)}
 {parsed.errors.length === 0 && (parsed.equalX || parsed.equalY ? (
 <div className="mt-2 text-xs text-amber-300 bg-slate-900/60 rounded-xl p-4 border border-slate-800">
 {parsed.equalY && <div>⚡ Есть равные y у кого-то раньше в списке</div> (сортировка стабильная)
 {parsed.equalX && <div>⚡ Есть равные x у кого-то раньше в списке</div> (сортировка стабильная)
 {!parsed.equalX && !parsed.equalY && <div>⚡ Нет равных в списке</div>}
 </div>
)}
 </div>
 {/* Координатная плоскость – НАВЕРХУ */}
 <div className="bg-emerald-950/20 rounded-xl p-4 border border-emerald-800">

```

```

 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-
go-600 text-white" : "bg-indigo-500/80 text
 >
 соединить сам
 </button>
 </>
 })
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: <b cl
 {comparisons} (n·log2n).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>

```

```

 })

 {phase === "game" && {
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
 })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && {
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
 ществует." />
 </div>
})

{/* Дерево */}
{order === "y-desc" && userTree && {
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап
 </div>
 })
</div>
});
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

```

```

<button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
<button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
шаг {i
</div>

<div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
</div>

<div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
</div>

{done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3 l
 <h5 className="text-sm font-bold text-sky-3
 <TreeView root={lTree} width={480} height={
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-rose-
 <TreeView root={rTree} width={480} height={
 </div>
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-9
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой – д

```

```

 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-900
 {cur?.chosenId ? (
 <>⊗ {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в
 />
 </>
)}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald-300
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900
 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])

```

```

 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783"
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300" >
 <TreeView root={res.root} subtitle="Валидность" >
 <ul className="mt-2 space-y-1 text-sm" >
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по у (куча целая) –

 </div>
)}

 <div className="text-xs text-slate-300 bg-slate-900/40" >
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/40" >
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая
 </div>
 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h

```

```

 </div>
 <p className="text-xs text-slate-400">Полное
 → $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-rose-300">
 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2">
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
 .map((i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded-
 der-slate-700` : "bg-rose-950/40 border-rose-
 500 text-slate-500 text-[0.8em] font-sans">
 {id ? <div className="text-white font-sans">
 </div>
 </div>
);
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смо
 то левая ветка ушла на 4 уровня и заняла два
 указателя (left/right), а не арифметикой.</p>
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900/40 p-4">
 Грабля «дети в массиве по $2k/2k+1$ »: формируе
 мся: слои дырявые, и «адрес» ребёнка посчитан
 </div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

```

```

<div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти у для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSte
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 C6poc</button>
 </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s
 const isMid = stage === 2 && (() => { const s
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class=
 nsition-all ${highlight ? "border-rose-500 l
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
);
)}
)}
</div>

```

```

<div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}

```



```

 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(` Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

const win = v.win;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0,
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(n
 3 py-1.5 rounded-lg text-xs font-bold bg-slate
 <button onClick={() => { setEdges(CANON_GAME.ma
); }} className="px-3 py-1.5 rounded-lg text-
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4 l
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class
 <defs>
 <pattern id="gm-grid" width="14" height="14"
 <path d="M 14 0 L 0 0 0 14" fill="none" s
 </path>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height=
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P

```

```

 {v.hear ? " " : " " } каждое ребро – вниз
 {v.oneParent ? " " : " " } у каждой точки
 {v.acyclic ? " " : " " } циклов нет
 {v.bst ? " " : " " } обход слева-направо –
 {v.complete ? " " : " " } линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-eme
 ения просто не существует.</p>
 }}
</div>

 <div className="text-xs text-slate-300 bg-slate-9
</div>
);
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИ
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаёт кадры Split / E
 * height, n – высота дерева и число узлов (показ
 */
const useTreapLive = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(va
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {
 if (Array.isArray(it) && it.length >= 2) {

```

```

// вкладка = подрежим страницы: панель Python подстав.
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар.
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
))}
 </div>

 {/* что сейчас сообщил компилятор */}
 <div className="mb-4 flex flex-wrap items-center">
 <span className="text-[10px] uppercase tracking-
 <Terminal className="w-3.5 h-3.5" /> из компи.

 {live.vars ? (

```

```
import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";
```

```
interface VizLiveStatusProps {
 /** Название демонстрации – подписывается в режиме vs
 demoTitle?: string;
 className?: string;
}
```

```
/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",
```

```
/**
```

```
* Строка состояния связи «компилятор → демонстрация» д
*
```

```
* Раньше компонент молча проигрывал собственный встроен
* редакторе Python был пустой код, код с другими именами
* размерности, – выглядело так, будто компилятор исполн
* картинка живёт отдельной жизнью. Теперь источник данн
* «данные из компилятора (i, j, dist 5x5)», «компилятор
* пока нет» (отладчик стоит до первой строки) или «встр
*/
```

```
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name)
 const linked = names.length > 0;
```

```
const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
const matrix = liveMatrix(matrixSource?.value);
```

```
const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length >
 : undefined;
```

```

import { SalmonAutomatonWidget } from "../SalmonAutomaton";
import { SegmentTreeVisualizer } from "../SegmentTreeVis";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithms";
import { TopologicalSortViz } from "../TopologicalSortVi";
import { WaterfallAnimationWidget } from "../WaterfallAn";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumVi";
import { SparseTableViz } from "../visualizers/SparseTab";
import { TreapBuildViz } from "../TreapBuildViz";
```

```
export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}
```

```
/**
```

```
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
```

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
)
 },
}
```

```

 Component: SplayPlayground,
 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: SplayPlayground },
 "top-sort": { title: "Топологическая сортировка", Component: SplayPlayground },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: SplayPlayground },
 "graph-articulation": { title: "Точки сочленения", Component: SplayPlayground },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: SplayPlayground },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: SplayPlayground },
 "planarity-euler-formula": {
 title: "Планарность и покраска: K5/K3,3, $\chi \leq \Delta+1$, д
 hint: "Четыре вкладки: непланарные K5 и K3,3, жадна
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
 },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: DijkstraViz,
 },
 "bellman-ford": {
 title: "Форд–Беллман",
 Component: BellmanFordViz,
 },
 floyd: {
 title: "Флойд–Уоршелл",
 Component: FloydViz,
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonViz },
 mst: {
 title: "Остовное дерево: Краскал, Прима, Борувка",
 hint: "Три алгоритма (билеты 18–20) на одном графе и
 переменные parent/mst/taken/u/v/w перекрашивают граф",
 Component: KruskalSimulator,
 },
 "graph-components": {
 title: "Компоненты связности: обход и DSU",
 hint: "Две вкладки: покраска обходом (каждый новый
 nt из вашего кода красят вершины.",
 },

```

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";

interface VizSourceBadgeProps {
 /**
 * true – картинку рисуют переменные из выполненного Python-кода
 * false – компонент показывает собственную встроенную картинку
 */
 linked: boolean;
 /**
 * Компилятор уже выполнил код и прислал шаг, но переключатель не нажат
 * (отладчик стоит ДО первой строки или в коде другие комментарии)
 * подпись должна объяснять, почему картинка ещё не инициализирована
 */
 awaiting?: boolean;
 /** Строка пользовательского кода, на которой стоит отладчик */
 line?: number;
 /** Короткая подпись данных компилятора: «матрица 5×5» */
 detail?: string;
 /** Как называется встроенная демонстрация (для подписи) */
 demoTitle?: string;
 /** Размеры демо-сетки и данных пользователя не совпадают */
 mismatch?: string | null;
 className?: string;
}

/**
 * Явный индикатор источника данных визуализации.
 *
 * Без него демонстрация молча проигрывала собственный Python-код
 * когда редактор Python пуст или в нём другие имена/переменные
 * так, будто компилятор «исполняет какой-то свой код».
 * «из компилятора» или «встроенное демо».
 */
export function VizSourceBadge({ linked, awaiting = false, line, detail, demoTitle, mismatch, className }) {
 const tip = linked
 ? `Значения берутся из выполненного Python-кода${detail}`
 : `Встроенная демонстрация${demoTitle}`;
 return (

{linked ? Cpu : Film}

{detail}
 {demoTitle}

{mismatch}


```

```
 0"
 >
 размерность не совпадает

 </Tooltip>
) : null}
</div>
);
}
```

---

ФАЙЛ 82/111: src/components/WaterfallAnimationWidget.ts

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
```

```
// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
 // Ветка H -> E -> R -> S
```



```

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveText = vars ? vizString(vars.text) ?? vizSt
const liveState = vars ? vizNumber(vars.state) ?? viz
const liveI = vars ? vizNumber(vars.i) ?? vizNumber(v
const liveMode = vars ? vizString(vars.mode) : null;
const liveMatchesRaw = vars ? vizArray(vars.matches)
const liveMatchesSig = useMemo(() => (liveMatchesRaw
const liveFail = vars ? vars.fail : undefined;

useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) set
 else if (liveMode && /train|обуч|bfs/i.test(liveMod
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
 if (liveState !== null && liveState >= 0 && WATERFA
 if (liveI !== null && liveI >= 0) setCurrentIndex(M
 ToScan).length));
 if (liveMatchesSig) {
 const parsed = (JSON.parse(liveMatchesSig) as unk
 .map((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const idx = vizNumber(it[0]);
 const word = vizString(it[1]);

```

```

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 }
 }
};

```

```

 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

```

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В

```

const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },

```

```

desc: 'Переходим на Depth 3 к #3 (HER). Родитель
 е подходят. fail[HER] = ROOT.',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'R',
checkingBranchesFrom: 0,
},
{
targetNodeId: 6,
title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
},
{
targetNodeId: 9,
title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
codeLine: 4,
scoutPos: 1,
inspectedParentFail: 1,
checkTargetChar: 'E',
checkingBranchesFrom: 1,
},
{
targetNodeId: 4,
title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,

```

```

 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
 </div>

 { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
 {activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3 gap-4" >
 { /* Управление шагами обучения */ }
 <div className="bg-slate-900/90 p-5 rounded-xl" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (BFS)
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 аргумента 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow-hidden" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 py-2
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }
 `}
 >

```

```

 Вершина: #{targetTrainingNode.id} ({t

</h5>
<div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
тыт}
 </div>
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefi
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex ite
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
</div>
</div>
</div>

```

```

 <span
 key={idx}
 className={`w-7 h-8 flex items-center
 idx === currentIndex
 ? 'bg-blue-600 text-white shadow
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emera
 : 'bg-slate-800 text-slate-500'
 }`
 >
 {char}

)}
 {currentIndex >= textToScan.length && (
 <span className="bg-emerald-600 text-
 ГОТОВО ✓

)}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.len
 className={`w-full py-2.5 rounded-lg fo
 isSearchDone || textToScan.length ===
 ? 'bg-slate-700 text-slate-500 curs
 : 'bg-gradient-to-r from-blue-600 to
ue-900/40 active:scale-[0.98]`
 }`
 >
 {currentIndex === 0 ? 'Начать пла
 <ArrowRight className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60
 <div>

```

```

 </div>
 </div>
</div>
</div>
}}

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
 <div className="absolute inset-0 opacity-10 bg-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>

 </div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">
 <g>
 <!-- Горизонтальные линии и подписи уровней -->
 <g>

```



```

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}` /* Линия течения воды */>
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 className={isHighlighted ? 'highlighted' : 'normal'}
 />
 /* Символ перехода */
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'} strokeWidth={isHighlighted ? 2 : 1}
 />
 <text
 x={{fromNode.x + toNode.x} / 2}
 y={{fromNode.y + toNode.y} / 2 + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </text>

 /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В ПРАВИЛЬН. ОТВЕТЕ */
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`} >
 <rect x="-15" y="-12" width="30" height="24"
 fill={isMatchBranch ? '#10b981' : '#f43f5f'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 fill={isMatchBranch ? 'white' : 'black'}
 textAnchor="center" />
 </g>
)}
)
)
}

```

```

 <text x={cx} y={cy - 10} fill="#10b98f">
 >
 fail[{node.label}] = {targetNode.label}
 </text>
 })
 </g>
);
 }}}

```

```

 { /* Рендер вершин (бассейнов водопада) */ }
 { Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'transition';
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className="node">
 { /* Внешнее свечение для активной вершины */ }
 { isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="#10b98f" stroke-width={2} />
) }
 { isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="#f08080" stroke-width={2} />
) }

```

```

 { /* Основной круг вершины */ }
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24px' : '20px'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f08080' : 'none'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#ffffff' : 'none'}
 strokeWidth={isCurrent || isTargetChild ? 2 : 0}
 className="transition-all duration-300"
 />

```

```

 { /* Текст внутри вершины */ }
 <text

```

```

 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 /* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
 </div>
 </div>

 /* Список найденных совпадений (только в режиме
 {activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded
 <h5 className="text-white font-bold mb-3 text
 Улов лосося (Найденные слова
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic
 Пока ничего не поймали. Запустите шаги си
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (

```

```

) : (

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <code>
 an-300">matches</code> поведут лосося по этим

)}
 </div>
</div>
);
};

```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 10.5 КБ

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from '../../../data/vizStepBus';

```

```

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива

```

```

 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`}
 >
 {label}
 </button>
)))
</div>
{mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[
 []
);

 /** Что сейчас под курсором: префикс P[i] или элемент
 const [hover, setHover] = useState<{ kind: "pref" | "
 const [range, setRange] = useState<{ l: number; r: num
 /** Снимок i/P из автоматически выполненного Python-к
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveP = vizArray(vars?.P);
 const hasLiveP = !!vars && Object.prototype.hasOwnProperty
 const compilerLinked = liveI !== null || hasLiveP;
 // i/P управляют ячейками напрямую; индекс Python-стр
 const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(live
 : Math.max(0, Math.min(A1.length, (liveP?.length
 : null;

```

```

 : inCover
 ? "bg-indigo-500 text-white ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-100"
 : "bg-slate-800 text-slate-300"
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === range.l;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue === null || liveValue === 0);
 const masked = !hasLiveP && shownDriven !== null;
 const isDrivenCur = shownDriven !== null && i === shownDriven;
 const shownValue = hasLiveP ? liveValue : null;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref", i })}
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help ${
 isDrivenCur
 ? "bg-emerald-600 text-white ring-emerald-600"
 : blank
 ? "bg-slate-950 text-slate-600"
 : masked
 ? "bg-slate-800/50 text-slate-700"

```

```

 { " "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 { " "}
 входит во все префиксы, начиная с{ " "}

 подвинуть границу запроса.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-indigo-400">у
 у <b className="text-indigo-400">a – ку,
 </p>
)}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo-400">
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву)
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-300">
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1}..{range.l}]
 {pref[range.r + 1]}
 {pref[range.l]}
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) по префиксам
 </p>

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
 ? Array.from({ length: prefRows }, (_, i) =>
 Array.from({ length: prefCols }, (_, j) => liveS[i][j]
)
 : P;

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from({ length: srcCols }, (_, j) => liveA[i][j]
)
 : A2;

const cellAt = (i: number, j: number): number | null {
 if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) return grid[i][j];
 return null;
}

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка префикса имеет размер ${liveS.sizeLabel}). Ниже нарисована таблица`
 : null;

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2 === null) return;
 setRect({
 r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(r1))),
 c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(c1))),
 r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(r2))),
 c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(c2)))
 });
});

```



```

{srcGrid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1"
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&
 return {
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.max(r.r2, j), c2: Math.max(r.c2, i) }
 : { ...r, r2: i, c2: j }
)
 }
 className={` ${cellBase} cursor-pointer ${
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 >
 {v === null ? "." : v}
 </div>
 }
)}
 </div>
});
}}
</div>
)}}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource}` : ""}
 </div>
 <div className="inline-block">
 {grid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1"
 {row.map((v, j) => {
 const blank = v === null || v === undefined;
 const shownValue = v;

```

```

 }}}
 </div>
 </div>
 </div>

 <div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? (
 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P"}
 {" "}
 – сумма всего прямоугольника от левого верхнего угла
 </p>
) : (
 <p className="text-slate-500">Наведите курсор на элемент</p>
)}
 </div>

 <div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A *
 C + D
 = {" "}
 {
 </p>
 <p className="font-mono text-[11px] text-slate-300">
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
</div>
);
};

```

---

```

const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],

```

```

 className={`px-3 sm:px-4 py-2 rounded-lg text-
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
 const compilerLinked = liveI !== null || liveJ !== null;
 const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;
 const liveTarget =

```

```

 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 {dimMismatch ? {
 <p className="mt-2 max-w-[560px] rounded-lg">
 {dimMismatch}
 </p>
 } : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:opacity-50"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 >
 <RotateCcw size={20} />
 </button>
 </div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <div className="text-[11px] uppercase tracking-wider">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1
 })}
 </div>
 </div>
</div>

```

```

 Наведите курсор (или тапните) на любое чи
 </p>
 })
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-500">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 { /* Headers */ }
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>

 { /* Matrix */ }
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono text-sm">
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;

```

```

return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,170px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-300"
) : (
 <motion.div
 onMouseEnter={() => (compilerLinked ? setHover(1) : setActive())}
 onMouseLeave={() => setHover(0)}
 onClick={() => (compilerLinked ? setSrc1() : setSrc2())}
 >
 {displayValue}
 </motion.div>
)}
 >
 <div className="relative flex flex-col items-center justify-center"
 initial={{ opacity: 0, scale: 1 }}
 animate={{
 // При st = [] каркас уже с
 opacity: compilerLinked ? 1 : 0,
 scale: compilerLinked || isSrc1 || isSrc2 ? 1 : 0,
 }}
 className={`w-[clamp(30px,9vw,170px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-300`}
 style={{
 background: `
 ${compilerLinked ? `
 ${isSrc1 ? "bg-sky-500 text-white" :
 isSrc2 ? "bg-indigo-500 text-white" :
 isSrc1 ? "bg-emerald-500 text-white" :
 isSrc2 ? "bg-amber-500 text-white" :
 isVisible ? "bg-slate-800 text-white" :
 compilerLinked ? "bg-slate-950 text-white" :
 "bg-transparent"
 }` : ""}
 `}
 title={compilerLinked ? `st[${j}]` : ""}
 >
 {displayValue}
 </div>
 </div>
)

```

```

 {(hasLiveTable || hasLiveSource) && <> = <source>
 ?.liveTarget.j ?? (liveTarget.j === 0 ? liveTarget.i : liveTarget.j)}
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(

 ST[{c.i}][{c.j - 1}]

 ,

 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D[{c.i}][{c.j}]}

 {st1D[{c.i + half}][{c.j - 1}]}

) = {" "}

 {st1D[{c.i}][{c.j}]}

 </p>
);
 })();
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)
)
</div>

```



```

 : k;

/** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
 ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
 ? `Демо 2D построено на матрице 8x8, а в вашем ко
 не её не отрисовываются, значения вашей матрицы
 : null
 : null;

// Уровень и клетка берутся прямо из k/r/c; номер стро
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 s
 <h3 className="text-xl font-bold text-indigo-400
 <p className="text-sm text-slate-400 mb-6 text-
 Для наглядности показываем только <b classNam

 <span className="text-rose-300 animate-pulse
 иц!
 </p>
 {dimMismatch2D ? (
 <p className="mb-4 max-w-[520px] rounded-lg b
 {dimMismatch2D}
 </p>
) : null}

 <div className="flex bg-slate-950 p-1 rounded-l
 {[0, 1, 2, 3].map(step => (

```

```

{Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 const liveValue = liveTable?.[`
 // Базовый слой (блоки 1x1) – э
 // одного элемента равен ему са
 // показываем из A сразу, как A
 // цикла построения st2. Уровни
 const sourceValue = step === 0
 const shownValue = compilerLink
 const isBlank = shownValue ===
 let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-600'
 : 'bg-slate-800/50 text-slate-600';
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
 ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код»
 // залитая пустая клетка вы
 highlightClass = isBlank
 ? `bg-indigo-500/10 text-white`
 : `bg-indigo-500 text-white`;
 == c ? 'ring-[3px] ring-white ring-offset-2'
 : `bg-indigo-500 text-white`;
 && locked.c === c ? 'ring-[3px] ring-white'
 : `bg-indigo-500 text-white`;
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK
 const isTop = r < activeCel
 const isLeft = c < activeCe

```



```

 <div className="space-y-2 pt-3 pl-2"
 {{{() => {
 const prevL = 1 << (shownK - 1)
 return (
 <>
 <div className="flex items-center"
 <span className="flex-grow-1"
 shadow-[0_0_8px_#f43f5e]"></div> ST[{activeCell.r}][activeCell.c][shownK-1][shownK-1]
 </div>
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 shadow-[0_0_8px_#3b82f6]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 </div>
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 shadow-[0_0_8px_#10b981]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 </div>
 </div>
 <div className="flex items-center"
 <span className="flex-grow-1"
 shadow-[0_0_8px_#f59e0b]"></div> ST[{activeCell.r}][activeCell.c + prevL][shownK-1][shownK-1]
 </div>
 </div>
 </>
)
 }}}()}}
 </div>

 <div className="pt-3 mt-3 border-top border-white"
 >
 0">{ST2D[activeCell.r][activeCell.c][shownK-1][shownK-1]}

```

```

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((name, i) => {
const queryMismatch = liveCoords.some((v) => v !== queryVars?.[i]
 ? `Координаты запроса в вашем коде выходят за дем
] ?? "-"`, r2=${liveCoords[2] ?? "-"}, c2=${liveCoords[3] ?? "-"}
 : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6" >
 <div className="flex-1 min-w-0 bg-slate-900" >
 <h3 className="text-xl font-bold text-rose-500" >Запрос
 <p className="text-slate-400 text-[13px] font-mono" >
 r1=${r1}, c1=${c1} - левый верх, r2=${r2}, c2=${c2} - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border border-rose-500 p-2" >
 {queryMismatch}
 </p>
) : null}

 <div className="flex flex-col items-center" >
 <div className="grid grid-cols-[repeat(8, 1fr)] gap-2" >
 <div className="active shadow-inner w-max max-w-full" >
 {Array.from({length: 64}).map((_, idx) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;
 return (

```

```

 <label className="flex items-center"
 <span className="text-slate-700"
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 </div>
 </label>
 <label className="flex items-center"
 <span className="text-slate-700"
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={x} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 <input type="number" min={0} max={100} value={y} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 </div>
 </label>
 </div>
 </div>
 </div>
 </div>

 <div className="flex-1 bg-slate-900 p-6 rounded"
 <h3 className="text-xl font-bold text-emerald-400">Задача 1</h3>
 <p className="text-sm font-sans text-slate-300">
 Мы знаем, что максимальная степень дв
 rounded text-white">{Lx}. Аналогично
 te">{Ly}.
 </p>
 <p className="text-sm font-sans text-slate-300">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded border
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl p-4"
 text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3"

```

```

border-slate-700 shadow-sm">ST[][][kx]][ky]
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - 1 && c === c1;
 let isBR = r === r2 - 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white";
 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-500 text-white";
 ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-500 text-white";
 -emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-500 text-white";
 mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
 tion-all duration-300 relative shrink-0 ${c === c2 ? 'text-right' : ''}
 {txt}
 {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
 </div>
)
 })}
</div>
</div>
</div>

```

```
function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 const aside = document.querySelector('aside')
 const headerMain = document.querySelector('h1')
 const questionsContainer = document.getElementById('questions')
 const wtfContainer = document.getElementById('wtf')

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display = 'none'
 if (questionsContainer) questionsContainer.style.display = 'none'
 if (wtfContainer) wtfContainer.classList.remove('hidden')

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.location.search)
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode')
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden')
 }
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);
}
```



```

 setTimeout(() => drawerOverlay.classList.add("open"))
 }
 document.body.style.overflow = "hidden"
} else {
 mobileDrawer.classList.add("-translate-x-100%")
 if (drawerOverlay) {
 drawerOverlay.classList.add("open")
 setTimeout(() => drawerOverlay.classList.add("open"))
 }
 document.body.style.overflow = ""
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => {
 toggleDrawer()
 })
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => {
 toggleDrawer()
 })
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => {
 toggleDrawer()
 })
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
 toggleDrawer()
 }
 })
})

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

```

```

<script id="MathJax-script" async src="https://cdn.
<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "../src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
 .uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}

```

```

.uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card-
.card-green { background-color: #339e35; } .card-
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

```

```

 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 ТЕМ
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook"
 transition-colors">Тетрадь</button>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6" color="yellow"

```

```

 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform l
 <svg xmlns="http://www.w3.org/2000/svg" class="l
 <path stroke-linecap="round" stroke-linejoin
 </svg>
 </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed ins

```



```
 <details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числа

 </summary>
 <div class="pl-6 space-y-2 text-pink-300">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля Виллиса

 </summary>
 <div class="pl-6 space-y-2 text-violet-300">

 </div>
</details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
 9. Алгоритмы нахождения простых чисел

 </summary>
 <div class="pl-6 space-y-2 text-green-300">

 </div>
</details>
```

```
нственности НОД

ПИРАЛЬ)

вращается!)
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

деление)
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

 </div>
 </details>

 <details class="group">
```



```

<div class="pl-6 mt-2 space-y-2"
 <a href="#q16-1" class="blo
 <a href="#q16-2" class="blo
 <a href="#q16-3" class="blo
</div>
</details>

<div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blur

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold ml
 ▲ Ликбез: Алгоритмы выживания


```

```

 a.style.fontWeight = 'normal';
 if (a.href.endsWith('#')) {
 a.style.fontSize = '1.2em';
 }
 });

 // Open the parent modal if the link is a details link
 const correspondingLink = detailsLink.parentNode.querySelector('a');
 if (correspondingLink) {
 const details = detailsLink.parentNode;
 if (details && details.open) {
 document.querySelector(detailsLink.getAttribute('data-parent')).click();
 if (otherDetails) {
 otherDetails.close();
 }
 }
 }
 });
 });
 }, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
 });

 sections.forEach(section => {
 observer.observe(section);
 });
});

function fallbackModal(contentRaw, detailsLink) {
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';

```

```

btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop =

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#333';
copyBtn.style.color = 'white';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = 'white';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
}

```

```
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-500');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-500');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 { color: #000000; }
 body.bg-white main { color: #000000; }
 body.bg-white main p { color: #000000; }
 /* Universal text color */
 body.bg-white .text-slate-500 { color: #000000; }
 body.bg-white .text-slate-400 { color: #000000; }
 body.bg-white .text-slate-300 { color: #000000; }
 body.bg-white [class*="text-slate"] { color: #000000; }
 `;
 } else {
 body.classList.remove('bg-slate-500');
 body.classList.remove('bg-white');
 document.getElementById('theme-auto').checked = true;
 }
 }
}
```

```

body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
body.classList.add('bg-black')
document.getElementById('theme')
style.innerHTML = `
body.bg-black .bg-slate-950 {
background-color: transparent !important;
}
body.bg-black section header {
, body.bg-black button {
color: #4ade80 !important;
border-color: #22c55e !important;
font-family: monospace;
}
body.bg-black svg path,
stroke: #22c55e !important;
fill: transparent !important;
}
`;
}

document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border" style="border: 1px solid #22c55e; padding: 10px;">
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">

```

```
<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
 -base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Высшая алгебра: теория и практика

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center"
 fter: always;">
 <h2 class="font-bold text-xl mb-4">Содержание
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Декомпозиция
 7. Алгебраические структуры
 8. Клеймента
 9. Координаты
 10. Матрицы
 11. Линейные преобразования
 12. Полиномы
 13. Факториалы
 14. Биномиальные коэффициенты
 15. Комбинаторика
 17. Теория вероятностей
 18. Статистика
 19. Математический анализ
 20. Дифференциальное исчисление
 21. Интегральное исчисление
 22. Дифференциальные уравнения

```

```

 <div class="font-bold text-slate-700">

 <a href="#question-10" onclick="setMainQuestion(10)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <a href="#question-3" onclick="setMainQuestion(3)"
- 4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 3: НОД и НОК / Алгоритм Евклида -->
 <a href="#question-7" onclick="setMainQuestion(7)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

 <a href="#question-12" onclick="setMainQuestion(12)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структурная теория -->
 <a href="#question-8" onclick="setMainQuestion(8)"
- 4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

 <a href="#question-11" onclick="setMainQuestion(11)"
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">


```

```

<div class="col-span-1 md:col-span-3" style="border: 2px solid #008000; padding: 10px; margin-bottom: 10px;">

 <div class="text-xs text-teal-500" style="margin-bottom: 5px;">Вопрос 17: Какое из следующих утверждений верно?
```



```

 <span class="text-w
 <span class="text-w
ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuc
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s
 <span class="text-w
, r_2$).
 <span class="text-w
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-yel
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 s

```

```
 <p class="text-xs font-mono">
 $$$. Пока получается 0 – корень жив. Как то.
 корня минус один.</p>
```

```
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-mono">
 ициентов (они делятся на $$), кроме старшего
 зложить.</p>
```

```
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-mono">
 . Чаще всего после этого Фейс-контроль $$.
```

```
 </div>
```

```
</div>
```

```
 <h2 class="text-2xl font-bold bg-white">
 (Правила экстрасенса)</h2>
```

```
 <p class="text-sm text-slate-400">
 тношения), а ты их не помнишь. Юзай базовые
 нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
```

```
 <li class="bg-slate-800/50 p-4">

 <div>
```

```
 <h4 class="text-cyan">
 <p class="text-sm text-slate-400">
 <code class="text-xs font-mono">
```

```
 </div>
```

```

```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а комментарии
 * которые в приложении всплывают по наведению, собираюся в блок
 * в конце документа.
 */
```

```

.export-term .cx { color: #6ee7b7; font-size: .78rem; }
.export-foot { margin-top: 3rem; border-top: 1px solid #ccc; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; }
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string, opts: { selfId?: string; }) {
```

```
 body: string;
```

```
 termIds: string[];
```

```
} {
```

```
 const host = document.createElement("div");
```

```
 host.innerHTML = html;
```

```
 try {
```

```
 annotateTerms(host);
```

```
 } catch {
```

```
 /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
 }
```

```

if (!termIds.length) return "";
const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
 </div>`;
 })
 .join("\n");

return `<section class="export-gloss">
 <h2>${book ? "Словарик терминов пособия" : "Словарик терминов"}</h2>
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 10px 0">
 ны сюда.</p>
 ${items}
</section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string; }) {
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">

```

```

 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 img.setAttribute("src", data);
 } catch {
 /* картинка недоступна — глава останется со ссылкой */
 }
}
});
return doc.body.innerHTML;
}

```

/\*\*

\* Кадры визуализаций: книга встраивает снимки демонстраций, которые  
 \* файл показывал, что крутилось в приложении. Снимки делались скриптом  
 \* (capture-viz-shots.mjs) и публикует в /export/viz-shots. Если не удалось  
 \* без них (офлайн-сборка, локальный дев) книга остаётся без картинок,  
 \* нормально и молча пропускается.  
 \*/

```

async function vizFigures(chapterId: string): Promise<string> {
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-shots/manifest.json`);
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { files: string[] };
 const mine = manifest.filter((m) => m.file === `chapters/${chapterId}.mjs`);
 const figures: string[] = [];
 for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}export/viz-shots/${m.file}`);
 if (!res.ok) continue;
 const blob = await res.blob();
 if (blob.size > 3_000_000) continue;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 });
 figures.push(data);
 }
 return figures.join("\n");
 } catch {
 return "";
 }
}

```



```

* номер, но попасть на него не мог — ссылка была «битой»
* в никуда. Здесь такие упоминания превращаются в настоящее
*
* * в приложении — `

```

```

const ticketToChapter = new Map<number, string>();
for (const [id, tickets] of Object.entries(chapterTopics))
 for (const n of tickets) if (!ticketToChapter.has(n))
 ticketToChapter.set(n, id);

```

```

/** id страницы по номеру билета (undefined, если такого нет)
export const chapterForTicket = (ticket: number): string | undefined

```

```

/** «билет», «билеты», «билетов», «билете», «билетам»...
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{1,2})/i

```

```

/**
* Константы TreeWalker числами: глобальный `NodeFilter`
* во всех окружениях (в jsdom его нет) — с числами разницы нет
*/

```

```

const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

```

```

/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, button,

```

```

/** Одна страница на диапазон? «билеты 18–20» → mst, а не mst18
function chapterForRange(from: number, to: number): string | undefined

```



```

 return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) return whole;
 return `<a class="ticket-link" data-goto="${id}
 }));
 });
 .join("");
}

/**
 * DOM-версия – для интерактивной страницы. Проходит то.
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок – его проверяют DOM
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (!/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
);
 let count = 0;
 while (current = walker.nextNode()) {
 targets.push(current as Text);
 current = walker.nextNode();
 }
}

```

```

 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, deps);
}
```

## =====

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

## =====

-----

ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

-----

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
});
```

```

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num} || "-" | ${r.title} | ${r.hasAnalogy}
 ge)) |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log());
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**

```

```

 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *

```

```

 *

```

```

 * node scripts/audit-terms.mjs

```

```

 */

```

```

import { build } from "esbuild";

```

```

import path from "node:path";

```

```

 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 4)
);
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length, "шт.");

```

ФАЙЛ 92/111: `scripts/export/build-guide-pdf.mjs`

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 865 | РАЗМЕР: 10.5 КБ

```

/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * - текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz/
 * capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменные
 * Python-компилятор (работает и без браузера – локально)
 * - БЕЗ кода панели компилятора и без навигации приложения
 *
 */

```

```

 { u: "C", v: "E", w: 8 },
 { u: "G", v: "D", w: 1, highlighted: true },
 { u: "G", v: "E", w: 2, highlighted: true },
 { u: "D", v: "F", w: 2 },
 { u: "E", v: "F", w: 1, highlighted: true },
],
 table: [
 ["Вершина", "Старт", "Кратчайшее dist", "Предшест"],
 ["Пиццерия A", "s = A", "0", "-", "зафиксирована"],
 ["Парк C", "из A", "2", "A", "зафиксирована"],
 ["Дом 1 B", "из C", "3", "C", "зафиксирована"],
 ["Метро G", "из C", "5", "C", "зафиксирована"],
 ["Офис D", "из G", "6", "G", "зафиксирована"],
 ["Дом 2 E", "из G", "7", "G", "зафиксирована"],
 ["ТЦ F", "из E", "8", "E", "зафиксирована"],
],
},
"bellman-ford": {
 title: "Векторный граф релаксации (Форд–Беллман)",
 nodes: [
 { id: "S", label: "S", x: 40, y: 60, dist: "0", s},
 { id: "A", label: "A", x: 130, y: 30, dist: "4", s},
 { id: "B", label: "B", x: 130, y: 90, dist: "5", s},
 { id: "C", label: "C", x: 220, y: 60, dist: "3", s},
 { id: "D", label: "D", x: 310, y: 30, dist: "6", s},
 { id: "E", label: "E", x: 310, y: 90, dist: "7", s},
 { id: "F", label: "F", x: 400, y: 60, dist: "8", s},
],
 edges: [
 { u: "S", v: "A", w: 4, highlighted: true },
 { u: "S", v: "B", w: 5, highlighted: true },
 { u: "B", v: "C", w: -2, highlighted: true },
 { u: "A", v: "C", w: 1 },
 { u: "C", v: "D", w: 3, highlighted: true },
 { u: "C", v: "E", w: 4, highlighted: true },
 { u: "D", v: "F", w: 2 },
 { u: "E", v: "F", w: 1, highlighted: true },
 { u: "D", v: "A", w: -2 },
],
}

```

```

 { u: "2", v: "5", w: 5 },
 { u: "3", v: "6", w: 2, highlighted: true },
 { u: "5", v: "6", w: 6 },
 { u: "6", v: "7", w: 3, highlighted: true },
],
 table: [
 ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)"],
 ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлено"],
 ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавлено"],
 ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлено"],
 ["(2, 3)", "3", "добавлено (объединение)", "добавлено"],
 ["(6, 7)", "3", "добавлено (финиш)", "добавлено в"],
],
 },
};

```

/\*\* A4 в пунктах. \*/

```

const PAGE = { width: 595.28, height: 841.89, margin: 40 };
const CONTENT_W = PAGE.width - PAGE.margin * 2;
const TOP = PAGE.margin;
const BOTTOM = PAGE.height - PAGE.margin - 16;

```

/\* \_\_\_\_\_ шрифты \_\_\_\_\_ \*/

```

const CANDIDATES = {
 regular: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans.ttf",
 "/usr/local/share/fonts/DejaVuSans.ttf",
 "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
 "C:/Windows/Fonts/arial.ttf",
],
 bold: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
 "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
 "C:/Windows/Fonts/arialbd.ttf",
],
};

```

```

async function loadAppData() {
 ensureDir(path.dirname(ENTRY));
 fs.writeFileSync(
 ENTRY,
 [
 'export { chapters, chapterTopics } from "../src/'
 'export { PAGE_SYNC } from "../src/data/vizSync";
 'export { VIZ_REGISTRY } from "../src/components/'
 'export { quizzes } from "../src/data/quizzes";',
].join("\n")
);
 const res = await build({
 entryPoints: [ENTRY],
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 logLevel: "silent",
 loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "data:
 ss": "empty" },
 });
 return import("data:text/javascript;base64," + Buffer
}

```

/\* ————— снимки визуализаций ————— \*/

```

function shotsFor(chapterId) {
 const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
 for (const dir of dirs) {
 if (fs.existsSync(dir)) {
 const files = fs
 .readdirSync(dir)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .filter((f) => f === `${chapterId}.png` || f.st
 .sort();
 if (files.length > 0) {

```

```

 });
 this.doc.text(`${this.footerTitle} · ${this.pages}`
 width: CONTENT_W * 0.38,
 align: "right",
 lineBreak: false,
 ellipsis: true,
 });
 this.doc.page.margins.bottom = prevBottom;
 this.doc.y = this.cy;
}

newPage(footerTitle) {
 this.stampFooter();
 if (footerTitle !== undefined) this.footerTitle = footerTitle;
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
 }

firstPage(footerTitle) {
 if (footerTitle !== undefined) this.footerTitle = footerTitle;
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
 }

/** Сколько места нужно, чтобы элемент не оторвался от
ensure(h, extra = 0) {
 if (this.cy + h + extra > BOTTOM) this.newPage();
}

height(text, { font, size, width, lineGap = 0 }) {
 return this.doc.font(font).fontSize(size).heightOfS
}

/**
* Режет текст на фрагменты не выше max пунктов: сначала
* предложений, слишком длинное предложение — по слов

```



```

rule(color = "#c7d2fe", width = 1) {
 const y = this.cy;
 this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin,
 this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true }) = {}
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
 this.cy += gapBefore;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 if (rule) this.rule();
}

h2(rawText, { size = 13.5, color = "#1e293b" } = {}) = {}
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 6, 20);
 this.cy += 6;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(
 this.cy = Math.max(this.cy, this.doc.y) + 3;
}

h3(rawText, { size = 11.2, color = "#334155" } = {}) = {}
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 4, 16);
 this.cy += 4;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(
 this.cy = Math.max(this.cy, this.doc.y) + 2;
}

```

```

 });
}

/** Код: режим на фрагменты по высоте страницы, каждый
code(rawText, { size = 8.0 } = {}) {
 const font = this.fonts.mono;
 const inner = CONTENT_W - 22;
 const lines = String(rawText ?? "").replace(UNSUPPORTED);
 // собираем фрагменты, которые влезают в одну рамку
 let i = 0;
 while (i < lines.length) {
 // Если у низа страницы осталось меньше минимальной
 // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit с
 // в обход рендерера (без колонтитула и нумерации)
 if (BOTTOM - this.cy - 16 < 60) this.newPage();
 const avail = BOTTOM - this.cy - 16;
 const maxH = avail;
 let chunk = [];
 let h = 0;
 while (i < lines.length) {
 const candidate = chunk.concat(lines[i]).join("\n");
 const ch = this.height(candidate, { font, size,
 if (chunk.length && ch > maxH) break;
 chunk.push(lines[i]);
 h = ch;
 i += 1;
 }
 const body = chunk.join("\n");
 const boxH = h + 14;
 this.ensure(boxH);
 const y0 = this.cy;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH);
 this.doc.font(font).fontSize(size).fillColor("#1f77b4");
 this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
 if (i < lines.length) this.newPage();
 }
}

```

```

 if (e.w !== undefined) {
 const mx = (x1 + x2) / 2;
 const my = (y1 + y2) / 2;
 this.doc
 .circle(mx, my, 7)
 .fillAndStroke("#ffffff", strokeColor);
 this.doc
 .font(this.fonts.bold)
 .fontSize(7)
 .fillColor("#0f172a")
 .text(String(e.w), mx - 7, my - 3.5, { width:
 }
 }

 // Вершины графа
 for (const n of nodes) {
 const nx = PAGE.margin + n.x;
 const ny = startY + n.y;
 const r = 12;

 const fillColor = n.status === "done" ? "#dcfce7"
 const strokeColor = n.status === "done" ? "#16a34a"

 this.doc
 .circle(nx, ny, r)
 .lineWidth(1.5)
 .fillAndStroke(fillColor, strokeColor);

 this.doc
 .font(this.fonts.bold)
 .fontSize(8.5)
 .fillColor("#0f172a")
 .text(String(n.label ?? n.id), nx - r, ny - 4,

 if (n.dist !== undefined) {
 this.doc
 .font(this.fonts.mono)
 .fontSize(7)

```

```

 return h;
 };

 this.ensure(rowHeight(header) + 8, 14);
 let y = this.cy;
 rows.forEach((cells, idx) => {
 const row = [];
 for (let i = 0; i < cols; i++) row.push(cells[i]);
 const h = rowHeight(row);
 if (y + h > BOTTOM) {
 this.cy = y;
 this.newPage();
 y = this.cy;
 if (idx > 0) y += drawRow(header, y, true, false);
 }
 y += drawRow(row, y, idx === 0, idx % 2 === 1);
 });
 this.cy = y + 8;
}

quote(rawText) {
 const text = plain(rawText);
 if (!text) return;
 const font = this.fonts.oblique ?? this.fonts.regular;
 const width = CONTENT_W - 22;
 const opts = { font, size: 9.5, width, lineGap: 1 };
 // Длинная цитата режется по страницам заранее: рис
 // текст уйдёт за BOTTOM и pdfkit добавит страницу
 const chunks = this.splitToFit(text, opts, BOTTOM);
 chunks.forEach((chunk) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 10);
 const y0 = this.cy;
 this.doc.rect(PAGE.margin, y0, 3, h + 4).fill("#a6a6a6");
 this.doc.font(font).fontSize(9.5).fillColor("#37478c");
 this.cy = y0 + h + 10;
 if (this.cy > BOTTOM) this.newPage();
 });
};

```

```

 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;
 const taken = [];
 let h = 0;
 while (rest.length) {
 const ch = this.height(rest[0], { font: this.fonts.regular });
 if (taken.length && h + ch > avail) break;
 taken.push(rest.shift());
 h += ch;
 }
 const boxH = headH + h + 12;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH, 10);
 this.doc.font(this.fonts.bold).fontSize(9.2).fill('black').text(
 'СБОРКА',
 PAGE.margin + 10, y0 + 7 + headH);
 for (const l of taken) {
 this.doc.font(this.fonts.regular).fontSize(size).fill('black').text(
 l,
 PAGE.margin + 10, y += this.height(l, { font: this.fonts.regular, size }));
 }
 this.cy = y0 + boxH + 7;
 if (rest.length) this.newPage();
 }
}
}

```

/\* \_\_\_\_\_ сборка \_\_\_\_\_ \*/

```

async function main() {
 const fonts = {
 regular: pick(CANDIDATES.regular),
 bold: pick(CANDIDATES.bold),
 oblique: pick(CANDIDATES.oblique),
 mono: pick(CANDIDATES.mono),
 };
 if (!fonts.regular) throw new Error("Не найден TTF-шрифт");
 fonts.bold = fonts.bold ?? fonts.regular;
}

```

```

 { size: 9.6, color: "#374151" }
 });
 R.para(`Дата сборки: ${today}`, { size: 9, color: "#647E8F" });

 const missingViz = chapters.filter((c) => !VIZ_REGISTERED[c.id]);
 R.box("Как устроено пособие", [
 "• Страница = один или несколько билетов: текст, анимация, видео",
 "• У каждой страницы есть интерактивная демонстрация. Если есть, то ссылки на неё

 совпадают с её названием.",
 "• В этом PDF демонстрации показаны снимками (когда вы нажимаете на них, они

 управляют демонстрацией).",
 missingViz.length
 ? `• Страницы без демонстрации: ${missingViz.map(c => c.id).join(", ")}`
 : "• Демонстрация и синхронизация с компилятором",
]);

 /* — содержание — */
 R.newPage("Содержание");
 R.h1("Содержание", { size: 18 });
 chapters.forEach((c, i) => {
 const topics = chapterTopics[c.id] ?? [];
 R.para(
 `${String(i + 1).padStart(2, "0")}. ${c.title}${topics.length > 0 ? " — " : ""} `,
 { size: 9.3, gapAfter: 2 }
);
 });

 /* — темы — */
 let shotsTotal = 0;
 for (const chapter of chapters) {
 const topics = chapterTopics[chapter.id] ?? [];
 const shortTitle = chapter.title.replace(/^\d+[\.\-\]/g, "");
 R.newPage(shortTitle);

 if (topics.length) {
 R.para(`Билет${topics.length > 1 ? "ы" : ""} ${topics.map(t => t.id).join(", ")}`,
 { size: 9.3, gapAfter: 2 });
 }
 }

```

```

 if (viz.hint) R.para(viz.hint, { size: 9.1, color: "black" });
 }

 if (vecViz) {
 if (vecViz.nodes && vecViz.edges) {
 R.vectorGraph(vecViz.nodes, vecViz.edges, { height: 150 });
 }
 if (vecViz.table) {
 R.table(vecViz.table);
 }
 }

 for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const demo = key.includes("#") ? key.split("#")[0] : "";
 const title = demo ? `Вкладка «${demo}» ${sync.variables.length} переменных` : "";
 const lines = [];
 if (sync.stepNote) lines.push(`Шаг демонстрации: ${sync.stepNote}`);
 sync.variables.forEach((v) => lines.push(`• ${v.name} = ${v.value}`));
 lines.push("• Референсный код открывается в интернете");
 if (lines.length) R.box(title, lines);
 }
}

/* блиц: те же вопросы, что в приложении, но сразу по 5 вопросов
const quiz = (quizzes && quizzes[chapter.id]) || [];
if (quiz.length) {
 R.h2("Блиц: вопросы по теме");
 quiz.forEach((q, qi) => {
 R.para(`${qi + 1}. ${q.question}`, { size: 9.6, color: "black" });
 q.options.forEach((option, oi) => {
 R.bullet(oi === q.correctIndex ? `${option} — верный ответ` : `${option}`);
 });
 R.para(q.explanation, { size: 8.8, color: "#4b596b" });
 });
}
}

```

```
*
* Склеивает отрендеренные страницы tmp/export-pages/pages/
* в единый документ public/export/full_code_all_pages.pdf
*/
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие — полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });
};
```



```

* npm run export:shots # сервер уже подня
* SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export
*
* В CI это делает rebuild-pdf.yml: сборка → vite preview
* npm run export:guide. Локально без браузера скрипт пр
* честно пишет, что снимки добавляются при сборке в CI
*/
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
import { ROOT } from "./common.mjs";

const OUT_DIR = path.join(ROOT, "tmp", "viz-shots");
const BASE_URL = process.env.SHOTS_BASE_URL || "http://

/** Вкладки, которые стоит снять отдельно: глава → подп
const TABS = {
 "sparse-table": ["1D", "2D: построение", "2D: запрос"
 mst: ["Краскал", "Прим", "Борувка"],
 treap: ["Собрать", "Split", "Merge"],
 "splay-tree": ["Обход", "Повороты", "Прогулка"],
};

const slug = (s) =>
 s
 .toLowerCase()
 .replace(/[^a-za-яё0-9]+/gi, "-")
 .replace(/^-+|-+$/g, "")
 .slice(0, 24);

async function vizIds() {
 const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 },
```

```

 const root = document.querySelector("#chapter-

 if (!root) return false;

 const btn = Array.from(root.querySelectorAll(

 (b.textContent ?? "").trim().startsWith(label) ? b : null

));

 if (!btn) return false;

 btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));

 return true;

 }, tab);

 if (!clicked) continue;

 await new Promise((r) => setTimeout(r, 1200));

 const el = await page.$("#chapter-viz");

 await el.screenshot({ path: path.join(OUT_DIR, `chapter-${tab}.png`),

 done += 1;

 } catch (error) {

 console.log(`[export:shots]   вкладка «${tab}»   FAIL: ${String(error)}`);

 }

}
console.log(`[export:shots] ok ${id}`);
} catch (error) {
 failed += 1;
 console.log(`[export:shots] FAIL ${id}: ${String(error)}`);
}
}

const manifest = fs
 .readdirSync(OUT_DIR)
 .filter((f) => /\. (png|jpe?g)$/i.test(f))
 .sort()
 .map((f) => ({ file: f, name: f.replace(/\. (png|jpe?g)/, "") }));
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), JSON.stringify(manifest));

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков: ${failed}`);
// Пропущенные страницы не повод ронять пайплайн: PDF документация

// текстовое описание демонстрации.
process.exit(0);

```

```
[/^src\\/, "Ядро приложения"],
[/^scripts\\/, "Скрипты сборки и экспорта"],
[/^\\.github\\/, "CI / автоматизация"],
[/^[^/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep).join("/");
 if (e.isDirectory()) {
 walk(abs);
 } else {
 files.push(rel);
 }
 }
 };
 walk(ROOT);
 }
}
```

```

 return na - nb;
 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { time: true });
 const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ ИСКЛЮЧЕНЫ)");
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода:            ${humanSize(totalBytes)}`);
 push(`Глав/билетов в пособии:          ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
 push(HR);
 chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
 push(HR);
 let current = null;

```

```
const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);

main();
```

---

ФАЙЛ 96/111: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

---

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 */
* код -> full_code_all_pages.txt (collect-code.mjs)
* txt -> page-XXXX.png (render-pages.mjs)
* png -> full_code_all_pages.pdf (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся в CI) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
```



```

 while ((a = attrRe.exec(attrsStr || "")) !== null) {
 const node = { tag, attrs, children: [] };
 stack[stack.length - 1].children.push(node);
 if (!VOID.has(tag) && !selfClose) stack.push(node);
 continue;
 }

 if (textChunk !== undefined && textChunk.length) {
 stack[stack.length - 1].children.push({ tag: "#text", text: textChunk });
 }
 }
 return root;
}

```

```

const decodeEntities = (s) =>
 s
 .replace(/</g, "<")
 .replace(/>/g, ">")
 .replace(/"/g, '"')
 .replace(/'/g, "'")
 .replace(/ /g, "\u00a0")
 .replace(/—/g, "—")
 .replace(/–/g, "–")
 .replace(/«/g, "«")
 .replace(/»/g, "»")
 .replace(/…/g, "...")
 .replace(/×/g, "×")
 .replace(/≤/g, "≤")
 .replace(/≥/g, "≥")
 .replace(/≠/g, "≠")
 .replace(/∞/g, "∞")
 .replace(/→/g, "→")
 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+)/g, (_, d) => String.fromCharCode(d));

```

```

const clean = (s) => decodeEntities(s).replace(/\s+/g, " ");

```

```
 return node.children.map(collectRawPre).join("");
 }

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
 const rows = [];
 const visit = (n) => {
 if (n.tag === "tr") {
 const cells = n.children.filter((c) => c.tag === "td");
 if (cells.some((c) => c.length)) rows.push(cells);
 return;
 }
 n.children.forEach(visit);
 };
 node.children.forEach(visit);
 return rows;
}

/**
 * Обход дерева: собираем блоки в порядке документа.
 * @returns {Array<{type:string,text?:string,rows?:string[]}>}
 */
export function htmlToBlocks(html) {
 const root = parseHtml(html);
 const out = [];

 const push = (block) => {
 const prev = out[out.length - 1];
 if (prev && prev.type === block.type && prev.text === block.text) {
 out.push(block);
 }
 };

 const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal });
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c, depth + 1);
 });
 };
}
```



```

 }
 if (SKIP.has(c.tag)) continue;
 if (!BLOCK.has(c.tag)) {
 buf += inlineText(c) + " ";
 continue;
 }
 if (c.tag === "ul" || c.tag === "ol") {
 flush();
 emitList(c, listDepth);
 continue;
 }
 flush();
 walk(c, listDepth);
}
flush();
return;
}

// листовой блок: один блок по типу тега
const text = inlineText(node);
if (!text) return;
if (HEADING.has(node.tag)) push({ type: "heading",
else if (node.tag === "li") emitListItem(node, null
else if (node.tag === "summary") push({ type: "summ
else if (node.tag === "blockquote") push({ type: "q
else if (node.tag === "figcaption") push({ type: "c
else push({ type: "p", text });
});

root.children.forEach((c) => walk(c, 0));
return out;
}

/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat

```

```

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/^
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}

function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

```

```
 if (done % 25 === 0 || done === total) {
 process.stdout.write(` отрендерено ${done}.
 }
 }
}

await Promise.all(Array.from({ length: concurrency },
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.p
if (missing.length) throw new Error(`Не отрендерены с

const bytes = jobs.reduce((s, j) => s + fs.statSync(j
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(` выход: ${path.relative(ROOT, PAG
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});
```

---

ФАЙЛ 99/111: scripts/verify/browser-checks.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 1065 | РА

---

```
/**
 * DOM-харнес: настоящее приложение в jsdom + настоящий
 * Монтирует App, кликает кнопки, печатает в редакторе
 * визуализациям матрицы чужих размерностей и ловит пад
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
```

```
function allButtons(): string[] {
 return Array.from(document.querySelectorAll("button")
 (b) => (b.getAttribute("aria-label") || b.getAttribute("title"))
);
}
```

```
/** Кнопки без доступного имени: ни aria-label, ни title */
function namelessButtons(scope: ParentNode = document): HTMLButtonElement[] {
 return Array.from(scope.querySelectorAll("button"))
 .filter((b) => {
 const name = [b.getAttribute("aria-label"), b.getAttribute("title")]
 .filter(Boolean)
 .join(" ")
 .replace(/\s+/g, " ")
 .trim();
 return name.length === 0;
 })
 .map((b) => `<button class="${(b.className || "").trim()}"`);
}
```

```
function findButton(match: string | RegExp): HTMLButtonElement | null {
 const list = Array.from(document.querySelectorAll("button"));
 return list.find((b) => {
 const hay = [b.getAttribute("aria-label"), b.getAttribute("title")]
 .filter(Boolean)
 .join(" ")
 .replace(/\s+/g, " ")
 .trim();
 return typeof match === "string" ? hay.includes(match) : match.test(hay);
 });
}
```

```
async function click(el: Element | undefined | null, label: string) {
 if (!el) throw new Error(`не найдена ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}
```

```
/** Панель компилятора – все поиски внутри неё, чтобы не
```

```
const RUN_DONE = [
 "Трасса готова автоматически",
 "Нет шагов трассы",
 "Пустой код",
 "Код не выполнен",
 "Выполнение остановлено после",
 "Не удалось выполнить код",
 "Ошибка:",
];

let root: Root | null = null;
let host: HTMLElement | null = null;

function freshHost(): HTMLElement {
 const div = document.createElement("div");
 document.body.appendChild(div);
 return div;
}

async function mountApp(topic: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) {
 host.remove();
 }
 host = freshHost();
 window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
}
```

```

function domDefects(scope: HTMLElement | null): string[] {
 if (!scope) return [];
 const found: string[] = [];
 const walker = document.createTreeWalker(scope, 4 /*
 let node = walker.nextNode() as Text | null;
 while (node) {
 const text = (node.nodeValue ?? "").replace(/\s+/g, " ");
 if (text.length > 0 && text.length < 60) {
 for (const bad of BAD_DOM) {
 if (bad.re.test(text)) found.push(`${bad.why}: `);
 }
 }
 node = walker.nextNode() as Text | null;
 }
 return [...new Set(found)];
}
```

```
async function test(group: string, name: string, fn: () => void) {
 if (skipGroups.has(group[0])) return true;
 consoleErrors = [];
 const t0 = Date.now();
 let ok = true;
 let detail = "";
 try {
 const d = await fn();
 detail = d ?? "";
 } catch (error) {
 ok = false;
 detail = error instanceof Error ? `${error.message}` : String(error);
 }
 const ms = Date.now() - t0;
 const errs = consoleErrors.filter((e) => !/not wrapped/.test(e));
 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length} errors)
 }
 results.push({ group, name, ok, ms, detail, consoleErrors });
 const mark = ok ? "PASS" : "FAIL";
```

```
});
```

```
await test(group, `${chapter.id}: панель открывается
 await mountApp(chapter.id);
 const toggle = findButton("Боковая панель Python");
 await click(toggle, "кнопка панели Python в навбаре");
 const ta = textarea();
 if (!ta) throw new Error("редактор (textarea) не найден");
 if (!ta.value.includes("# «")) return `редактор еще не открыт`;
 const eye = findButton(/эта?лонный код/i);
 if (!eye) throw new Error("нет кнопки-«глаза» (вспомогательная)");
 const run = findButton(/Запустить код|автопроход/i);
 if (!run) throw new Error("нет кнопки запуска ▶");
 return `кнопок в панели: ${allButtons().length}`;
});
```

```
await test(group, `${chapter.id}: автозапуск инициализации
 await mountApp(chapter.id);
 await click(findButton("Боковая панель Python"), "кнопка панели Python");
 const done = await waitFor(() => RUN_DONE.some((m) => m === chapter.id));
 if (!done) throw new Error(`запуск не завершился`);
 const msg = RUN_DONE.find((m) => bodyText().includes(m));
 const sync = PAGE_SYNC[chapter.id];
 if (!sync?.code) return `кода синхронизации нет: ${chapter.id}`;
 if (/Ошибка|Не удалось/.test(msg)) throw new Error(msg);
 // после настоящего прогона визуализация обязана быть активной
 const linked = await waitFor(() => /данные из комманды/);
 if (!linked) return `${msg}; визуализация осталась неактивной`;
 return `${msg} → визуализация переключилась на данные`;
});
```

```
}
```

```
/* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полноранговой матрицы
const floydCases: { name: string; vars: Record<string, number> }[] = [
 { name: "5×5 (меньше демо-7×7)", vars: { n: 5, k: 1, a: [
 [1, 2, 3, 4, 5], [2, 3, 4, 5, 6], [3, 4, 5, 6, 7], [4, 5, 6, 7, 8], [5, 6, 7, 8, 9]
], [9, 6, 9, 9, 0]] }, expect: "5×5" },
 { name: "10×10 (больше демо – старый краш nodeNames)", vars: { n: 10, k: 1, a: [
 from({ length: 10 }, (r, c) => (r === c ? 0 : (r + c) % 10))
] } }
];
```

```

 pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2: 1,
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2d-bu",
 9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
 { viz: "sparse-table", chapter: "sparse-table", name: "sparse-table",
 [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не совпадает|данные/ },
 { viz: "sparse-table", chapter: "sparse-table#2d-bu", name: "sparse-table",
 0, i: 2, j: 3, st: Array.from({ length: 4 }, () => Array(4).fill(0)),
 pa/ },
 { viz: "sparse-table", chapter: "sparse-table#2d-qu", name: "sparse-table",
 30, j2: 40, res: 2 }, expect: /не совпадает|данные/ },
];
 for (const c of gridCases) {
 await test("С. матричные страницы", `${c.viz}: ${c.name}`);
 await mountViz(c.viz);
 await act(async () => {
 emitVizState(c.viz, { line: 2, func: "<module>" });
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s/g, "");
 if (!c.expect.test(txt)) throw new Error(`ожидали ${c.expect}, а получили ${txt}`);
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
 return /размерность не совпадает/.test(txt) ? "проблема с размерностью" : "ошибка";
 });
}

```

```

await test("С. матричные страницы", "2D-построение: sparse-table");
await mountViz("sparse-table");
const A8 = Array.from({ length: 8 }, (_, i) => Array(8).fill(0));
await act(async () => {
 emitVizState("sparse-table", { line: 7, func: "<module>" });
 await sleep(40);
});
const grids = () => Array.from((host ?? document).querySelectorAll("table"), (t) => {
 let g0 = grids()[0];
 if (!g0) throw new Error("нет ни одной матрицы блока");
 const firstCell = () => (g0.children[0]?.textContent ?? "");
 return firstCell();
});

```



```

 await sleep(30);
 });
 const step2 = findButton(/^Шаг$|Шаг /) as HTMLBut
 if (step2 && step2.disabled) throw new Error("кно
 return `кнопок до/после: ${before}/${allButtons()}
 });
}

```

/\* — 5. Все референсные коды: настоящий Python → нас

```

const syncKeys = Object.keys(PAGE_SYNC).filter((k) =>
for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const chapterId = key.split("#")[0];
 await test("Е. эталонный код", `${key}: выполняется
 const raw = await py.runPythonAsync(buildDebugRun
 const result = JSON.parse(String(raw)) as DebugRe
 if (result.error) throw new Error(`Python-ошибка:
 if (result.steps.length === 0) throw new Error("н
 // объявленные переменные синхронизации реально п
 const seen = new Set<string>();
 for (const s of result.steps) for (const n of Obj
 for (const n of Object.keys(result.finalValues ??
 // в документации имя может быть видом «хк / х»,
 // проверяем, что хотя бы один python-идентификат
 const declared = sync.variables.map((v) => v.name
 const ident = (t: string) => /^[A-Za-z_][A-Za-z0-
 const missing = declared
 .map((raw) => raw.split(/[\/[\],,]\s+или\s+/).m
 .filter((alts) => alts.some(ident))
 .filter((alts) => !alts.some((a) => ident(a) &&
 .map((alts) => alts.join(" / "));
 if (missing.length) throw new Error(`код не созда
);
 }
}

```

```

if (!VIZ_REGISTRY[chapterId]) return `шагов ${res
await mountViz(chapterId);
const finalVars = result.finalValues ?? result.st
await act(async () => {

```



```

const linked = await waitFor(() => /данные из компи.
if (!linked) throw new Error(`эталон выполнен до ко

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "гла
const cleared = await waitFor(() => bodyText().incl
if (!cleared) throw new Error(`после выключения гла
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после вы
if (ta3.value.trim() !== "") throw new Error(`глаз
if (!/встроенное демо/.test(bodyText())) throw new
if (/данные из компилятора/.test(bodyText())) throw
const afterButtons = allButtons();
const lost = editorButtons.filter((b) => b && !afte
if (lost.length > 2) throw new Error(`после снятия

// и возвращает инициализацию страницы
await click(findButton("Сбросить к шаблону страницы
const back = await waitFor(() => textarea()?.value
if (!back) throw new Error(`и не вернул инициализац
return "эталон вставляется в редактор, ведёт демонс
));

```

```

await test("F. редактор", "код не перетекает между стр
await mountApp(ch);
await click(findButton("Боковая панель Python"), "п
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint
await waitFor(() => RUN_DONE.some((m) => bodyText())
if (!bodyText().includes("42")) throw new Error("св
// переходим на другую страницу, не закрывая панель
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "п
const ta2 = textarea();
if (!ta2) throw new Error("на новой странице нет ре
if (ta2.value.includes("MY-MARKER-14")) throw new E
if (!ta2.value.includes("# «")) throw new Error("на

```

```

 await waitFor(() => panel() !== null, 4000);
 await click(findButton("Заккрыть компилятор"), "крес
 await settle(150);
 if (panel()) throw new Error("панель не закрылась")
 const backToDemo = await waitFor(() => /встроенное
 if (!backToDemo) throw new Error(`после закрытия па
 await settle(3000); // ждём, пока догоняющий прогон
 if (/данные из компилятора/.test(bodyText())) {
 throw new Error("закрытая панель всё-таки опублик
 }
 if (!/встроенное демо/.test(bodyText())) throw new
 return "панель закрыта во время выполнения – демонс
));

```

```

await test("Н. доступность", "панель: все кнопки подп
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const namelessEditor = namelessButtons(panel() ?? d
 const editorBtns = allButtons().length;
 const listing = await waitFor(() => !!findButton(/О
 if (!listing) throw new Error("панель не показала тр
 const namelessListing = namelessButtons(panel() ??
 const listingBtns = allButtons().length;
 if (namelessEditor.length) throw new Error(`в редак
 , ")} `);
 if (namelessListing.length) throw new Error(`в листин
 (" , ")} `);
 const required = [/Остановить прогон трассы/, /Сверн
 начало трассы/, /В конец трассы/, /автопроход/i]
 const btns = allButtons().join(" | ");
 const missing = required.filter((re) => !re.test(btns)
 if (missing.length) throw new Error(`в листинге нет
 return `редактор: ${editorBtns} кнопок, листинг: ${
 });

```

```

await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
 // 1) демо: компилятор ещё ничего не присылал

```

```

 if (root()!.querySelector("input")) throw new Error
 await click(toggle(), "скрыть");
 if (!/lg:w-0/.test(root()!.className)) throw new Error
 await click(toggle(), "развернуть");
 if (!/lg:w-56/.test(root()!.className)) throw new Error
 return "широкое → узкое (темы на месте, поиска нет)
 });

```

```

await test("J. шапка", "кнопки загрузки не прячутся и
 await mountApp("segment-trees");
 const menu = document.querySelector("#download-menu
 if (!menu) throw new Error("нет меню загрузки в шап
 const want = [
 { id: "download-guide-pdf-btn", file: "export/guide
 { id: "download-pdf-btn", file: "export/full_code
 { id: "download-txt-btn", file: "export/full_code
];
 for (const w of want) {
 const el = menu.querySelector<HTMLAnchorElement>(
 if (!el) throw new Error(`в шапке нет кнопки «${w
 const href = el.getAttribute("href") ?? "";
 if (!href.endsWith(w.file)) throw new Error(`«${w
 const name = [el.getAttribute("aria-label"), el.g
 if (name.trim().length < 5) throw new Error(`у кн
 }
 if (!menu.querySelector("#download-html-btn")) throw
 const py = document.querySelector<HTMLButtonElement>
 if (!py) throw new Error("нет кнопки панели Python"
 if (!py.getAttribute("aria-label")) throw new Error
 if (!py.getAttribute("title")) throw new Error("у к
 return "4 загрузки на месте, href ведут на файлы, к
});

```

```

await test("I. ссылки", "упоминание «билет 5» кликабе
 await mountApp("sparse-table");
 const links = Array.from(document.querySelectorAll<
 if (links.length === 0) throw new Error("на страниц
 const toPrefix = links.find((a) => a.dataset.goto =

```

```

 }
 // порядок чтения: номера билетов сверху вниз не уб
 const nums = Array.from(sidebar.querySelectorAll("b
 .map((b) => (b.querySelector("span")?.textContent
 .filter((t) => /^\\d/.test(t))
 .map((t) => Number.parseInt(t, 10)));
 const ticketed = chapters.filter((c) => (chapterTop
 if (nums.length !== ticketed) {
 throw new Error(`номера билетов видны у ${nums.le
 }
 for (let i = 1; i < nums.length; i += 1) {
 if (nums[i] < nums[i - 1]) throw new Error(`поряд
 }
 if (!sidebar.textContent?.includes("Порядок чтения"
 throw new Error("в содержании нет подсказки, в ка
 }
 return `разделов ${headers.length}, 11+12 соседу
));

```

```

await test("I. ссылки", "введение: базовая база, обзор
 await mountApp("intro");
 const txt = bodyText();
 for (const must of ["Дерево как массив", "Куча", "О
 if (!txt.includes(must)) throw new Error(`во введе
 }
 if (!txt.includes("2n") || !txt.includes("2n+1")) t
 if (!txt.includes("≤ 1")) throw new Error("во введе
 if (/Обзор: Кратчайшие пути/.test(txt)) throw new E
 const goto = Array.from(document.querySelectorAll("
 if (!goto.includes("dijkstra")) throw new Error(`об
 if (domDefects(document.querySelector("#chapter-viz
 throw new Error("артефакты в DOM введения");
 }
 return "введение = куча/очередь/бинпоиск/сортировка
));

```

```

await test("I. блиц", "вопросы отвечают, объясняют и
 await mountApp("sparse-table");

```

```

const rows = Array.from(document.querySelectorAll
const row = rows.find((r) => /^\\s*Память/.test(r)
if (!row) throw new Error(`на странице «${topic}»
return Array.from(row.querySelectorAll("td")).map
});
const sparse = await memoryRow("sparse-table");
const prefix = await memoryRow("prefix-sums-2d");
// билет 2: у префиксной $N \times M$, у разреженной – с л
if (!/log/.test(sparse.join(" "))) throw new Error(
;
const prefixCell = sparse.find((c) => !/log/.test(c)
if (!prefixCell) throw new Error(`в сравнении билет
// билет 5: зеркально – у префиксной без логарифмов
if (!/log/.test(prefix.join(" "))) throw new Error(
;
if (/log/.test(prefix[1] ?? "")) throw new Error(`в
const txt = bodyText();
if (/префиксн[^\.]{0,40}(жирн|больше памяти)/i.test(
throw new Error("на странице билета 5 префиксная
}
await mountApp("sparse-table");
const full = bodyText();
if (!/100 (коржей|копий|матриц)|сто (коржей|матриц)/
throw new Error("на странице билета 2 не объяснен
}
if (!/не конкуренты|разные задачи/i.test(full)) {
throw new Error("на странице билета 2 не сказано,
}
// аналогии: торт стоит РЯДОМ с вафельными трубочка
for (const topic of ["sparse-table", "prefix-sums-2d
await mountApp(topic);
const full = bodyText();
if (!/Аналогия 1: Сетка из вафельных трубочек/.te
if (!/Аналогия 2: Торт-пирамидка/.test(full)) thro
if (!/крем обратно/.test(full)) throw new Error(`
if (!/трубочки можно не только докладывать, но и
throw new Error(`на «${topic}» не объяснено, что
}

```

```

const ready = await waitFor(() => bodyText().includes("4x4"))
if (!ready) throw new Error(`своя программа не выполнилась`)
if (!bodyText().includes("[0, 3, 5, 6]")) {
 throw new Error(`Флойд на своей матрице 4x4 посчитал`)
}
if (!textarea()) throw new Error("редактор исчез, попробуйте еще раз")
await click(findButton("В конец трассы"), "в конец трассы")
const linked = await waitFor(() => /данные из компилятора/)
if (!linked) throw new Error("демонстрация не принята")
const page = bodyText();
if (!/4x4/.test(page)) throw new Error(`демонстрация не принята`)
const defects = domDefects(document.querySelector("#defects"))
if (defects.length) throw new Error(`артефакты в демонстрации`)
return "набрал 4x4 с INF=999 → d[0] = [0, 3, 5, 6]";
});

```

```

await test("К. свой код", "своя разреженная таблица на Python")
await mountApp("sparse-table");
await click(findButton("Боковая панель Python"), "поиск")
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(
 ta,
 [
 "a = [5, 1, 9, 3, 7, 2]",
 "n = len(a)",
 "LOG = 3",
 "i, j = 0, 0",
 "st = []",
 "for row in a:",
 " st.append([row] + [None] * (LOG - 1))",
 "for j in range(1, LOG):",
 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j)][j - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("4x4"))

```



```

 if (host) host.remove();
 host = freshHost();
 window.history.replaceState({}, "", "/?lite=1&topic=");
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(120);
 const txt = bodyText();
 if (txt.length < 2000) throw new Error(`lite-режим: текст слишком короткий`);
 if (/main\.py|Pyodide/.test(txt)) throw new Error("в lite-режиме не работает main.py");
 if (!/разрежен/i.test(txt)) throw new Error("в lite-режиме не работает неразреженный текст");
 if (findButton("Боковая панель Python")) throw new Error("в lite-режиме не работает боковая панель");
 // текстовая версия: ссылки на другие билеты ведут к главной странице
 // а блиц напечатан списком с ответами
 const liteLinks = Array.from(document.querySelectorAll('a[href="#ch-"]'));
 const bad = liteLinks.filter((a) => !a.getAttribute('href'));
 if (liteLinks.length === 0) throw new Error("в lite-режиме не работает список вопросов");
 if (bad.length) throw new Error(`в lite-версии ${bad.length} ссылок не работает`);
 const unknown = liteLinks.filter((a) => !chapters.some(c => c.id === a.getAttribute('href')));
 if (unknown.length) throw new Error(`в lite-версии ${unknown.length} ссылок не работает`);
 if (!/Блиц: вопросы по теме/.test(txt)) throw new Error("в lite-режиме не работает блиц");
 if (!/Верно:/.test(txt)) throw new Error("в lite-режиме не работает проверка ответа");
 return `${txt.length} знаков, ссылок в lite ${liteLinks.length}`;
 });
}

await test("G. lite-режим", "HTML-книга: оглавление, словарь терминов", async () => {
 const { buildBookHtml } = await import("../src/buildBookHtml.js");
 const html = await buildBookHtml(chapters);
 if (!html.includes('<nav id="toc"')) throw new Error("в lite-режиме не работает оглавление");
 const missing = chapters.filter((c) => !html.includes(c.id));
 if (missing.length) throw new Error(`в книге потеряны ${missing.length} глав`);
 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`в lite-режиме не работает оглавление: ${chapters.length - tocLinks} глав не найдено`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error("в lite-режиме не работает словарь терминов");
 if (!/↑ к оглавлению/.test(html)) throw new Error("в lite-режиме не работает ссылка на оглавление");
 if (html.length < 200_000) throw new Error(`книга слишком маленькая: ${html.length} байт`);
 return `${(html.length / 1024).toFixed(0)} КБ, глав ${chapters.length}`;
});

```

```

await test("G. lite-режим", "HTML-книга: оглавление, словарь терминов", async () => {
 const { buildBookHtml } = await import("../src/buildBookHtml.js");
 const html = await buildBookHtml(chapters);
 if (!html.includes('<nav id="toc"')) throw new Error("в lite-режиме не работает оглавление");
 const missing = chapters.filter((c) => !html.includes(c.id));
 if (missing.length) throw new Error(`в книге потеряны ${missing.length} глав`);
 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`в lite-режиме не работает оглавление: ${chapters.length - tocLinks} глав не найдено`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error("в lite-режиме не работает словарь терминов");
 if (!/↑ к оглавлению/.test(html)) throw new Error("в lite-режиме не работает ссылка на оглавление");
 if (html.length < 200_000) throw new Error(`книга слишком маленькая: ${html.length} байт`);
 return `${(html.length / 1024).toFixed(0)} КБ, глав ${chapters.length}`;
});

```

```

* 6. glossary.simulator существует в реестре и (кроме
* имеет свою страницу — иначе модалка не получит
* 7. glossary.demo — работающая мини-демо;
* 8. у каждого ключа PAGE_SYNC есть запись в реестре
* 9. у каждой страницы есть визуализация;
* 10. страницы связаны перекрёстными ссылками (темы не
*/
```

```
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
const NOTES = [];
const ok = (cond, message) => (cond ? true : (FAILS.push(message), false));
```

```
mkdirSync("tmp/verify", { recursive: true });
writeFileSync(
 "tmp/verify/links-entry.ts",
 [
 'export { chapters, chapterTopics, SECTIONS, section',
 'export { GLOSSARY } from "../../src/data/glossary"',
 'export { VIZ_REGISTRY } from "../../src/components',
 'export { PAGE_SYNC } from "../../src/data/vizSync"',
 'export { quizzes } from "../../src/data/quizzes";',
 'export { MINI_DEMO_KINDS } from "../../src/componen',
].join("\n")
);
```

```
await esbuild.build({
 entryPoints: ["tmp/verify/links-entry.ts"],
 bundle: true,
 outfile: OUT,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 logLevel: "error",
 define: {
 "process.env.NODE_ENV": '"development"',
 },
});
```

```

const bySection = SECTIONS.map((s) => ({
 s,
 nums: chapters.flatMap((c) => (sectionOf(c.id).id =
})));
for (let i = 1; i < bySection.length; i += 1) {
 const prevMax = Math.max(...bySection[i - 1].nums);
 const curMin = Math.min(...bySection[i].nums);
 ok(curMin > prevMax, `разделы пересекаются: «${bySe
}
const all = bySection.flatMap((x) => x.nums);
for (let n = all[0]; n <= all[all.length - 1]; n += 1
 ok(all.includes(n), `в содержании дыра: билет ${n}
}
}

```

/\* 3–5. ссылки внутри текста страниц \*/

```

const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
let ticketMentions = 0;
let explicitLinks = 0;
const withoutLinks = [];
for (const chapter of chapters) {
 const html = chapter.content ?? "";
 let mentionsHere = 0;

```

// 3. упоминания «билет N» — только внутри текста, не

```

const textOnly = html
 .replace(/<pre[\s\S]*?<\/pre>/gi, " ")
 .replace(/<code[\s\S]*?<\/code>/gi, " ")
 .replace(/<[>]*>/g, " ");
for (const m of textOnly.matchAll(TICKET_RE)) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapt
 }
}
ticketMentions += mentionsHere;

```

```

for (const id of registryIds) {
 if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
 FAILS.push(`реестр: визуализация "${id}" не является
 }
}
for (const chapter of chapters) {
 ok(registryIds.includes(chapter.id), `страница «${chapter.id}» не найдена`);
}

/* 96. файлы, на которые ведут кнопки шапки, действительно существуют */
{
 const { existsSync, statSync } = await import("fs");
 for (const file of ["public/export/guide_lite.pdf", "public/export/guide_lite.pdf"]) {
 ok(existsSync(file), `шапка ссылается на ${file}, но файла нет`);
 if (existsSync(file) && statSync(file).size < 1000) {
 FAILS.push(`файл ${file} слишком мал`);
 }
 }
}

/* 10. связанные темы + блиц */
for (const id of withoutLinks) NOTES.push(`нет перекрёстных ссылок на тему ${id}`);
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ? true : false));

console.log("————— ссылки и структура содержания —————");
console.log(` страниц: ${chapters.length} • разделов: ${chapters.length / 10}`);
console.log(` упоминаний «билет N» в тексте: ${ticketMentions.length}`);
console.log(` визуализаций в реестре: ${registryIds.length} + ${GLOSSARY.length}`);
console.log(` блиц есть у ${Object.keys(quizzes).length} тем`);
if (NOTES.length) console.log(` заметки:\n ${NOTES.join("\n")}`);
if (noQuiz.length) console.log(` без блица: ${noQuiz.join(", ")}`);

if (FAILS.length) {
 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(` x ${f}`));
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");

```

```
);
```

```
const built = await esbuild.build({
 entryPoints: [`${OUT_DIR}/contract-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/contract-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 sourcemap: false,
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
});
```

```
void built;
```

```
const mod = await import(`${process.cwd()}/${OUT_DIR}/contract-bundle.mjs`);
```

```
const py = await loadPyodide({ indexURL: "node_modules/pyodide/dist/" });
```

```
const printed = [];
```

```
py.setStdout({ batched: (m) => printed.push(m) });
```

```
py.setStderr({ batched: (m) => printed.push("[stderr] " + m) });
```

```
const only = process.argv.slice(2);
```

```
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/.test(t);
```

```
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python-идентификаторов
```

```
const alts = (raw) => raw.split(/[\/[\]],)|\s+или\s+/).map((s) => s.replace(/ /g, ""));
```

```
const rows = [];
```

```
let problems = 0;
```

```
for (const [key, entry] of Object.entries(mod.sync)) {
```

```
 if (only.length && !only.some((o) => key.includes(o))) continue;
```

```
 printed.length = 0;
```

```
 let result;
```

```
 try {
```

```
 result = JSON.parse(String(await py.runPythonAsync(`import sys; sys.stdout = sys.stderr;`)));
```

```
 } catch (error) {
```

```
 problems++;
```

```
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: ${error}` });
```

```
 continue;
```

```

 if (!isEmpty(v)) sawNonEmpty = true;
 }
 return sawValue && !sawNonEmpty;
 });
 if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

 if (issues.length) problems++;
 rows.push({ key, steps: result.steps.length, vars: se
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===
console.log("ЭТАЛОННЫЕ КОДЫ СТРАНИЦ (настоящий CPython
console.log("=".repeat(96));
for (const r of rows) {
 const mark = r.status === "ок" ? " ок " : " x ";
 console.log(`${mark}${r.key.padEnd(34)} шагов ${String
 === "ок" ? "" : r.status}`);
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.st
process.exit(problems > 0 ? 1 : 0);

```

ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМ

```

/**
 * Браузерное окружение для проверок: jsdom + те API, к
 * Создаётся ПОСЛЕ загрузки Pyodide (иначе pyodide реши
 */
import { JSDOM, VirtualConsole } from "jsdom";

```

```

 addEventListener() {},
 removeEventListener() {},
 dispatchEvent: () => false,
 }));
window.scrollTo = () => {};
window.HTMLInputElement.prototype.scrollToView = () => {};
window.HTMLInputElement.prototype.focus = window.HTMLInputElement.prototype.focus;
window.URL.createObjectURL = window.URL.createObjectURL;
window.URL.revokeObjectURL = window.URL.revokeObjectURL;
Object.defineProperty(window, "innerWidth", { value: window.innerWidth });
Object.defineProperty(window, "innerHeight", { value: window.innerHeight });
Object.defineProperty(window, "devicePixelRatio", { value: window.devicePixelRatio });
window.Element.prototype.getBoundingClientRect = function() {
 return { x: 0, y: 0, top: 0, left: 0, right: 400, bottom: 400 };
};

```

```

const copy = [
 "window",
 "document",
 "navigator",
 "HTMLInputElement",
 "HTMLInputElement",
 "HTMLTextAreaElement",
 "HTMLButtonElement",
 "HTMLAnchorElement",
 "HTMLDivElement",
 "HTMLSelectElement",
 "Element",
 "Node",
 "Event",
 "CustomEvent",
 "MouseEvent",
 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",

```

```
* пользователя вместо захардкоженной демо-сетки.
*/
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
 `${OUT_DIR}/unit-entry.ts`,
 `export { liveMatrix, pickLiveVar, clampIndex } from
);
await esbuild.build({
 entryPoints: [`${OUT_DIR}/unit-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/unit-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 loader: { ".css": "empty" },
 logLevel: "warning",
});
const { liveMatrix, pickLiveVar, clampIndex } = await i

let passed = 0;
const failures = [];
function check(name, actual, expected) {
 const a = JSON.stringify(actual);
 const e = JSON.stringify(expected);
 if (a === e) passed++;
 else failures.push(`${name}\n получено: ${a}\n
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size
};
const cells = (v) => {
 const r = m(v);
```



```

check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"

/* — clampIndex -----
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE_INF

console.log("=".repeat(72));
console.log(`liveMatrix / pickLiveVar / clampIndex: пр
for (const f of failures) console.log(`  x ${f}`);
console.log("=".repeat(72));
process.exit(failures.length ? 1 : 0);

```

ФАЙЛ 104/111: scripts/verify/probe-compiler.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 121 | РАЗМ

```

/**
 * Зонд состояния панели Python во времени: что происхо
 * Запуск: node tmp/harness/run-probe.mjs
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTim
async function settle(ms = 40) {
 await act(async () => {
```

```

 });
}
function findButton(match: RegExp): HTMLButtonElement |
 return Array.from(document.querySelectorAll("button")
 match.test([b.getAttribute("aria-label"), b.getAttr
) as HTMLButtonElement | undefined;
}

export async function probe() {
 const host = document.createElement("div");
 document.body.appendChild(host);
 window.history.replaceState({}, "", "/?topic=dijkstra");
 const root = createRoot(host);
 await act(async () => {
 root.render(<App />);
 });
 await settle(80);

 process.stdout.write("\n=== 1. открыли панель, ждём а
 await click(findButton(/Боковая панель Python/), "пан
 state("t=0");
 for (const t of [300, 1000, 3000, 8000]) {
 await settle(t - (t === 300 ? 0 : 0));
 state(`t+${t}`);
 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.tes
 }

 process.stdout.write("\n=== 2. вводим код ТОЛЬКО с ко
 const ed = editor();
 if (!ed) {
 process.stdout.write(" редактора нет – жмём Esc и
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
 }
 const ed2 = editor();
 if (!ed2) throw new Error("редактор так и не появился

```

Универсальное пособие • полный исходный код

-----  
ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

-----  
# Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в чужих браузерах. Если нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Если нужен DOM 3.12, что грузится в браузере пользователя, только без `jsdom`.

```bash

```
npm run verify          # всё сразу: unit → эталонные коды
npm run verify:unit      # 38 проверок разбора живых данных
npm run verify:code      # 46 эталонных кодов страниц по DOM
npm run verify:dom       # 139 сценариев: страницы, панели
npm run verify:probe     # диагностика панели Python во всех браузерах
```
```

Отчёты кладутся в `tmp/verify/` (каталог в `.gitignore`):  
помарочно по DOM-сценариям, `reference-code.json` – по кодам.

## Что именно проверяется

### `verify:unit` – `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу 2D. Матрица 2D размерности: список списков, рваные строки, словарь слов. Матрица 2D: `(0, 3)` (так снапшотятся tuple-ключи Python), `inf` – числа-строки, обрезка до 64x64. Отдельно – что матрицей 2D: список и словарь по именам (`dist`, `prev`): иначе визуализация чужие данные за сетку. Плюс `pickLiveVar` (приоритет имен). (индексы пользователя вне диапазона демо).

### `verify:code` – `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протрассировка трасса не оборвана по лимиту шагов, шагов больше нуля (или пустой листинг), и \*\*каждая переменная, обещанная страницей кодом\*\* – иначе визуализация физически не может за ним

ФАЙЛ 106/111: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

```
/**
 * DOM-проверки приложения: jsdom + настоящий Pyodide и
 *
 * npm run verify:dom
 *
 * Фильтры:
 * HAR_CHAPTERS=floyd,dijkstra только эти страницы
 * HAR_SKIP=A,E пропустить группы (A
 * С матрицы, D графы,
 * F редактор, G lite,
 * HAR_SYNC=floyd,mst#prim только эти записи эт
 *
 * Оркестратор: грузит CPython ДО создания jsdom (иначе
 * в браузере, и лезет за wasm на CDN), кладёт runtime
 * window.__algv0PythonRuntime и собирает реальные комп
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "../dom-env.mjs";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
const jsdomErrors = [];

log("гружаю CPython (pyodide из node_modules)...");
const py = await loadPyodide({ indexURL: "node_modules/"
log(` Python готов за ${Date.now() - t0} мс`);

log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
window.__algv0PythonRuntime = py;
```

```

console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(` ${group.padEnd(26)} ok: ${String(g.ok)}
}
console.log(`\n ВСЕГО ${sum.total}: прошло ${sum.passed}
if (failed.length) {
 console.log("\nПРОВАЛЫ:");
 for (const f of failed) console.log(` x [${f.group}]
}
if (jsdomErrors.length) {
 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченные
 for (const e of [...new Set(jsdomErrors)].slice(0, 10)
}
const stray = consoleErrors.filter((e) => !/not wrapped
if (stray.length) {
 console.log(`\nCONSOLE.ERROR вне тестов (${stray.length}
 for (const e of [...new Set(stray)].slice(0, 10)) console
}
console.log(`\nотчёт: ${OUT_DIR}/report.json | время:
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?

function log(message) {
 process.stdout.write(`[verify:dom] ${message}\n`);
}

```

ФАЙЛ 107/111: scripts/verify/run-probe.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 45 | РАЗМЕР: 1.1 КБ

```

/**
 * Зонд панели Python: печатает состояние редактора/терминала
 * Не тест, а диагностика — когда нужно понять, что панель
 * ввода кода (автозапуск, переход в листинг, сообщения)
 *
 * npm run verify:probe
 */

```

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 108/111: .github/workflows/deploy-pages.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

```
name: Deploy app to GitHub Pages
```

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
(у бота-интеграции нет admin-прав на них – API отвечает)
#
1. Settings → Pages → Build and deployment → Source
Если там «Deploy from a branch» (legacy) – шаг D
2. Settings → Environments → github-pages → Deployment
добавьте паттерн «arena/**» и ветку «main» (или main/*)
иначе деплой из новой рабочей ветки отклоняется
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 - "arena/**"
```

```
 workflow_dispatch:
```

```
permissions:
```

```
 contents: read
```

```
 pages: write
```

```
 id-token: write
```

```
concurrency:
```

```
 group: pages
```

```
 cancel-in-progress: true
```

```
Ранняя и понятная ошибка вместо невнятного «Fail»
когда в репозитории Pages не переключён на сбор
- name: Preflight – Pages собирается через GitHub
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages/build_type")
 echo "Pages build_type: $bt"
 if ["$bt" != "workflow"]; then
 echo "::error title=Pages не настроен::Setте этот запуск. См. комментарий в начале деплоя"
 exit 1
 fi

- name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

---

ФАЙЛ 109/111: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРOK: 155 | РАЗМЕР: 6.1 КБ

---

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отдаёт GitHub)
промежуточные PNG сохраняются как artifacts запуска.
```

on:

- ```
node-version: "24"
cache: npm
```
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\(*\)\s*/rights="none" pattern="\1"/g' \$policy
 fi
 done
 convert -version
 - name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.density:-300}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Step 4 – собрать приложение и поднять preview
run: |
 # headless-браузер для снимков: chrome ставит
 # этот вызов – страховка, если кэш оказался пустым
 npx puppeteer browsers install chrome || true
 npm run build
 mkdir -p tmp
 nohup npx vite preview --host 127.0.0.1 --port 3000 &


```
    name: full-code-export
    path: |
      public/export/full_code_all_pages.pdf
      public/export/full_code_all_pages.txt
    if-no-files-found: error
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.t/viz-shots
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
```

ФАЙЛ 110/111: .github/workflows/verify.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3 КБ

name: Verify

Проверки приложения на каждом пуше и PR:

#

```
# тянет Chromium – пропускаем, чтобы установка не
- name: Зависимости
  run: npm ci
  env:
    PUPPETEER_SKIP_DOWNLOAD: 1
    PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

- name: Типы и сборка
  run: |
    npx tsc --noEmit -p tsconfig.json
    npm run build

- name: Модульные проверки (liveMatrix)
  run: npm run verify:unit

- name: Эталонные коды страниц (настоящий CPython)
  run: npm run verify:code

- name: DOM-проверки приложения (jsdom)
  run: npm run verify:dom

- name: Отчёт проверок
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: verify-report
    path: tmp/verify/report.json
    if-no-files-found: ignore
```

РАЗДЕЛ: ПРОЧЕЕ

- `content` – шаблонная строка. Внутри **никаких** ` ` экранирования: это сломает сборку.

2. Каркас страницы

```
```html
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <!-- шапка: бейдж билета + заголовок -->
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <!-- смысловой блок (один или несколько) -->
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-emerald-400 mb-4">
 ...определение, абзацы, стат-сетка, аналогии, спор
 </div>
 </div>
 </div>
</section>
```
```

Цвет левой полосы и заголовка блока – семантика раздела

| Полоса / цвет | Смысл раздела |
|---|-----------------|
| --- | --- |
| `border-blue-500` + `text-blue-400` | определение, «о |
| `border-emerald-500` + `text-emerald-400` | алгоритм, |
| `border-indigo-500` + `text-indigo-400` | теория, док |
| `border-amber-500` + `text-amber-400` | ловушки, част |
| `border-rose-500` + `text-rose-400` | ограничения, «к |

3. Библиотека блоков (точные классы)

Слева – неформальная аналогия, справа – техническая аль

```
```html
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border border-
 <div class="absolute -top-3 left-4 bg-slate-700 tex
 ald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>
```

```

 <div class="bg-slate-900 p-6 rounded-lg border-2 bord
 <div class="absolute -top-3 left-4 bg-rose-900 text
 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>
</div>
```

```
```
```

Второй карточкой может быть «Аналогия 2», «Ограничения»
«Когда не работает» – маккуп тот же (пунктирная rose-ра

3.5 Инфо-блок с таблицей

```
```html
```

```
<div class="bg-slate-800/70 p-5 rounded-lg border borde
 <p class="text-sm font-bold text-white mb-2"> Заголо
 <p class="text-slate-300 text-sm mb-3">Пояснение пере
 <table class="w-full text-xs text-left border-collaps
 <thead>
 <tr class="text-slate-500 border-b border-slate-7
 <th class="py-1.5 pr-2 font-bold">Колонка</th>
 <th class="py-1.5 pr-2 font-bold">Колонка</th>
 </tr>
```

```

 Переполнение: <span class="font-mono text-emera

</div>
```
```

4. Порядок разделов на странице

1. Шапка: бейдж билета + `h2`.
2. «Одной фразой» / «Суть алгоритма» + центрированное о
3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чи
4. Аналогии (3.4) – обязательный блок.
5. Механика: пошаговое описание, таблица состояний/срав
6. Спойлеры (3.6): референсный код, доказательство, форм
7. Ловушки (3.7) + границы применимости.
8. Связь с другими билетами («тот же find/union – сердце

Демонстрацию вставлять не нужно: `ChapterView` монтирует
по `id` главы после текста.

5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхронизацию

- `src/components/vizRegistry.tsx` – запись с ключом `\*\*`
(`title`, опциональный `hint`, `Component`).
- `src/data/vizSync.ts` – запись `PAGE\_SYNC[id]` или `P`
где `demoId` – второй аргумент `emitVizDemo(demoId, t`
(вкладки одной демонстрации).
- Поля записи: `vizTitle`, `stepNote` (что считается од
`variables: [{ name, role, range }]`, `code` (референ
- Переменные, которые читает визуализация, должны быть
(`int`, `list`, `dict`, `str`): экземпляры классов сн
до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – `buildInit`